

ACM 算法大全—By 月雪

ACM 算法大全—By 月雪

零. 快速确认模糊方法

一. 基础

1. 一些定义

2. 寻找中位数 (中位数相关问题)

2.1 Method 1

2.2 Method 2

2.2.1 Code

3. 贪心策略

4. 四舍五入的办法

4.1 使用 *Round* 函数

4.2 注意事项

5. 双指针 VS 二分

5.1 Example

6. 并查集(DSU)

普通并查集(常规题够用)

启发式合并·并查集

特殊操作

需要DSU内计数的问题

树上DSU

搜寻最近的未使用坐标

7. 基础操作

7.1 输入输出加速

7.2 开启 c++14

7.3 *sort*改降序

7.4 `__int128`使用方法

7.5 快读

8. 负数进制转换方法

9. 三角公式

1. 求极角

2. 反三角函数

10. 三分

10.1 整点三分

Code (先增后减模板)

浮点三分

Code

黄金分割点优化 (常数/2优化)

11. 浮点数相关 (浮点二分注意事项)

12. 前缀和 (二维+三维)

12.1 二维前缀和

12.1.5 二维差分前缀

12.2 三维前缀和

13 对拍(未完成)

13.1 随机数据生成

14. 知道(前/后)序和中序, 还原二叉树

二. *trick*(一些简单的技巧)

1. 衷心建议 (踩过的坑)

2. *trick*

3. 2-sat(并查集/*tarjan*, m 对 $bool$ 关系)

总思路

3.1 情况1

经典思路

可行性判断

一组可行解

3.2 情况2

可行性判断

答案数计

例题1 (伪2 - *sat*求答案数)

4. n 个取 k 个最快时间方法

Code

5. 快速判断树上节点 u 是否为 v 的祖先

Solution

Proof

Code

6. 判断任意子集求和构成的答案集合

方法1

方法2

7. 构造排列 a , 有 $(\prod_{i=1}^x a_i) \% n$ 在 x 不同时互不相同

7.1 构造条件

7.2 构造方法

8. 向上取整

三. *STL*

1. *BitSet*

可替代场景:

具体用法:

2. *complex*

用法

3. *map*

3Plus. *unordered_map*

4. *multiset*

5. *priority_queue*

自定义比较器

6. *set*

用法:

7. *vector*

定义

8. *deque*双端队列

四. *DP*

1. 一般 *DP* 常用套路

2. 状压*DP*

2.1 情况1(99%用这种, 无脑用就行, 除非wa了)

2.2 情况2(一般不考虑这种)

例题

CF : Prime Gaming

Solution

关键代码

3. 树形*DP*

例题

1. 有线电视网

4. 其他常见*DP*套路

4.1 网格*DP*

4.1.1 求路径方案

4.1.2 网格有障碍物

Method 1

Method 2

五. 数据结构

1. 线段树

1.1 基础线段树

1.1.1 引入

1.1.(1.5)

1.1.2 基本结构&建树

1.1.3 查询

1.1.4 区间修改 (懒标记)

1.1.5 例题

1.1.5.1 Code

1.2 线段树的动态开点

1.2.1 Code

1.3 线段树合并

1.3.1 Code

1.3.2 线段树合并例题

1.3.2.1 solution

1.3.3 Code

1.4 线段树例题

P12846 [蓝桥杯 2025 国 A] 翻转硬币

P1502 窗口的星星

Solution

Code

P2824 [HEOI2016/TJOI2016] 排序

Solution

Code

2. 笛卡尔树

建树方法(以小根堆为例)

3. 树状数组

3.1 基本结构

3.2 区间内不同元素个数

Code

3.3 逆序对个数

3.4 维护差分序列

4. 树链剖分

4.1 剖分

4.2 修改+查询

4.3 Code

5. 主席树(可持久化线段树, 区间第k大)

6. 平衡树(第k大)

7. 单调栈

六. 数学相关

1. 一些常用结论

2. 线性基(异或相关问题, $GF(2)$ 和 $GF(3)$)

2.1 构造方法

简单版本

记录每个线性基来自于原数组哪几个 a_i 异或而来的版本

2.2 线性基合并

2.3 常用方法

2.3.-1 查找是否能 $XOR = k$

2.3.0 高斯消元

2.3.1 查询原数组子集的最大异或和

2.3.2 查询原数组子集的最小异或和

2.3.3 寻找第 k 小值和第 k 大值 (从小到大第 k 个)

Code

2.3.4 求 XOR 和为0的子集个数

2.3.5 图上最大异或和

Code

2.3.6 树上最大异或和

2.4 GF(3)意义下的异或线性基 (三进制)

3. 求质因数

3.1 单个数求质因数

Code

3.2 多个数求质因数

Code

4. 线性筛&埃氏筛

4.1 线性筛 (欧拉筛)

4.2 埃氏筛

5. 欧拉函数&欧拉降幂

5.1 欧拉函数

5.2 前置: 欧拉定理

5.3 欧拉降幂

5.4 例题

6. 逆元的多种求法

6.1 Method 1

6.2 Method 2

6.3 Method 3

6.4 Method 4

6.5 Method 5

6.6 进阶版: $O(n)$ 求 n 个数的逆元

7. 快速乘&快速幂

7.1 快速乘

7.1.1 用途

7.1.2 代码

7.2快速幂

7.2.1 用途

7.2.2 代码

8. 矩阵快速幂 (矩阵加速)

8.1 Code

8.2 特殊矩阵快速幂

9. 矩阵的积和式 (二分图完美匹配方案数)

9.1 二分图完美匹配

9.2 求解方法

Code(在 $mod = 1e9 + 7$ 的意义下)

其他适用情况 (实际例子)

10. 超高效求质因数: Pollard-Rho 算法

10.1 Code

11. 多项式乘法 FFT

11.1 Solution

11.2 Code

12. 多项式乘法NTT

12.1 Solution

12.2 Code

13. 多项式乘法逆

13.1 Solution

13.2 Code

14. 分治FFT/NTT

14.1 Solution-1

Code

14.2 Solution-2

伪代码:

15. 任意模数多项式乘法(高精度 FFT)

15.1 Solution

15.2 Code

16. 多项式中的一些概念

17. 原根

17.1 求解方法

17.2 Code

18. 拉格朗日插值 (多项式求值)

Solution

特殊优化 $O(n)$

快速拉格朗日插值·一般优化 $O(n\log^2 n)$

Code

19. 二维凸包

方法1. 模拟法

方法2. *Graham*算法

Code

20. 向量叉积

表示含义

21. 扩展欧几里得 *Exgcd*

21.1 基本代码

21.2 通解

21.3 最小非负整数解

七. 图论

1. 网络流

1.1 网络流-最大流

1.1.1 使用条件

1.1.2 Solution

Edmonds-Karp算法 (不推荐)

Code

1.1.3 Dinic算法 (下面还有更优解)

Code

1.1.4 *HLPP* 预流推进算法

Code

2. 分层图(注意事项)

3. LCA的求法

3.1 预处理

3.2 求LCA

4. 图论·一些定义

4.1 重链

5. 最小生成树

5.1 *Kruskal*算法

5.2 *Prim*算法

6. 简单环&最小环

6.1 简单环求法

Code

6.2 最小环求法

Code

7. 二分图

7.1 二分图最大匹配 (最小顶点覆盖)

8. 树上差分

八. 字符串

1. Manacher-马拉车算法 (回文字符串)

Code:

Code

2. 双模数哈希(降低冲突率)

具体做法

3. *KMP*

4. *Trie*

5. *AC*自动机

5.1 *AC*自动机的优化

5.3 优化版本各种求法

5.3.1 每个模式串在文本串出现次数

Code

5.3.2 无文本串，每个模式串在所有模式串出现次数

Code

6. 回文自动机 (PAM)

九. 计算几何

1. 扫描线

1.1 注意!

1.2 Code

十. 杂项

1. 莫队

1.1 莫队阅读须知

1.2 莫队(基础版)

1.2.1 解法

1.2.2 Code

1.2.3 Solution & Code

1.3 回滚莫队

例题：回滚莫队&不删除莫队

Code

例题2：历史研究

Code

1.4 带修莫队(带修改)

1.4.1 New Code

例题：数颜色 / 维护队列

Code

1.5 莫队·例题

1.5.1 大爷的字符串题

Code

1.5.2 小清新人渣的本愿

Solution

Code

2. 博弈论

总结1:

十一. 专题合集

专题1：线段+区间类问题

专题2：括号序列·相关问题

前置知识 (通用)

专题3：DP相关问题

树形DP

十二. 进阶

12.1 异或相关

12.1.1 线性基

12.1.2 异或和方程

零. 快速确认模糊方法

n 对关系 $\rightarrow$ 2-sat

树 $\rightarrow$ 图论/树形dp/DSU On Tree

字符串 $\rightarrow$ 字符串相关

n 很小 $\rightarrow$ 状压

n^2 能过 $\rightarrow O(n^2)$ dp/搜索

$n \in [1e5, 5e5] \rightarrow O(N \log N)$ (暴力(dp/搜索)+数据结构优化(st表/线段树/树状数组)/图论)/
 $O(n\sqrt{n})$ 相关算法(分块/莫队/sqrt(n)-trick套路题)

$n > 1e6 \rightarrow$ 线性算法(dfs+优化/bfs)/卡常+ $O(N \log N)$ 算法(见上)

一. 基础

1. 一些定义

1. 子数组 (subarray) : 连续的一段
 2. 子串: 字符串连续的一段
 3. 子序列 (subsequence) : 不用连续的一段
-

2. 寻找中位数 (中位数相关问题)

寻找中位数有两种方法, 两种方法各有用处

2.1 Method 1

常用于数据较多是二分找中位数

用二分, $\geq mid$ 的看成 1, $< mid$ 的看成 -1, 区间和 > 0 代表区间中位数 $\geq mid$

时间复杂度 $O(N\log N)$

2.2 Method 2

用 *multiset*。一个储存较小的一半, 一个储存较大的一半

保证较小的一半数量减去较大的一半数量 ≤ 1

2.2.1 Code

```
1 void Balance() { //保证 较小-较大  $\leq 1$ 
2     int s1 = ms1.size(), s2 = ms2.size();
3     if(s1 - s2  $\geq 2$ ) {
4         auto it = ms1.end();
5         it--;
6         int now1 = (*it);
7         ms2.insert(now1);
8         ms1.erase(it);
9     }
10    else if(s2 > s1) {
11        auto it = ms2.begin();
12        int now2 = (*it);
13        ms1.insert(now2);
14        ms2.erase(it);
15    }
16 }
17 for(int i = 1; i  $\leq$  n; i++) {
18     if(ms1.size() == 0) {
19         ms1.insert(a[i]);
20     }
21     else {
22         auto it = ms1.rbegin();
23         if(a[i]  $\leq$  (*it)) {
24             ms1.insert(a[i]);
25         }
26         else {
27             ms2.insert(a[i]);
28         }
29     }
30     Balance();
31 }
```

C++

注意, 此方法时间复杂度常数可能较大。时间复杂度 $O(N\log N)$

此方法还能维护数据，比如能满足删除元素并快速重新找到新的中位数

只需要判断需要删除的数和中位数的大小就能知道从哪个 *multiset* 删数据

3. 贪心策略

1. 对于两个数的贪心，常见如下：

```
1 按照a+b, a-b, a*b, a/b 排序
```

C++

4. 四舍五入的办法

| 4.1 使用 *Round* 函数

对 x 精确到第 L 位：

```
1 double x;
2 int L, p = 1;
3 cin >> x >> L; //L意义如上
4 for(int i = 1; i ≤ L; i++) p *= 10;
5 x = round(x * p) / p;
6 //如果只需要输出，那么：cout << fixed << setprecision(L) << x;即可
```

C++

| 4.2 注意事项

p 不能用 *pow*，会精度不够

5. 双指针 VS 二分

如果发现二分会 *TLE*，那么首先考虑 *check* 是否有问题，如果没问题就考虑能否用双指针

当「答案区间」或「指针移动方向」具有单调性（不回退）时，优先考虑双指针；

| 5.1 Example

1. 计算最长/最短长度
 2. 满足某种条件的区间个数
-

6. 并查集(DSU)

普通并查集(常规题够用)

直接上操作

```
1 void GetFa(int x) {return x == fa[x]?x:fa[x] = GetFa(fa[x]);}
2 for(int i = 1; i ≤ n; i++) fa[i] = i, num[i] = 1; // num: 并查集内节点数量
3 // 合并
4 int fau = GetFa(u), fav = GetFa(v);
5 if(fau ≠ fav) {
6     fa[fau] = fav;
7     num[fav] += num[fau];
8 }
```

C++

注意：最后求每个并查集的根节点之前需要再 $GetFa$ 一次，因为可能并没有完全更新完成

```
1 for(int i = 1; i ≤ n; i++) {
2     int Fa = GetFa(i);
3     cout << Fa << " is " << i << "'s father\n";
4 }
5 // 不能直接输出fa[i]!!!!
```

C++

例题: [P4185 \[USACO18JAN\] MooTube G](#)

启发式合并·并查集

假设你需要保存每个并查集有哪些点，那每次合并时间复杂度就会多一维，显然不行

但是如果我们每次选择 cnt 较小的那个合并到较大的那个，那么每一次合并大小都会翻倍

所以总时间复杂度不超过 $O(n \log n)$

注意： DSU 中常常会用到各种 STL ，一般用 $vector$, map , $multiset$ ，一定要判断清楚用哪个时间复杂度能承受！

$vector$ ：只记录编号或者大小，不能修改，但是时间复杂度低

map ：可以当桶用，常用来记录出现多少次，可以修改，但是时间复杂度 $\log$ 非常高

$multiset$ ：记录编号或大小，能修改，读取时间复杂度平均 $O(1)$ ，修改时间复杂度 $\log$

所以一般都是混用，需要修改对时间复杂度要求不高的用 map

处于启发式合并核心算法的只能用 $vector$ 和 $multiset$ 来遍历

```
1 vector<int> dsu[N]; // 并查集内id的集合
2 multiset<int> val[N]; // 并查集内点权值的集合，用multiset更好修改，如果不需要修改就用vector即可
```



```

3  map<int, int> mp[N]; // 记录并查集内某个权值出现的次数
4  for(int i = 1; i ≤ n; i++) {
5      a[i] = read();
6      fa[i] = i;
7      cnt[i] = 1;
8      dsu[i].push_back(i); // 每个并查集的点集
9      val[i].insert(a[i]); //
10 }
11 // 合并操作
12 int fau = getfa(u), fav = getfa(v);
13 if(fau == fav) continue;
14 if(cnt[fau] > cnt[fav]) swap(fau, fav); // 保证较小的为fau
15
16 for(auto u: dsu[fav]) or for(auto u: val[fav]) // 启发式合并核心算法, 遍历小并查集内元素
17 // 这都是可行的, 具体看需要哪个
18
19 for(auto u: val[fav]) val[fav].insert(u); // multiset的插入方法
20 dsu[fav].insert(dsu[fav].end(), dsu[fau].begin(), dsu[fau].end()); // 用vector的直接插入法
21
22 cnt[fav] += cnt[fau];
23 fa[fav] = fav; // 基本操作
24
25 // 如果需要高效单个查询, 可以改用set进行
26 // s[fav].insert(s[fav].begin(), s[fau].end()); // set合并方法

```

C++

特殊操作

如果还需要遍历某个并查集内, 每个点连接的点, 比如如下所示

```

1  int fau = getfa(u), fav = getfa(v);
2  for(auto u: dsu[fau]) { // 遍历并查集内每个点
3      for(int i = fi[u]; i; i = ne[i]) { // 遍历这个点连接的边
4          int v = to[i];
5          // 一系列操作 ...
6      }
7  }

```

C++

这样总时间复杂度是 $O(\sum |A| * deg(A) = Ave(deg) * \sum |A| = Ave(deg) * n \log n)$, 即平均度数*总和

平均度数为: $Ave = m/n$, 所以平均时间复杂度为 $O(m \log n)$, 完全可行 (实际上通过数学证明可以证得时间复杂度确实为 $O(m \log n)$, 而并不是平均时间复杂度)

需要DSU内计数的问题

如果不仅需要并查集内有哪些点, 还需要每个点的权值, 可以用到map套vector

```

1 map<int, vector<int>> mp[N];
2 //mp[u][x]代表以u为根节点的并查集内，其中权值为x的节点编号有哪些
3 //注意，auto x: mp[u], x相当于一个pair<int, vector<int>>
4
5 map<int, int> mp[N];
6 //mp[u][x]代表u为根节点的并查集内，权值为x的点出现过多少次

```

C++

树上DSU

对于树上的DSU，从叶子节点往上合并，肯定需要用启发式合并，时间复杂度为 $O(n \log n)$ ，不证明了

搜寻最近的未使用坐标

假如你有一个数组，你已经使用了某些数，这些已经使用的不能再使用

那么对于 a_i ，要么它未被使用，你能使用它，要么你只能使用它右边最近的未使用的

如何加速搜寻过程？

显然用并查集。

初始状态令 $f_{a_i} = i$ ，假如使用了 i ，那么 $merge(i, i + 1)$ ，这样就能自动把 i 和最右侧未使用的坐标连起来

7. 基础操作

7.1 输入输出加速

```

1 std::ios::sync_with_stdio(false);
2 std::cin.tie(nullptr);
3 //不能使用endl
4 //不能使用快读和快输

```

C++

7.2 开启 c++14

工具 → 编译选项 → 编译时加入以下命令中添加：

```
1 -std=c++14
```

C++

7.3 sort改降序

```
1 sort(a + 1, a + n + 1, greater<int>());
```

C++

7.4 `__int128`使用方法

输入用快读，输出用快输

```
1  __int128 read() {
2      __int128 x = 0;
3      int f = 1;
4      char ch = getchar();
5      while(ch < '0' || ch > '9') {
6          if(ch == '-') f = -1;
7          ch = getchar();
8      }
9      while(ch ≥ '0' && ch ≤ '9') {
10         x = x * 10 + ch - '0';
11         ch = getchar();
12     }
13     return x * f;
14 }
15 void print(__int128 x) {
16     if (x < 0) {
17         putchar('-');
18         print(-x);
19         return;
20     }
21     if (x ≥ 10) print(x / 10);
22     putchar(x % 10 + '0');
23 }
```

C++

7.5 快读

```
1  int read() {
2      int x = 0, f = 1; char ch = getchar_unlocked();
3      while(ch < '0' || ch > '9') {
4          if(ch == '-') f = -1;
5          ch = getchar_unlocked();
6      }
7      while(ch ≥ '0' && ch ≤ '9') {
8          x = x * 10 + ch - 48;
9          ch = getchar_unlocked();
10     }
11     return x * f;
12 } //getchar_unlocked比getchar快了1/3
```

C++

8. 负数进制转换方法

```
1 while(true) {
2     int x = n % p;
3     if(x < 0) {
4         x -= p;
5         n += p;
6     }
7     ans[++cnt] = x;
8     n /= p;
9     if(!n) break;
10 }
```

C++

9. 三角公式

1. 求极角

极角： xy 坐标系中一点 $P(x, y)$, $\vec{OP}$ 与 x 正半轴的夹角

```
1 double r = atan2(y, x); // 先y后x
2 // 范围 $[-\pi, \pi]$ 
```

C++

2. 反三角函数

C++中用的弧度制

```
1 // 求 $\pi$ 
2 long double pi = acosl(-1.0); // acos()是double
```

C++

10. 三分

二分是枚举答案，用合法性来判断

三分是求最值，不能用合法性判断，只能和值比较

两者都需要满足答案在区间内具有单峰性

10.1 整点三分

Code (先增后减模板)

```
1 int calc(int x) {
```

```

2     return 100 - (x - 37) * (x - 37);
3 }
4 void solve() {
5     int l = 1, r = 100;
6     while(r - l > 2) { // 整点三分最后会剩1~3个点
7         int lmid = l + (r - l) / 3;
8         int rmid = r - (r - l) / 3;
9         if(calc(lmid) > calc(rmid)) r = rmid;
10        else l = lmid;
11    }
12    // 整点三分最后会剩1~3个点
13    int ans = -INF;
14    for(int i = l; i ≤ r; i++) {
15        ans = max(ans, calc(i));
16    }
17    cout << ans << endl;
18 }

```

C++

浮点三分

Code

```

1 void solve() {
2     double l = , r = ;
3     double eps = 1e-9;
4     while(r - l > eps) {
5         double lmid = l + (r - l) / 3;
6         double rmid = r - (r - l) / 3;
7         if(calc(lmid) > calc(rmid)) r = rmid;
8         else l = lmid;
9     }
10    printf("%.8lf\n", calc(l)); // 无需遍历
11 }

```

C++

黄金分割点优化（常数/2优化）

```

1 const double phi = (sqrt(5) - 1) / 2; // 0.6180339887, 黄金分割点
2 const double eps = 1e-9;
3 void solve() {
4     int l = , r = ;
5     double lmid = phi * l + (1.0 - phi) * r;
6     double rmid = (1.0 - phi) * l + phi * r;
7     double lval = calc(lmid);
8     double rval = calc(rmid);
9     while(r - l > eps) {
10        if(lval > rval) {
11            r = rmid;
12            rmid = lmid;
13            rval = lval;
14            lmid = phi * l + (1.0 - phi) * r;
15            lval = calc(lmid);
16        }
17        else {

```

```

18         l = lmid;
19         lmid = rmid;
20         lval = rval;
21         rmid = (1.0 - phi) * l + phi * r;
22         rval = calc(rmid);
23     }
24 }
25 l = (l + r) / 2.0;
26 printf("%.8lf\n", calc(l));
27 }

```

C++

11. 浮点数相关（浮点二分注意事项）

$double$ 精度为 $1e-15$, $long double$ 精度为 $1e-19$

需要注意！！

如果二分的时候浮点数很大，需要**限制二分次数**！！

```

while(r-l>eps){
    if(num > Max) break;
}

```

12. 前缀和（二维+三维）

先令 $sum_{i,j} = f_{i,j}$

12.1 二维前缀和

```

1 sum[i][j] = f[i][j]
2 sum[i][j] = sum[i-1][j] + sum[i][j-1] - sum[i-1][j-1]

```

C++

12.1.5 二维差分前缀

给 n 个矩形，判断每个格子被多少矩形覆盖

```

1 //每个矩形左上角(l1,r1), 右下角(l2,r2)
2 dif[l1][r1]++;
3 dif[l2 + 1][r2 + 1]++;
4 dif[l1][r2 + 1]--;
5 dif[l2 + 1][r1]--;
6 // 上述为差分过程
7
8 dif[i][j] = dif[i][j] + dif[i - 1][j] + dif[i][j - 1] - dif[i - 1][j - 1];
9 //如此dif[i][j]就为单点值, 即(i,j)被多少格子覆盖

```

C++

12.2 三维前缀和

```

1 sum[i][j] = f[i][j]
2 sum[i][j][k] = sum[i - 1][j][k] + sum[i][j - 1][k] + sum[i][j][k - 1] - sum[i - 1][j - 1][k]
3               - sum[i - 1][j][k - 1] - sum[i][j - 1][k - 1] + sum[i - 1][j - 1][k - 1]
4               + sum[i][j][k];

```

C++

(两次容斥)

13 对拍(未完成)

13.1 随机数据生成

简单写法(对于 *int* 内的数据生成有效):

```

1 #include<bits/stdc++.h>
2 long long RandLL(long long L, long long R) {
3     return (rand() * rand()) % (R - L + 1) + L;
4 }
5 int main() {
6     srand(time(0)); //注意, srand只能放全局!!
7     std::cout << RandLL(-100, 23333);
8 }

```

C++

*long long*类型:

```

1  #include<bits/stdc++.h>
2  #define ll long long
3  std::mt19937_64
   rng(std::chrono::steady_clock::now().time_since_epoch().count());
4  ll RandLL(ll L, ll R) {
5      return std::uniform_int_distribution<ll>(L, R)(rng);
6  }
7  int main() {
8      std::cout << RandLL(-100, 333333333333);
9      return 0;
10 }

```

C++

*double*类型:

注意, *double*类型范围是 $[L, R)$

```

1  #include<bits/stdc++.h>
2  double RandDouble(double L, double R) {
3      static std::mt19937_64 rng([]{
4          auto seed = chrono::steady_clock::now().time_since_epoch().count();
5          std::random_device rd;
6          seed ^= rd() + 0x9e3779b97f4a7c15ULL + (seed<<6) + (seed>>2);
7          return seed;
8      }());
9      return uniform_real_distribution<double>(L, R)(rng);
10 }
11 int main() {
12     cout << RandDouble(-12.3, 9.1);
13     return 0;
14 }

```

C++

14. 知道(前/后)序和中序, 还原二叉树

关键发现: 前序的第一个/后序的最后一个是当前子树的根

所以每次判断哪些区间属于当前子树的左子树, 哪些区间是右子树即可

前序: 根节点→左子树→右子树

中序: 左子树→根节点→右子树

后序: 左子树→右子树→根节点

```

1  int get_tree(int l1, int r1, int l2, int r2) { // 当前子树, a中[l1,r1]代表当前子树中
   序, b中[l2,r2]代表当前子树前/后序
2      if(l1 > r1 || l2 > r2) return -1; // 不合法, 返回-1
3      int root = b[r2], x = pos[root]; // root: 当前子树的根, x: 当前root在a中的位置
4      int y = (x - l1) + l2 - 1; // (x-l1)意思是, 左子树大小, 这样就能快速求出b中左右子树分
   界点在哪
5      // b中[l2,y]是这颗子树的左子树区间, [y+1,r2-1]是右子树区间 (注意r2是根节点所以不计入)
6      // 而a中[l1,x-1]是左子树, [x+1,r1]是右子树
7      l[root] = get_tree(l1, x - 1, l2, y);
8      r[root] = get_tree(x + 1, r1, y + 1, r2 - 1);

```



```

9     return root;
10 }
11 int main() {
12     n = read();
13     for(int i = 1; i ≤ n; i++) {
14         a[i] = read();
15         pos[a[i]] = i; //位置
16     }
17     for(int i = 1; i ≤ n; i++) b[i] = read();
18     return 0;
19 }

```

C++

二. *trick*(一些简单的技巧)

1. 衷心建议 (踩过的坑)

1(★★★★★). 一定要考虑清楚!! 不要想当然的做

2(★). 如果遇到 *WA*, 就考虑是不是条件漏了, 或者考虑少了

3(★). 如果是 *TLE*, 那就是算法错了, 或者**数组越界**, 或者死循环了, 或者常数大了 (把 `#define int long long` 关了试试)

4(★★). 可以试着从小情况考虑到大情况

5(★★★). 不要使用 *pow*, 不要使用 *pow*, 不要使用 *pow*!!! 精度非常非常低!!!

6(★★★★★). 如果遇到了交互题, 一定不要用快读, 可能会 *TLE*!

7(★★★★). 如果遇到了觉得做对了但是 *wa* 了, 请检查: 是否有混用变量名, 数组大小, 范围问题等**尤其是开 `__int128` 试试**

8(★★★★). 对于变量名, 最好用一个有区分度的名字

9(★★★★★). 遇到需要取模的题目, 一定要仔细检查每个数的范围, 遇到 $a = x - y$ 的一定改成 $a = (x - y + M)!!!$

10(★★★). 并查集的题, 最后查询的时候不能直接用 $fa[i]$, 要再 *get_father* 一次才是对的

11(★★★★). 不能令 $x = 1e16 + 2$ 之类的, 存不了这么大, 最后 $x = 10000000000000000$, $+2$ 不会被算进去, 因为 $1e16$ 是 *double*, 有误差

12(★★★★★). $1 \ll k$ 中 1 是 *int*, 注意别爆了, 记得写 $1ll \ll k!!!$

13(★★★★★). 快速幂 a^b 记得先把 a 取模!! 因为可能是负数

14(★★★★★). 如果卡常 *TLE*, 把 `define int long long` 去掉再试试

15(★★★★). 如果做不出来, 但是过的人多, 说明思路想复杂了!

1(★★★★★). 一定要考虑清楚!! 不要想当然的做

16(★★★★★). 浮点二分的时候, 如果浮点数很大, 一定要限制二分次数, 不然会TLE!

17(★★★). 函数内定义的时候, 一定要初始化(vector初始化见stl)

18(★★★★★). 快速幂中, 遇到 0^0 需要额外注意, 因为幂0可能实际上不为0, 那应该返回0而不是1s

2. trick

1. 二分的优化, 把 $\geq mid$ 的看成 1, $< mid$ 的看成 0 (或者其他数也行), 最后只需要计算这个 01 序列即可 (或者 $> mid$ 看成 1, $\leq mid$ 看成 0)。即: 遇到二分的时候, 如果无法直接求, 那大概率把 $\geq mid$ 的看成某个数, 再把 $< mid$ 的看成某个数, 这样便能化简

2. 遇到 异或 相关的问题, 80% 和 线性基 有关

3. 遇到 中位数 相关的问题, 99%和二分 有关 (见第一条), $\geq mid$ 看作 1, $< mid$ 看作 -1, 中位数条件: $\sum > 0$ (具体条件具体分析)

4. 经典问题: 把数组变成相同元素, 最优方案为把所有数变为中位数, 如果是遇到 $a_{i+1} - a_i = 1 \rightarrow a_{i+1} - (i+1) = a_i - i$, 即令 $b_i = a_i - i$, 变为把数组 b 变为相同元素

5. 对于一个问题, 如果求的是最大长度, 如果用二分会 TLE, 不妨考虑能否 用双指针

6. 计算答案数的题, 或许可以用猜想法? 猜答案是 C_x^y

7. 对于一个变化的量, 找到不变的东西或许能优化。比如数组 $a_i = b_i + x$ (x 不定), 那么不变量就是差分 $d_i = a_i - a_{i-1} = b_i - b_{i-1}$, 再根据不变量找关系。

同样的, 把答案的式子退出来后, 可以试着补全某一部分让它变为定值, 这样不变的就少一部分。比如 $\sum_1^l a_i + \sum_{l+1}^n b_i = \sum_1^n a_i + \sum_{l+1}^n (b_i - a_i)$

8. 对于数组的操作, 如果与数的个数相关, 那可以考虑拆分为两部分: 一部分 $num_x \leq \sqrt{n}$, 另一部分 $num_x \geq \sqrt{n}$, 不同部分进行不同操作。被称为: *sqrt-trick*

9. 使用并查集时, “删除”很困难, 那就反向思维改成“添加”

10. map被卡log, 就尝试用unordered_map

11. 假设有 $a < b < c$, 那么, $\gcd(a, b, c) = \gcd(a, b - a, c - b)$

3. 2-sat(并查集/tarjan, m 对bool关系)

总思路

如果遇到：给你 m 对关系，那大概率就要用2-sat了。

如果关系是 $A \text{ or } B$ ，那么需要把or转变为and，这样一来就能连边了，一定是单向边

如果关系是 $A \text{ and } B$ ，那么就需要的是双向边

3.1 情况1

假设有 n 个bool型变量 p_i ，有 m 个条件，第 i 个条件要求：要么第 a_i 个数为0要么第 b_i 个数为1，或者要么第 a_i 个数为1要么第 b_i 个数为1等等等等。即有 m 组关系， $(x_i \text{ or } y_i)$ ，满足其中一个就行

求是否有可行解，输出一组可行解

经典思路

首先，2-sat用到的是图论，假设 $1 \sim n$ 号点代表 p_i 为0， $n+1 \sim 2 \cdot n$ 号点代表 p_i 为1

现在如果有一组条件是： $(X = \{p_{a_i} = 1\} \text{ or } Y = \{p_{b_i} = 0\})$ ，注意 X, Y 是逻辑事件，那么需要建立两条边， $(\neg X \rightarrow Y)$ 和 $(\neg Y \rightarrow X)$ ，代表如果 $\neg X$ 的话就必须 Y ，反之亦然

注意，是单向边!!! 所以这种情况只能用tarjan

就如上例子，应该往 $(a_i, \rightarrow b_i)$ 和 $(b_i + n \rightarrow a_i + n)$ 分别建边，即如果满足了 $p_{a_i} = 0$ 那么 p_{b_i} 必须为0；如果 $p_{b_i} = 1$ 那么 p_{a_i} 必须为1

可行性判断

可行性可以用tarjan，如果发现某个 i 和 $i+n$ 在一个连通块，说明有冲突，答案为False，否则一定有解

(如果用tarjan，那么就判断 $col_i = col_{i+n}?$ ， col 是连通块编号，一定在tarjan中按照顺序来)

一组可行解

如果有解，根据如下规则可以构造一组可行解：

假设 $1 \sim n$ 号点代表答案为f1， $n+1 \sim 2n$ 代表答案为f2

如果 $col_i > col_{i+n}$, 那么 $ans_i = f1$, 反之 $ans_i = f2$

注意, 这里的意思是, 判断谁的拓扑序更大, 谁更大就赋值为谁, $1 \sim n$ 号点代表答案为 $f1$, $n+1 \sim 2n$ 代表答案为 $f2$, 谁的 col 更大就赋值为谁

```
1  #include<bits/stdc++.h>
2  #include<cmath>
3  #define ll long long
4  #define int long long
5  using namespace std;
6  const ll N = 4e6 + 10, M = 1e9 + 7, INF = 1e14;
7  // 注意所有东西都要开两倍空间, 因为每个变量存了两次
8  int n, m, f[N], to[N], ne[N], num, dfn[N], scc[N], low[N];
9  int vis[N], st[N], tot, top, cntx;
10 ll read() {
11     ll x = 0, f = 1; char ch = getchar();
12     while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
13     while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
14     return x * f;
15 }
16 void add(int x,int y)
17 {
18     ne[++num]=f[x];
19     f[x]=num;
20     to[num]=y;
21 }
22 void tarjan(int x)
23 {
24     dfn[x]=low[x]=++tot;
25     vis[x]=1;st[++top]=x;
26     for(int i=f[x];i;i=ne[i])
27     {
28         int v=to[i];
29         if(!dfn[v])
30         {
31             tarjan(v);
32             low[x]=min(low[x],low[v]);
33         }
34         else if(vis[v])low[x]=min(low[x],dfn[v]);
35     }
36     int temp;
37     if(low[x]==dfn[x])
38     {
39         ++cntx; //一定要从0开始加, 这样才是拓扑序
40         while((temp=st[top--]))
41         {
42             vis[temp]=0;
43             scc[temp]=cntx; //此处scc为连通块编号
44             if(x==temp)break;
45         }
46     }
47 }
48 void solve() {
49     n = read(), m = read();
50     for(int i = 1; i ≤ m; i++) {
51         int x = read(), f1 = read();
52         int y = read(), f2 = read();
```

```

53     if(f1 && f2) {
54         add(x, y + n);
55         add(y, x + n);
56     }
57     if(f1 && !f2) {
58         add(x, y);
59         add(y + n, x + n);
60     }
61     if(!f1 && f2) {
62         add(x + n, y + n);
63         add(y, x);
64     }
65     if(!f1 && !f2) {
66         add(x + n, y);
67         add(y + n, x);
68     }
69 }
70 for(int i = 1; i ≤ 2 * n; i++) {
71     if(!dfn[i]) tarjan(i);
72 }
73 //注意scc是从0开始的
74 for(int i = 1; i ≤ n; i++) {
75     if(scc[i] == scc[i + n]) {
76         cout << "IMPOSSIBLE";
77         exit(0);
78     }
79 }
80 cout << "POSSIBLE\n";
81 for(int i = 1; i ≤ n; i++) {
82     if(scc[i] > scc[i + n]) cout << 1 << " "; //谁大选谁
83     else cout << 0 << " ";
84 }
85 }
86 signed main() {
87     solve();
88     return 0;
89 }
90 //月雪·薇婷

```

C++

3.2 情况2

有一种伪2-sat问题，同样给你 m 组关系，但是每组关系需要满足： $(x_i \& y_i)$ 或 $(\neg x_i \& \neg y_i)$ （情况1是 $(x_i \text{ or } y_i)$ ）

这样一来，需要的就是双向边，就能用并查集或者tarjan

连边方法： $(x_i - y_i)$ 和 $(\neg x_i - \neg y_i)$

可行性判断

并查集和 $tarjan$ 皆可

答案数计

那么答案数就为 $2^{c/2}$, c 是连通块数量 (显然连通块一定是有对称性的)

例题1 (伪2-sat求答案数)

见 2023ICPC济南 $\rightarrow G$

大致题意:

有一个 01 矩阵, 有多少种翻转方案, 使得选择一些行进行翻转之后每一列至多只有一个 1?

假设对于第 i 行和第 j 行它们在某一列 (或者中心对称的列上) 上都有 1, 如果它们的 1 在同一列, 那么这两行翻转关系应该一致, 即 (i, j) 和 $(i + n, j + n)$ 连边

否则则不一致, 即 $(i, j + n)$ 和 $(i + n, j)$ 连边

最后计算即可

4. n 个取 k 个最快时间方法

在一个 01 串取恰好 k 个为 1, 最快生成时间复杂度为 $O(C_n^k)$

Code

枚举第 t 个 1 在哪个位置即可

同时注意第 t 个最远到 $n - (k - t) + 1$ 这个位置取, 否则取不满

```
1 #include<bits/stdc++.h>
2 #include<cmath>
3 #define ll long long
4 #define int long long
5 using namespace std;
6 const ll N = 2e6 + 10, M = 998244353, INF = 1e18;
7 int n, k;
8 ll read() {
9     ll x = 0, f = 1; char ch = getchar();
10    while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
11    while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
```

```

12     return x * f;
13 }
14 void dfs(int t, int now) {
15     if(t == k) {
16         return;
17     }
18     for(int i = now; i ≤ n - (m - t) + 1; i++) {
19         dfs(t + 1, i + 1);
20     }
21 }
22 signed main() {
23     n = read(), k = read();
24     dfs(0, 1);
25     return 0;
26 }
27 //月雪·薇婷

```

C++

5. 快速判断树上节点 u 是否为 v 的祖先

Solution

先用 dfs 跑一遍整个图，记录每个点第一次访问的时间戳 $in[u]$ 和离开的时间戳 $out[u]$

假如 $in[u] \leq in[v]$ 并且 $out[u] \geq out[v]$ ，那么 u 是 v 的祖先节点（相等的时候 $u = v$ ）

（祖先节点： $u = fa[fa[\dots fa[v]]]$ ）

Proof

假设 u, v 不是祖先关系，那么必定能找到最近公共祖先 s

假设 dfs 的时候先从 s 进入 u 这一支，那么 $in[u] \leq in[v]$ ，但肯定会先结束回溯，所以 $out[u] \leq out[v]$

反之亦然

Code

```

1 int in[N], out[N], tm;
2 void dfs(int u, int fa) {
3     in[u] = ++tm;
4     for(int i = fi[u]; i; i = ne[i]) {
5         int v = to[i];
6         if(v == fa) continue;
7         dfs(v, u);
8     }
9     out[u] = ++tm;

```

```

10 }
11 bool query(int u, int v) {
12     if(in[u] ≤ in[v] && out[u] ≥ out[v]) return true;
13     return false;
14 }

```

C++

6. 判断任意子集求和构成的答案集合

有 n 个数，任选任意个求和，求答案集合的大小

方法1

用 dp , f_i 代表 i 是否能被组合得到

时间复杂度 $O(N * S)$

```

1 f[0] = 1; //边界
2 for(int i = 1; i ≤ n; i++) { //外层 枚举序列
3     for(int j = Max; j ≥ 0; j--) {
4         f[j + a[i]] |= f[j];
5     }
6 }

```

C++

方法2

用 $bitset$ 优化。如果数的总和 $\leq 1e5$ 左右则可以用

bs 代表答案集合，即 $bs_i = 1$ 代表可以组合为 i ，反之不能

时间复杂度 $O(N * S/w)$

```

1 for(int i = 1; i ≤ n; i++) bs |= (bs << a[i]);
2 //bs是当前答案集合，往左移a[i]位则代表+a[i]之后的集合，再OR一下则是总集合

```

C++

7. 构造排列 a ，有 $(\prod_{i=1}^x a_i) \% n$ 在 x 不同时互不相同

7.1 构造条件

n 为质数 (或者 $n = 1$ 或 4)

7.2 构造方法

假设 $(\prod_{i=1}^x a_i) \% n = x$, 那么显然有 $\prod_{i=1}^x a_i \equiv x \pmod n$

那么有: $a_x \cdot \prod_{i=1}^{x-1} a_i \equiv x \pmod n$

可以得到 $a_x \equiv x \cdot (\prod_{i=1}^{x-1} a_i)^{-1} \pmod n$

可以验证发现 a_i 恰为一个排列

8. 向上取整

注意, 除数和被除数异号的时候直接除就行了

```
1 int calc_up(int x, int y) {
2     if(x % y == 0) return x / y;
3     if((x <= 0 && y < 0) || (x >= 0 && y > 0))
4         return x / y + 1;
5     return x / y;
6 }
```

C++

三. STL

1. BitSet

bitset 本质是一个 01 串

但是 可以进行位运算

可替代场景:

记录某个数出现过没有? 同时需要 $a \leq 1e7$

具体用法:

```
1  bitset<N> bs1, bs2;
2  bs1[1] = bs1[100] = bs1[4] = 1;
3  bs2[1] = bs2[5] = 1;
4  bitset<N> b3 = b1 & b2;
5  b3 = b1 | b2; b3 = b1 ^ b2; // 位运算都能进行
6  b3 = b1 >> 1;
7  cout << b3.count() << endl; // 1出现的数量
8  b1.reset(); // 清空(置零)
9
10 b1.set(x); // 把第x位变成1
11 b1.test(x); // 检查第x位是否为1, 0(1)
12 b1.flip(x); // 重要! 相当于b1^(1<<x), 或者翻转某一位, 0(1)
13
```

C++

位运算时间复杂度: $O(N/w)$, 其中 $w = 64$ (对于 64 位电脑)

2. *complex*

应该只有在多项式里面才会用到?

用法

```
1  complex<double> a{0,1}; // 定义
2  complex<double> b[N]; // 定义
3  a.real(); // 实数
4  a.imag(); // 虚数系数
5  double pi = acos(-1.0); // 代表π
6
7  // 输入实部的方法:
8  cin >> a;
9  // 或者
10 double x; cin >> x; a.real(x);
11
12 // 输入虚部只有一种方法
13 double x; cin >> x; a.imag(x);
```

C++

3. *map*

map 自动按照第一维进行排序

unordered_map 整体更优, 如果 *map* 被卡, 只能尝试 *unordered_map*

```
1  map<int, int> mp;
```

```

2  mp[2] = 100;
3  mp[5] = 103;
4  mp[3] = 109;
5  auto it = lower_bound(4);
6  //查询的*it=103
7  //即map的lower_bound/upper_bound按照的是第一个值查询，因为第一个值是有序的
8  for(const auto& x: mp) {
9      cout << x.first << " " << x.second << endl; //查询方法
10 }
11
12 //map套set
13 map<int, vector<int>>> mp[N];
14 //mp[u][x]代表以u为根节点的并查集内，其中权值为x的节点编号有哪些
15 //注意，auto x: mp[u], x相当于一个pair<int, vector<int>>

```

C++

3 Plus. unordered_map

整体来说，*unordered_map* 优于 *map*

```

1  unordered_map<int, int> ump;
2  //ump的平均时间复杂度是O(1)，最坏会被卡到O(n)
3  //如果map被卡，就只能试着用unordered_map
4  //注意，unordered_map不能用lower_bound

```

C++

4. multiset

set 升级版，能重复储存

```

1  multiset<int> ms;
2  ms.insert(x);
3  ms.clear();
4  auto it = ms.find(x); // 第一个为x的地址
5  ms.erase(x); // 擦除**所有**为x的数据
6  ms.erase(it); // 擦除地址为it的数据
7  auto it = ms.upper_bound(x); // 找到第一个大于x的地址
8  ms.count(x); // 为x数据有多少个，时间复杂度为(O(logn+k)，n是大小，k是x的个数)
9  ms.size(); // 总大小(时间复杂度O(1))
10 ms.begin()/ms.end();
11 // 注意，和别的一样，end()是最后一个地址+1
12 // 上述所有操作时间复杂度 ≤ log(n)，除count
13 multiset<int> ms2(ms); // 拷贝构造
14 swap(ms2, ms); // 交换

```

C++

注意，*erase(int)* 会把所有等于 *int* 的都删掉，而 *erase(it)* 删除地址则只会删除一个

要查询最后一个地址：

```
1 auto it = ms.end(); it--;
2 cout << (*it);
```

C++

5. *priority_queue*

自定义比较器

```
1 struct Time {
2     int id, time;
3 };
4 struct cmp {
5     bool operator()(const Time& x, const Time& y) {
6         if(x.time == y.time) return x.id > y.id; //time相同，以id从小到大
7         return x.time < y.time; //以time从大到小
8         //priority的比较和一般的是反过来运用的★
9     }
10 };
11 priority_queue<Time, vector<Time>, cmp> q;
12 priority_queue<int, vector<int>, less<int>> q1; //大根堆
13 priority_queue<int, vector<int>, greater<int>> q2; //小根堆
```

C++

6. *set*

set 主要用途：去重+自动排序

存入 *set* 的数据没有重复的，自动去重

并且会按照 从小到大的顺序自动排序 ($\log n$)

用法：

```
1 struct cmp{
2     bool operator()(int x, int y) {
3         return x > y;
4     }
5 }
6 set<int, cmp> s;
7
8 //插入：
9 s.insert(10); //插入
10 s.insert(20); //排序
11 s.insert(10); //去重
12 auto it = s.find(10); //找到10这个元素的指针地址
```

```

13  it++;/it--;//10元素前一个元素/后一个元素
14
15  //删除:
16  s.erase(20);//删除20这个元素
17  s.erase(it);//同时也能通过指针删除元素
18
19  cout << s.size() << endl;//set的大小, 时间复杂度O(1)
20  for(const auto& a : s) cout << a;//普通循环输出
21  for(set<int>::iterator it = s.begin(); it!=s.end(); it++)//迭代器循环
22      cout << *it;
23
24  it = s.lower_bound(x)/s.upper_bound(x);//set也可以用lower/upper, 时间复杂度O(logn)
25  //注意, 不能用lower_bound(s.begin(),s.end(),x);时间复杂度O(n)!!
26  if(it != s.end()) cout << *it;//若存在则可以输出
27
28  set<pair<int, int>> sp;
29
30  s[i].insert(s[j].begin(), s[j].end());//set合并方法

```

C++

自定义比较器形式和 *priority_queue* 的一样

但是 *set* 中 *<* 就代表从小到大, *>* 就代表从大到小

而 *priority_queue* 中的是反过来

7. *vector*

定义

```

1  vector<int> v;//一维vector
2  vector<vector<int>> v;//二维vector

```

C++

vector 一定要定义大小

```

1  v.resize(n + 1, 0);//一维定义大小 记得+1 (也可以用这个就定义二维vector的第一维大小)
2  v.resize(n + 1, vector<int>(m + 1, 0));//二维定义大小
3  v[i].push_back(x);//或者也不用定义大小, 直接pushback
4  //注意!! 分配空间之后相当于直接填充了, 就不能push_back了!

```

C++

定义大小之后就 and 数组一样了

```

1  v.size();
2  //以下操作一定要v[i]不为空才能操作
3  sort(v[i].begin(), v[i].end());//排序
4  v[i].clear();//清空数据
5  v[i].shrink_to_fit();//清除空间
6  v[i].insert(v[i].end(), v[j].begin(), v[j].end());//把v[j]插入v[i]
7  for(const auto& x: v[i]) {
8      cout << x;//查询
9  }

```

```

10 auto it = upper_bound(v.begin(), v.end(), value); //也不一定是v.begin,视情况而定
11
12 binary_search(v.begin(), v.end(), x); //二分查找vector中是否有x这个元素,返回bool
13 vector<vector<vector<int>>>> (
14     n, vector<vector<int>>>(
15         m, vector<int>(k, 0))); //三维定义

```

C++

8. deque双端队列

```

1  std::deque<int> dq;
2  dq.push_back(x);
3  dq.push_front(x);
4  dq.pop_back();
5  dq.pop_front();
6  dq.back();
7  dq.front();
8  //dq是类似于vector,连续指针
9  //同时可以用下标访问
10 cout<<dq[0];
11 std::sort(dq.begin(), dq.end());

```

C++

四. DP

1. 一般 DP 常用套路

忠告：如果发现思路很混乱，那一般都是状态弄错了，记得及时止损

以下我会列出见过的比较常用的套路：

1. f_i 或者 $f_{i,j}$ 表示从假设以 i 结尾 (j) 的最优解 (j 也可以代表体积, 价值等)
2. $f_{i,j}$ 表示 从 $1 \sim i$ 中选取 j 个 的最优解 (j 也可以代表体积, 价值等)
3. $f_{i,j}$ 表示 区间 $[i, j]$ 的最优解
4. **树形 DP**: $f_{i,j}$ 表示从 i 的子树选 j 个的最优解 (或者表示在 i 的叶子结点中选 j 个的最优解)
5. **状压 DP**: f_i 表示状态为 i 时候的最优解
6. $f_{i,j,\dots}$ 代表状态为 $i, j, \dots$ 的时候, 是否满足某种状态 (f 为 $bool$ 变量)
7. 如果 n 很大, 可以考虑矩阵快速幂加速, 参见矩阵快速幂章节

1. $f_{i,j}$ 或者 f_i 表示从假设以 i,j 结尾 ($1 \sim i$ and $1 \sim j$) 的最优解 (i/j 也可以代表体积, 价值等)

注意, 如果转移方程为: $f_i = f_{i-x}$ 之类的, 要从大往小搜, 这样才能保证次序性!!

2. 状压DP

是不是状压DP其实很好分辨

如果碰到某个值 $n \leq 20$ 之类的, 非常小, 90%概率是状压

状压有两种情况

2.1 情况1(99%用这种, 无脑用就行, 除非wa了)

如果发现所有状态都会用到 (2^n 个状态都会用到, 没有不用的), 那么外层**必须枚举状态数**

```
1 for(int i = 0; i < (1 << n); i++) {
2     ....
3 }
4
```

2.2 情况2(一般不考虑这种)

如果发现**下一层状态是由这一层推出来的**, 那么就用队列储存状态, 而且第一次访问的就是最佳答案

```
1 queue<int> q;
2 q.push(initial_mask);
3 dp[initial_mask] = 0; // 初始状态
4
5 while (!q.empty()) {
6     int mask = q.front(); q.pop();
7     for (int i = 0; i < n; ++i) {
8         if (!(mask & (1 << i))) {
9             if(...) q.push(...)
10        }
11    }
12 }
```

例题

CF : Prime Gaming

- n 堆石头，每堆 1 或 2 颗 ($m=2$)，给定 good 下标集合；只能删 good 位置的堆。
- Alice 与 Bob 轮流删一堆，Alice 先手，都最优，剩最后一堆时，其数值为 x ：Alice 要 x 最大，Bob 要 x 最小。

任务： 对 所有 2^n 种初始配置，求 最终 x 之和 ($\text{mod } 1e9+7$)。

Solution

显然要用状压，但是这题用的是 *bool* 形 *dp*。 $f_{0/1,i,mask}$ 代表当前一共还剩 i 堆，当前以 $0 \rightarrow \text{Bob} / 1 \rightarrow \text{Alice}$ 开始选择，初始状态为 $mask$ ，以最优解进行，是否最后剩的一堆为 2，是 $\rightarrow f = 1$ ，否(最后剩 1) $\rightarrow f = 0$ 。 $mask$ 代表当前剩余的 i 堆里面每一堆是几个石头。

转移方程：

$$f_{1,i,mask} = OR_{lmask} f_{0,i-1,lmask}$$

$$f_{0,i,mask} = AND_{lmask} f_{1,i-1,lmask}$$

其中 $lmask$ 是在 $mask$ 的基础上选择某一堆去掉后得到

对于 $\text{Bob}(0)$ 来说，如果在 $f_{1,i-1,lmask}$ 中有为 0 的，说明这样选择能使得最终剩余 1 个，那当前的 f 也就为 0

对于 $\text{Alice}(1)$ 来说，如果在 $f_{0,i-1,lmask}$ 中有 1 的，说明这样选择能使得最终剩余 2 个，那当前 f 也就为 1

关键代码

```
1  for(int i = 1; i <= n; i++) {
2      if(i == 1) {
3          for(int j = 0; j <= 1; j++) {
4              for(int k = 0; k <= 1; k++) {
5                  f[j][i][k] = k;
6              }
7          }
8          continue;
9      }
10     for(int j = 0; j < (1 << i); j++) {
11         int s1 = 0, s0 = 1;
12         for(int k = 1; k <= p; k++) {
13             if(vis[k] > i) break;
14             int las = GetLas(j, i, vis[k] - 1); // lmask
15             s0 &= f[1][(i - 1) % 2][las];
16             s1 |= f[0][(i - 1) % 2][las];
17         }
18         f[0][i % 2][j] = s0;
19         f[1][i % 2][j] = s1;
20     }
```



```
20     }  
21 }
```

C++

3. 树形DP

顾名思义，在树上进行的DP叫树形DP，一般通过DFS进行。

例题

1. 有线电视网

题意：有一棵树，每条边有一个权值，每个叶子节点也有权值。非叶子节点可以选择把它和它的子树全部砍掉，最终收益为剩下树的边权和减去叶子节点权值和，要求收益 ≥ 0 ，求叶子节点最多能有多少

solution：考虑 $f_{u,i}$ 表示从 u 节点中选 i 个叶子节点能得到最大收益，对于 u 的某一个叶子节点 v ，我们可以有如下的状态转移： $f_{u,i} = \max(f_{u,i-j} + f_{v,j} - w_{u,v}, f_{u,i})$;

即假设从 v 中选 j 个叶子节点，那么转移方程就显而易见了

需注意的是， i 需要从大往小搜，否则不满足次序性

最后答案即为 $\max_{f_{1,i} \geq 0}(i)$

时间复杂度 $\sum cd_u^2(\text{枚举 } ij) = O(N * M)$

4. 其他常见DP套路

4.1 网格DP

4.1.1 求路径方案

每次能向下或者向右，初始在 $(1,1)$ 最后要到 (n,m)

```

1  for(int i = 1; i ≤ n; i++) {
2      for(int j = 1; j ≤ m; j++) {
3          if(i == 1 || j == 1) {
4              f[i][j] = 1;
5              continue;
6          }
7          f[i][j] = f[i - 1][j] + f[j - 1][i];
8      }
9  }

```

C++

其实 $f_{i,j} = C_{i+j-2}^{i-1}$

4.1.2 网格有障碍物

问题同上，新增了 k 个障碍物不能经过，求总方案数

Method 1

当 $O(N * M)$ 比较小的时候，直接暴力 DP

```

1  for(int i = 1; i ≤ n; i++) {
2      for(int j = 1; j ≤ m; j++) {
3          if(vis[i][j]) { // 有障碍物，且优先级高于判断(1,1)这个点
4              f[i][j] = 0;
5              continue;
6          }
7          if(i == 1 && j == 1) { // 在判断障碍物下面
8              f[i][j] = 1;
9              continue;
10         }
11         f[i][j] = f[i - 1][j] + f[i][j - 1];
12     }
13 }

```

C++

Method 2

当 $O(N * M)$ 比较大，而 $O(K^2)$ 比较小的时候，可以用容斥 + DP 来完成

令 f_i 为起点到第 i 个障碍物的方案数

注意，如果有重复点需要特别判断，本代码没有特别判断

```

1  void solve() {
2      n = read(), m = read(), k = read();
3      std::vector<std::pair<int, int>> v;
4      for(int i = 1; i ≤ k; i++) {
5          int x = read(), y = read();
6          v.push_back({x, y});
7          if((x == 1 && y == 1) || (x == n && y == m)) {
8              std::cout << 0;
9              return;
10         }
11     }
12 }

```

```

11     }
12     v.push_back({n, m}); // 一定要把终点push进去, 这样算出来才是对的, 终点不是(n,m)的话这
    里要改
13     std::sort(v.begin(), v.end()); // 自动先按照x升序排序, 再按照y升序排序
14     for(int i = 0; i < v.size(); i++) {
15         int dx = v[i].first - 1, dy = v[i].second - 1; // 注意, 如果初始点不是(1,1)这
    里就要改
16         // (1,1)到当前点的方案, 无果无障碍
17         f[i] = C(dx + dy, dx);
18         for(int j = 0; j < i; j++) {
19             if(v[j].second > v[i].second) continue;
20             // 必须满足j在i的左上角, x经过排序已经自动满足了
21             dx = v[i].first - v[j].first;
22             dy = v[i].second - v[j].second;
23             f[i] = (f[i] - f[j] * C(dx + dy, dy) % mod + mod) % mod;
24             // f[i]减去j到i的所有方案数
25         }
26     }
27     std::cout << f[v.size() - 1]; // 最后一个点是(n,m), 所以直接输出
28 }

```

C++

五. 数据结构

1. 线段树

1.1 基础线段树

1.1.1 引入

线段树是常用的用来维护 **区间信息** 的数据结构。

线段树可以在

的时间复杂度内实现单点修改、区间修改、区间查询（区间求和，求区间最大值，求区间最小值）等操作。

1.1.(1.5)

对于线段树的`init`操作，记得开大一点

```

1 void init() {
2     for(int i = 0; i ≤ 4 * (n + 1) + 100; i++) // 若线段树是[0, n]那就必须开(n+1)
3     // 或者
4     for(int i = 0; i ≤ 8 * n + 5; i++)
5 }

```

C++

1.1.2 基本结构&建树

不过多赘述

```

1 void build(int l, int r, int p) {
2     if(l == r) {
3         tr[p] = a[l];
4         return;
5     }
6     int mid = (l + r) >> 1;
7     build(l, mid, p * 2);
8     build(mid + 1, r, p * 2 + 1);
9     tr[p] = max(tr[p * 2], tr[p * 2 + 1]);
10    // 或者
11    tr[p] = tr[p * 2] + tr[p * 2 + 1];
12 }

```

C++

注意空间开 $O(4N)$

1.1.3 查询

依旧不过多赘述

```

1 ll query(int l, int r, int lx, int rx, int p) {
2     // [lx, rx]是查询区间, [l, r]是当前区间
3     if(rx < l || r < lx) return; // 我更喜欢写在这里, 更好想 (不在区间直接退出)
4     if(lx ≤ l && r ≤ rx) {
5         // 若当前区间包含查询区间, 有很多不需要的部分, 所以不能直接输出
6         return tr[p];
7     }
8     int mid = (l + r) >> 1;
9     ll ans = 0;
10    ans += max(ans, ask(l, mid, lx, rx, p * 2));
11    ans += max(ans, ask(mid + 1, r, lx, rx, p * 2 + 1));
12    // 判断是否需要查询这两个区间 (是否有交集)
13    return ans;
14 }

```

C++

1.1.4 区间修改 (懒标记)

并不是所有修改都有必要

只要对区间进行改变，然后需要的时候再改变即可大幅度优化

query 和 *update* 的时候都需要 *pushdown*

```
1 void Lazy(int p, int l, int r, int c) {
2     tr[p] += (r - l + 1) * c;
3     tag[p] += c; // p的子树需要改变的值
4 }
5 void pushdown(int p, int l, int r) {
6     int mid = (l + r) >> 1;
7     Lazy(p * 2, l, mid, tag[p]);
8     Lazy(p * 2 + 1, mid + 1, r, tag[p]);
9     tag[p] = 0;
10 }
11 void update(int p, int l, int r, int lx, int rx, int c) {
12     // [lx,rx]是修改区间, [l,r]是当前区间, c是改变值
13     if(rx < l || r < lx) return; //我更喜欢写在这里
14     pushdown(p, l, r);
15     if(lx ≤ l && r ≤ rx) {
16         Lazy(p, l, r, c); //简化代码
17         return;
18     }
19     int mid = (l + r) >> 1;
20     update(p * 2, l, mid, lx, rx, c);
21     update(p * 2 + 1, mid + 1, r, lx, rx, c);
22     tr[p] = tr[p * 2] + tr[p * 2 + 1];
23 }
24 ll query(int p, int l, int r, int lx, int rx) {
25     if(rx < l || r < lx) return 0;
26     pushdown(p, l, r);
27     if(lx ≤ l && r ≤ rx) return tr[p];
28     int mid = (l + r) >> 1;
29     ll ans = 0;
30     ans += query(p * 2, l, mid, lx, rx);
31     ans += query(p * 2 + 1, mid + 1, r, lx, rx);
32     return ans;
33 }
```

C++

ACM 小助手提醒您，不开 *long long* 见祖宗！

提醒： *pushdown* 是对于整个区间的修改的下传，就算你本次操作不涉及区间的所有范围也要全部修改，否则传递不及时，会出问题

提醒 2： *pushdown* 只需要传递即可，而真正的修改是在 $if(lx < l \&\& r \leq rx)$ 里面通过 *Lazy* 函数修改

1.1.5 例题

P3373 【模板】线段树 2

题目地址： [P3373 【模板】线段树 2

1.1.5.1 Code

```
1  #include<bits/stdc++.h>
2  #define ll long long
3  using namespace std;
4  const ll N = 4e5 + 100, M = 998244352, M2 = 998244353;
5  ll n, m, q, tr[N], mul[N], add[N], a[N];
6  ll read() {
7      ll x = 0, f = 1; char ch = getchar();
8      while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
9      while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
10     return x * f;
11 }
12 void build(int l, int r, int p) {
13     mul[p] = 1; //初始化
14     if(l == r) {
15         tr[p] = a[l] % m;
16
17         return;
18     }
19     int mid = (l + r) >> 1;
20     build(l, mid, p * 2);
21     build(mid + 1, r, p * 2 + 1);
22     tr[p] = tr[p * 2] + tr[p * 2 + 1];
23     tr[p] %= m;
24 }
25 void Lazy(int l, int r, int p, ll mulx, ll addx) {
26     tr[p] = tr[p] * mulx + addx * (r - l + 1);
27     mul[p] *= mulx;
28     add[p] = add[p] * mulx + addx;
29     mul[p] %= m; add[p] %= m; tr[p] %= m;
30 }
31 void pushdown(int l, int r, int p) {
32     int mid = (l + r) >> 1;
33     Lazy(l, mid, p * 2, mul[p], add[p]);
34     Lazy(mid + 1, r, p * 2 + 1, mul[p], add[p]);
35     mul[p] = 1;
36     add[p] = 0;
37 }
38 void update(int l, int r, int lx, int rx, int op, ll c, int p) {
39     if(lx ≤ l && r ≤ rx) {
40         if(op == 1) Lazy(l, r, p, c, 0);
41         else Lazy(l, r, p, 1, c);
42         return;
43     }
44     pushdown(l, r, p);
45     int mid = (l + r) >> 1;
46     if(lx ≤ mid) update(l, mid, lx, rx, op, c, p * 2);
47     if(mid + 1 ≤ rx) update(mid + 1, r, lx, rx, op, c, p * 2 + 1);
48     tr[p] = tr[p * 2] + tr[p * 2 + 1];
49     tr[p] %= m;
50 }
51 ll query(int l, int r, int lx, int rx, int p) {
52     if(lx ≤ l && r ≤ rx) return tr[p];
53     pushdown(l, r, p);
54     ll mid = (l + r) >> 1, ans = 0;
55     if(lx ≤ mid) ans = (ans + query(l, mid, lx, rx, p * 2)) % m;
```

```

56     if(mid + 1 ≤ rx) ans = (ans + query(mid + 1, r, lx, rx, p * 2 + 1)) % m;
57     return ans;
58 }
59 signed main() {
60     n = read(), q = read(), m = read();
61     for(int i = 1; i ≤ n; i++) a[i] = read();
62     build(1, n, 1);
63     for(int i = 1; i ≤ q; i++) {
64         int op = read(), l = read(), r = read();
65         if(op == 1 || op == 2) {
66             int k = read();
67             update(1, n, l, r, op, k, 1);
68         }
69         else printf("%lld\n", query(1, n, l, r, 1));
70     }
71     return 0;
72 }

```

C++

1.2 线段树的动态开点

对于要用几个甚至更多个的线段树，如果每个都全开那空间复杂度肯定承受不住

所以需要用到动态开点。顾名思义，为有多少开多少

首先，每一个点有如下状态：

```

1  p: 当前点的编号
2  s[x]: 第x颗线段树第一个点的编号 (与p同价)
3  sl[p]: p节点左子树的编号 (与p同价)
4  sr[p]: p节点右子树的编号 (与p同价)
5  tr[p]: 存放需要求的东西

```

C++

具体代码如下：

1.2.1 Code

```

1  int build(int l, int r, int p) {
2      if(!p) p = ++cnt;
3      if(l == r) {
4          tr[p] = a[l];
5          return p;
6      }
7      int mid = (l + r) >> 1;
8      sl[p] = build(l, mid, sl[p]);
9      sr[p] = build(mid + 1, r, sr[p]);
10     tr[p] = tr[sl[p]] + tr[sr[p]];
11 }

```

C++

注意：对于**动态开点**，*build* & *update*操作都**需要返回值**

注意：对于*update*操作，如果不在区间不应该返回0，应该返回*p*

具体如下：

```
1 int update(int l, int r, int lx, int rx, int c, int p) {
2     if(rx < l || r < lx) return p;
3     sl[p] = update();
4     sr[p] = update();
5 }
```

C++

如果不返回 p ，那么前面的 sl_p 和 sr_p 可能被除名

1.3 线段树合并

对于某些特殊问题，需要把两颗线段合并到一起

首先，对于线段树合并，必须要用到动态开点，见动态开点板块

合成方法：

一棵树为主树，另一颗为需要被合并的树（但不需要删除内容）（称为子线段树，子树）

如果查询到区间 $[l, r]$ ，若子树在这一区间上为空，那直接返回，不需要操作

如果主树为空，那么直接把主树接到子树上（令主树这一区间编号等于子树这一区间编号）

否则继续往下搜

线段树合并出现场景：

有 n 个点，每个点都有 m 种物品/颜色等会进行增加or减少操作，但是 $n * m$ 非常大不可能全部存下，这个时候就需要用线段树动态开点存放，最终通过线段树合并求答案

1.3.1 Code

```
1 void merge(int x, int y, int l, int r) {
2     //x为主树当前区间编号，y为子树当前区间编号
3     if(!x) return y; //若主树不存在，返回子树编号，自动连接
4     if(!y) return x; //若子树不存在，不进行操作
5     if(l == r) {
6         //do something
7         return x;
8     }
9     int mid = (l + r) >> 1;
10    sl[x] = merge(sl[x], sl[y], l, mid);
11    sr[y] = merge(sr[x], sr[y], mid + 1, r);
12    //主树和子树区间操作应当一致
13    update_tr(p); //对需要要求的值进行更新，例如tr[p]=tr[sl[p]]+tr[sr[p]]
14 }
```

C++

1.3.2 线段树合并例题

题目地址: [P4556 \[Vani有约会\] 雨天的尾巴](#)

1.3.2.1 solution

首先 n^2 不是能承受的空间复杂度,但是由于总修改次数并不高,所以考虑用线段树维护

对于每个点单独开一棵线段树来维护这些物资

最后我们从叶子结点往上进行合并即可

相当于求一个差分,对于路径 $x - y$

在 x, y 上各加1,然后在 $LCA(x, y)$ 和 $father[LCA(x, y)]$ 上各减1

这样能最后相加时,路径 $x - LCA(x, y) - y$ 上每个点恰好增加一次(差分思想)

线段树合并和树上差分一起出现是很常见的类型,需注意

树上的问题运用 LCA 是很常见的,需熟知

1.3.3 Code

```
1 #include<bits/stdc++.h>
2 #define ll long long
3 using namespace std;
4 const ll N = 1e5 + 100, M = 998244353, CN = 1e5;
5 int cnt, num, ne[2 * N], to[2 * N], fi[2 * N];
6 int sl[50 * N], s[50 * N], sr[50 * N];
7 int n, m, f[N][22], sum[50 * N], col[50 * N], ans[N], dep[N];
8 ll read() {
9     ll x = 0, f = 1; char ch = getchar();
10    while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
11    while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
12    return x * f;
13 }
14 void add(int u, int v) {
15     ne[++num] = fi[u];
16     fi[u] = num;
17     to[num] = v;
18 }
19 void dfs(int u, int fa, int d) {
20     dep[u] = d; f[u][0] = fa;
21     for(int i = fi[u]; i; i = ne[i]) {
22         int v = to[i];
23         if(v == fa) continue;
24         dfs(v, u, d + 1);
25     }
26 }
27 void init() {
28     dfs(1, 0, 1); // 计算深度和father节点
29     for(int j = 1; j ≤ 20; j++) {
30         for(int i = 1; i ≤ n; i++) {
```

```

31         f[i][j] = f[f[i][j - 1]][j - 1];
32         //LCA预备工作
33     }
34 }
35 }
36 void update_sum(int p) { //如果相同取编号小的, 下标即为种类编号
37     if(sum[sr[p]] > sum[sl[p]]) {
38         sum[p] = sum[sr[p]];
39         col[p] = col[sr[p]];
40     }
41     else {
42         sum[p] = sum[sl[p]];
43         col[p] = col[sl[p]];
44     }
45 }
46 int update(int l, int r, int p, int c, int val) {
47     if(r < c || l > c) return p;
48     //注意!!! 这里返回值一定是编号p, 以保持一致性
49     if(!p) p = ++cnt;
50     if(l == r) {
51         sum[p] += val;
52         col[p] = c;
53         return p;
54     }
55     int mid = (l + r) >> 1;
56     sl[p] = update(l, mid, sl[p], c, val);
57     sr[p] = update(mid + 1, r, sr[p], c, val);
58     update_sum(p);
59     return p;
60 }
61 int lca(int x, int y) { //求LCA
62     if(dep[x] < dep[y]) swap(x, y);
63     for(int i = 20; i ≥ 0; i--) {
64         if(dep[f[x][i]] ≥ dep[y]) {
65             x = f[x][i];
66         }
67     }
68     if(x == y) return x;
69     for(int i = 20; i ≥ 0; i--) {
70         if(f[x][i] ≠ f[y][i]) {
71             x = f[x][i];
72             y = f[y][i];
73         }
74     }
75     return f[x][0];
76 }
77 int merge(int x, int y, int l, int r) { //合并
78     if(!x) return y;
79     if(!y) return x;
80     if(l == r) {
81         sum[x] += sum[y];
82         return x;
83     }
84     int mid = (l + r) >> 1;
85     sl[x] = merge(sl[x], sl[y], l, mid);
86     sr[x] = merge(sr[x], sr[y], mid + 1, r);
87     update_sum(x);
88     return x;

```

```

89 }
90 void calc(int u, int fa) { // 差分求答案
91     for(int i = fi[u]; i; i = ne[i]) {
92         int v = to[i];
93         if(v == fa) continue;
94         calc(v, u);
95         s[u] = merge(s[u], s[v], 1, CN);
96     }
97     ans[u] = col[s[u]];
98     // 注意, 因为是差分计算的, 中途sum ≤ 0都计算了颜色, 所以需要特判
99     if(!sum[s[u]]) ans[u] = 0; // 只有sum=0才能说明 ans不存在
100 }
101 int main() {
102     n = read(), m = read();
103     for(int i = 1; i < n; i++) {
104         int u = read(), v = read();
105         add(u, v); add(v, u);
106     }
107     init();
108     for(int i = 1; i ≤ m; i++) {
109         int x = read(), y = read(), c = read();
110         int F = lca(x, y); int FF = f[F][0]; // FF为F的father
111         s[x] = update(1, CN, s[x], c, 1); // x的线段树上c的数量+1
112         s[y] = update(1, CN, s[y], c, 1); // y的线段树上c的数量+1
113         s[F] = update(1, CN, s[F], c, -1); // F的线段树上c的数量+1
114         s[FF] = update(1, CN, s[FF], c, -1); // FF的线段树上c的数量+1
115     }
116     calc(1, 0); // 从下到上计算, 用差分的思想计算
117     for(int i = 1; i ≤ n; i++) cout << ans[i] << endl;
118     return 0;
119 }

```

C++

1.4 线段树例题

P12846 [蓝桥杯 2025 国 A] 翻转硬币

见[!比赛题目合集] ---> [蓝桥杯] ---> [2025蓝桥杯CA国赛] ---> [G]

链接: [文件地址](#)

P1502 窗口的星星

题目地址: [P1502 窗口的星星](#)

Solution

和扫描线差不多的思路, 从下往上进行扫描合并, 并计算最值

注意! *cmp*里面第一关键字为*r*, 第二关键字为*c*, 从大到小, 这样才能求到最大值!

Code

```
1  #include<bits/stdc++.h>
2  #define ll long long
3  using namespace std;
4  const ll N = 1e6 + 100, M = 998244353, CN = 1e6 + 10;
5  ll n, m, ls[N], fr[N];
6  ll tr[N << 2], pd[N << 2];
7  ll read() {
8      ll x = 0, f = 1; char ch = getchar();
9      while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
10     while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
11     return x * f;
12 }
13 map<ll, int> mp;
14 struct Point {
15     ll l1, l2;
16     ll r, c;
17 } s[N << 2];
18 bool cmp(Point x, Point y) {
19     if(x.r == y.r) return x.c > y.c; //特别注意! 一定要正确排序
20     return x.r < y.r;
21 }
22 void build(int l, int r, int p) {
23     tr[p] = pd[p] = 0;
24     if(l == r) return;
25     int mid = (l + r) >> 1;
26     build(l, mid, p * 2);
27     build(mid + 1, r, p * 2 + 1);
28 }
29 void Lazy(int p, int c) {
30     tr[p] += c;
31     pd[p] += c;
32 }
33 void pushdown(int p) {
34     Lazy(p * 2, pd[p]);
35     Lazy(p * 2 + 1, pd[p]);
36     pd[p] = 0;
37 }
38 void update(int l, int r, int lx, int rx, int p, int c) {
39     if(rx < l || r < lx) return;
40     if(lx ≤ l && r ≤ rx) {
41         Lazy(p, c);
42         return;
43     }
44     int mid = (l + r) >> 1;
45     pushdown(p);
46     update(l, mid, lx, rx, p * 2, c);
47     update(mid + 1, r, lx, rx, p * 2 + 1, c);
48     tr[p] = max(tr[p * 2], tr[p * 2 + 1]);
49 }
50 int main() {
51     int T = read();
52     while(T--) {
53         memset(fr, 0, sizeof(fr));
54         memset(ls, 0, sizeof(ls));
55         memset(s, 0, sizeof(s));
```

```

56     n = read(); ll W = read(), H = read();
57     for(int i = 1; i ≤ n; i++) {
58         int x = read(), y = read(), c = read();
59         s[i].l1 = x; s[i].l2 = x + W - 1;
60         s[i + n].l1 = x; s[i + n].l2 = x + W - 1;
61         s[i].r = y; s[i + n].r = y + H - 1;
62         s[i].c = c, s[i + n].c = -c;
63         ls[i] = x, ls[i + n] = x + W - 1;
64     }
65     n *= 2;
66     sort(ls + 1, ls + n + 1);
67     sort(s + 1, s + n + 1, cmp);
68     int cnt = 0;
69     mp.clear();
70     for(int i = 1; i ≤ n; i++) {
71         if(!mp[ls[i]]) mp[ls[i]] = ++cnt;
72         fr[mp[ls[i]]] = ls[i];
73     }
74     for(int i = 1; i ≤ n; i++) {
75         s[i].l1 = mp[s[i].l1];
76         s[i].l2 = mp[s[i].l2];
77     }
78     build(1, cnt, 1);
79     ll ans = 0;
80     s[n + 1].r = s[n].r;
81     for(int i = 1; i ≤ n; i++) {
82         update(1, cnt, s[i].l1, s[i].l2, 1, s[i].c);
83         ans = max(ans, tr[1]);
84     }
85     cout << ans << endl;
86 }
87 return 0;
88 }
89 //月雪·薇婷

```

C++

P2824 [HEOI2016/TJOI2016] 排序

考思维的一道题

Solution

假设如果数组 a 是一个01串，那这两个操作我们很好进行维护

假设最后的数是 x ，那么我们把 $\geq x$ 的设为1，反之设为0

再模拟维护一下，最后判断第 p 位，如果 p 位上面的数为0那肯定假设有误

如果为1，那就有可能正确

用二分查找一下即可

尤其注意`pushdown`和`Lazy`的维护！要首先判断需不需要下传

Code

```
1  #include<bits/stdc++.h>
2  #define ll long long
3  using namespace std;
4  const ll N = 1e6 + 100, M = 998244353, CN = 1e6 + 10;
5  int n, m, q, pd[N], num[N], x[N], y[N], a[N], op[N];
6  ll read() {
7      ll x = 0, f = 1; char ch = getchar();
8      while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
9      while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
10     return x * f;
11 }
12 void build(int l, int r, int p, int c) {
13     pd[p] = 0;
14     if(l == r) {
15         if(a[l] ≥ c) {
16             num[p] = 1;
17         }
18         else num[p] = 0;
19         return;
20     }
21     int mid = (l + r) >> 1;
22     build(l, mid, p * 2, c);
23     build(mid + 1, r, p * 2 + 1, c);
24     num[p] = num[p * 2] + num[p * 2 + 1];
25 }
26 void Lazy(int l, int r, int p, int c) {
27     int cx = 1;
28     if(c == -1 || !c) c = 0, cx = -1;
29     num[p] = (r - l + 1) * c;
30     pd[p] = cx;
31 }
32 void pushdown(int l, int r, int p) {
33     if(!pd[p]) return;
34     int mid = (l + r) >> 1;
35     Lazy(l, mid, p * 2, pd[p]);
36     Lazy(mid + 1, r, p * 2 + 1, pd[p]);
37     pd[p] = 0;
38 }
39 void update(int l, int r, int p, int lx, int rx, int c) {
40     if(lx > rx) return;
41     if(rx < l || r < lx) return;
42     if(lx ≤ l && r ≤ rx) {
43         Lazy(l, r, p, c);
44         return;
45     }
46     pushdown(l, r, p);
47     int mid = (l + r) >> 1;
48     update(l, mid, p * 2, lx, rx, c);
49     update(mid + 1, r, p * 2 + 1, lx, rx, c);
50     num[p] = num[p * 2] + num[p * 2 + 1];
51 }
52 int query(int l, int r, int p, int lx, int rx) {
53     if(rx < l || r < lx) return 0;
```

```

54     if(lx ≤ l && r ≤ rx) return num[p];
55     int ans = 0;
56     int mid = (l + r) >> 1;
57     pushdown(l, r, p);
58     ans += query(l, mid, p * 2, lx, rx);
59     ans += query(mid + 1, r, p * 2 + 1, lx, rx);
60     num[p] = num[p * 2] + num[p * 2 + 1];
61     return ans;
62 }
63 bool check(int mid) {
64     build(1, n, 1, mid);
65     for(int i = 1; i ≤ m; i++) {
66         int t = query(1, n, 1, x[i], y[i]), Len = y[i] - x[i] + 1;
67         if(op[i] == 0) {
68             t = Len - t;
69             update(1, n, 1, x[i], x[i] + t - 1, 0);
70             update(1, n, 1, x[i] + t, y[i], 1);
71         }
72         else {
73             update(1, n, 1, x[i], x[i] + t - 1, 1);
74             update(1, n, 1, x[i] + t, y[i], 0);
75         }
76     }
77     if(query(1, n, 1, q, q)) return true;
78     return false;
79 }
80 int main() {
81     n = read(), m = read();
82     for(int i = 1; i ≤ n; i++) a[i] = read();
83     for(int i = 1; i ≤ m; i++) {
84         op[i] = read();
85         x[i] = read();
86         y[i] = read();
87     }
88     q = read();
89     int l = 1, r = n;
90     while(l < r) {
91         int mid = (l + r + 1) >> 1;
92         if(check(mid)) l = mid;
93         else r = mid - 1;
94     }
95     cout << l;
96     return 0;
97 }
98 // 月雪·薇婷

```

C++

2. 笛卡尔树

前提：每次操作都需要某个区间的最小/最大值，最好是一个排列

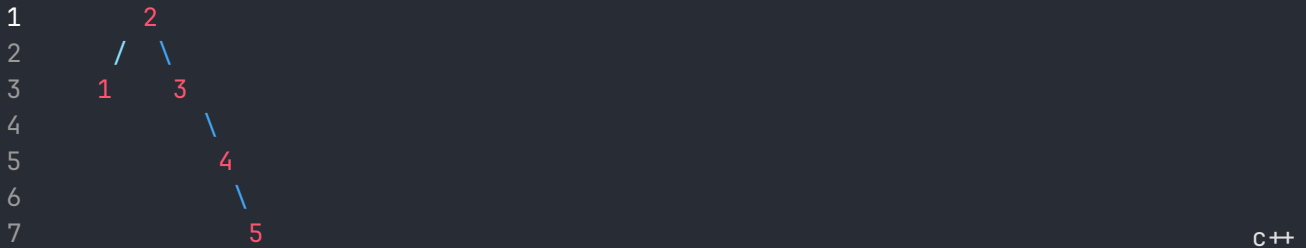
假设有一个序列，需要满足以下性质：

1. 左儿子节点编号 < 当前节点 < 右儿子节点编号

2. 当前节点权值 $<$ or $>$ 左右儿子权值 (小根堆 或者 大根堆)

性质:

1. 这样的树建树方式是唯一的
2. 每个点的左子树的节点编号集合是一个完整的区间, 右子树同理。比如:



对于2来说右子树节点编号的集合为 $[2, 5]$, 左子树节点编号为 $[1, 2]$

建树方法(以小根堆为例)

节点的权值分别为 $p_1 \sim p_n$ (一个排列)

按照节点编号从 $1 \sim n$ 插入。记当前需要插入的是 i , 现在判断到了 u 节点。假如 $p_i < p_u$ 那么说明 i 需要在 u 的上面, 令 $u = fa_u$, 如果 fa_u 不存在, 则直接令 i 为根节点。如果 $p_i > p_u$ 说明 i 应该当作 u 的右儿子, 而 u 原先的(如果有)右儿子接到 i 的左儿子

```
1 void build() {
2     //l[i]代表i的左儿子节点编号
3     //r[i]代表i的右儿子节点编号
4     int u = 1; //首先插入节点1
5     for(int i = 2; i ≤ n; i++) {
6         while(true) {
7             if(p[u] > p[i]) { //这里是小根堆, 如果是大根堆就改成'<'
8                 if(!fa[u]) { //如果u是根节点, 那么i当作根节点
9                     fa[u] = i;
10                    l[i] = u;
11                    u = i; //状态更新
12                    break;
13                }
14                else u = fa[u]; //否则继续往上搜
15            }
16            else {
17                int rs = r[u];
18                r[u] = i;
19                fa[i] = u;
20                if(rs) { //如果u有右儿子
21                    fa[rs] = i;
22                    l[i] = rs;
23                }
24                u = i; //状态更新
25                break;
26            }
27        }
28    }
```



```
28     }
29 }
```

C++

3. 树状数组

树状数组常应用于：

1. 维护差分序列
2. 区间内不同元素个数
3. 求逆序对
4. 区间第k小

如果需要区间修改，那还是老实的用线段树吧，树状数组优点只有代码简单

3.1 基本结构

```
1  #include<bits/stdc++.h>
2  #define int long long
3  using namespace std;
4  const int N = 1e6 + 100, M = 998244353, INF = 1e9 + 10;
5  int n, a[N], m, tr[N];
6  int read() {
7      int x;
8      scanf("%lld", &x);
9      return x;
10 }
11 int lowbit(int x) {
12     return x & (-x);
13 }
14 void add(int x, int y) {
15     while(x ≤ n) {
16         tr[x] += y;
17         x += lowbit(x);
18     }
19 }
20 int query(int x) {
21     int ans = 0;
22     while(x) {
23         ans += tr[x];
24         x -= lowbit(x);
25     }
26     return ans;
27 }
28 signed main() {
29     n = read(); m = read();
30     for(int i = 1; i ≤ n; i++) {
31         a[i] = read();
32         add(i, a[i]);
33     }
34     for(int i = 1; i ≤ m; i++) {
```

```

35     int op = read();
36     int x = read(), y = read();
37     if(op == 1) {
38         a[x] += y;
39         add(x, y);
40     }
41     else cout << query(y) - query(x - 1) << endl;
42 }
43 return 0;
44 }

```

C++

3.2 区间内不同元素个数

假设询问你 m 个区间，求每个区间内不同元素个数，可离线处理

步骤如下：

1. 对区间排序，按照 r 从小到大排序
2. 将 $[r_{i-1} + 1, r_i]$ 区间内所有元素 a_x 进行 $add(las[a_x], -1)$ 和 $add(x, 1)$ ， $las[a_x]$ 代表 a_x 上一次出现的位置

这样一来整个区间的和就只有到 r_i 为止，每个数最后出现位置有一个1

Code

```

1  #include<bits/stdc++.h>
2  #define int long long
3  using namespace std;
4  const int N = 1e6 + 100, M = 998244353, INF = 1e9 + 10;
5  int n, m, a[N], tr[N], las[N], ans[N];
6  int read() {
7      int x;
8      scanf("%lld", &x);
9      return x;
10 }
11 struct Qu {
12     int l, r, id;
13 } s[N];
14 bool cmp(Qu x, Qu y) {
15     return x.r < y.r;
16 }
17 int lowbit(int x) {
18     return x & (-x);
19 }
20 int query(int x) {
21     int ans = 0;
22     while(x) {
23         ans += tr[x];
24         x -= lowbit(x);
25     }
26     return ans;
27 }

```

```

28 void add(int x, int y) {
29     while(x ≤ n) {
30         tr[x] += y;
31         x += lowbit(x);
32     }
33 }
34 signed main() {
35     n = read();
36     for(int i = 1; i ≤ n; i++) {
37         a[i] = read();
38     }
39     m = read();
40     for(int i = 1; i ≤ m; i++) {
41         s[i].l = read();
42         s[i].r = read();
43         s[i].id = i;
44     }
45     sort(s + 1, s + m + 1, cmp);
46     for(int i = 1; i ≤ m; i++) {
47         for(int j = s[i - 1].r + 1; j ≤ s[i].r; j++) {
48             add(j, 1); // 当前位+1
49             if(las[a[j]]) add(las[a[j]], -1); // 上一个位置-1
50             las[a[j]] = j;
51         }
52         ans[s[i].id] = query(s[i].r) - query(s[i].l - 1);
53     }
54     for(int i = 1; i ≤ m; i++) cout << ans[i] << '\n';
55     return 0;
56 }

```

C++

3.3 逆序对个数

```

1 for(int i = 1; i ≤ n; i++) {
2     add(a[i], 1); // 在a[i]位置上+1
3     cout << query(a[i] - 1); // 从[1, a[i]-1]的和
4 }

```

C++

3.4 维护差分序列

如果真要用区间修改，单点查询

$d_i = a_i - a_{i-1}$ ，那么区间 $[l, r] + k$ 即为 $d_l + k, d_{r+1} - k$ (两次单点修改)

而 $a_x = \sum_{i=1}^x d_i$ ，即一次区间查询 $[1, x]$

```

1 for(int i = 1; i ≤ m; i++) {
2     int l = 1, r = n, k = read();
3     // 区间[l, r]+k, 即差分序列d[l]+k, d[r+1]-k
4     add(l - 1, -k);
5     add(r, k);
6 }
7 int x = read();
8 cout << query(x);

```

C++

4. 树链剖分

树链剖分，把一棵树分解为若干重链，于是可以对链用数据结构维护从而做到维护这棵树

如果只需要单点修改+区间查询，尽量用树状数组维护，否则只能用线段树

4.1 剖分

剖分完成之后，每个点有一个 dfs 序，有以下性质：

1. 每条重链 dfs 连续
2. 每个点的子树的 dfs 序编号都在 $[dfs[u], dfs[u] + sz[u] - 1]$ 内

注意， $dfs2$ 开始的时候 $ftop$ 是根节点！即 $dfs2(s, s)$ 而不是 $dfs2(s, 0)!!$

```
1 void dfs1(int u, int f) {
2     sz[u] = 1; // 记录子树大小
3     fa[u] = f; // 记录father节点
4     dep[u] = dep[f] + 1; // 记录深度
5     int Max = 0;
6     for(int i = fi[u]; i; i = ne[i]) {
7         int v = to[i];
8         if(v == f) continue;
9         dfs1(v, u);
10        if(sz[v] > Max) {
11            Max = sz[v];
12            son[u] = v;
13        }
14        sz[u] += sz[v];
15    }
16 }
17 void dfs2(int u, int ftop) { // ftop: 记录当前重链上的根节点
18     dfn[u] = ++idx;
19     rk[idx] = u; // rk代表dfn序对应哪个点
20     top[u] = ftop;
21     if(son[u]) dfs2(son[u], ftop); // 有重儿子，ftop继承
22     for(int i = fi[u]; i; i = ne[i]) {
23         int v = to[i];
24         if(v == fa[u] || v == son[u]) continue;
25         // 这里改成v != fa[u]，因为ftop不是fa[u]了
26         dfs2(v, v); // 其他点，ftop不继承，以自身开头
27     }
28     // 注意，dfn在每颗子树上也是连续的
29     // [ dfn[u], dfn[u] + sz[u] - 1 ] 代表u的子树
30 }
31 void solve() {
32     dfs1(s, 0);
33     dfs2(s, s); // 注意，这里是dfs2(s, s)!!
34 }
```

4.2 修改+查询

注意，不加上数据结构维护的时间复杂度，重链向上跳的时间复杂度是 $O(\log n)$

```
1 void tr_update(int x, int y, int t) {
2     //x-y链上所有点权值+t
3     //从x, y不停往上跳，直到跳到在一条重链上
4     while(top[x] != top[y]) {
5         if(dep[top[x]] < dep[top[y]]) swap(x, y);
6         //把x换成 重链根节点 更深的点
7         update(1, 1, n, dfn[top[x]], dfn[x], t);
8         //线段树以dfn序建立，所以更新时用dfn序
9         x = fa[top[x]]; //每次 让 x=fa[top[x]]，跳到下一条链
10    } //每次跳完之后，所在子树大小至少减半，所以最多logn次跳完
11    //当前x, y在同一条重链上
12    if(dep[x] > dep[y]) swap(x, y);
13    //dep[x] < dep[y]且x, y在一条重链的时候，dfn[x]<dfn[y]
14    update(1, 1, n, dfn[x], dfn[y], t);
15 }
16 int tr_query(int x, int y) {
17     //求x-y链上所有点权值和
18     int ans = 0;
19     while(top[x] != top[y]) {
20         if(dep[top[x]] < dep[top[y]]) swap(x, y);
21         ans += query(1, 1, n, dfn[top[x]], dfn[x]);
22         ans %= mod;
23         x = fa[top[x]];
24     }
25     if(dep[x] > dep[y]) swap(x, y);
26     ans += query(1, 1, n, dfn[x], dfn[y]);
27     ans %= mod;
28     return ans;
29 }
```

C++

4.3 Code

```
1 #include<bits/stdc++.h>
2 // #define getchar getchar_unlocked
3 #define int long long
4 using namespace std;
5 const int N = 1e6 + 10, M = 1e9 + 7;
6 int n, m, mod, r, fa[N], ne[N], to[N], fi[N], num;
7 int idx, sz[N], son[N], dfn[N], dep[N], rk[N], a[N];
8 int top[N], tr[N * 2], tag[N * 2];
9 int read() {
10     int x = 0, f = 1; char ch = getchar();
11     while(ch < '0' || ch > '9') {
12         if(ch == '-') f = -1;
13         ch = getchar();
14     }
15     while(ch ≥ '0' && ch ≤ '9') {
16         x = x * 10 + ch - 48;
17         ch = getchar();
18     }
19     return x * f;
20 }
```

```

20 }
21 void add(int u, int v) {
22     ne[++num] = fi[u];
23     fi[u] = num;
24     to[num] = v;
25 }
26 void dfs1(int u, int f) {
27     sz[u] = 1;
28     fa[u] = f;
29     dep[u] = dep[f] + 1;
30     int Max = 0;
31     for(int i = fi[u]; i; i = ne[i]) {
32         int v = to[i];
33         if(v == f) continue;
34         dfs1(v, u);
35         if(sz[v] > Max) {
36             Max = sz[v];
37             son[u] = v;
38         }
39         sz[u] += sz[v];
40     }
41 }
42 void dfs2(int u, int ftop) { //ftop:记录当前重链上的根节点
43     dfn[u] = ++idx;
44     rk[idx] = u; //rk代表dfn序对应哪个点
45     top[u] = ftop;
46     if(son[u]) dfs2(son[u], ftop); //有重儿子, ftop继承
47     for(int i = fi[u]; i; i = ne[i]) {
48         int v = to[i];
49         if(v == fa[u] || v == son[u]) continue;
50         //这里改成v!=fa[u], 因为ftop不是fa[u]了
51         dfs2(v, v); //其他点, ftop不继承, 以自身开头
52     }
53     //注意, dfn在每颗子树上也是连续的
54     //[ dfn[u], dfn[u]+sz[u]-1 ]代表u的子树
55 }
56 void pushup(int p) {
57     tr[p] = tr[p * 2] + tr[p * 2 + 1];
58     tr[p] %= mod;
59 }
60 void build(int p, int l, int r) {
61     //线段树以dfn序建树
62     //因为每条重链的dfn序都是连续的, 可以很好的维护
63     //注意, dfn在每颗子树上也是连续的
64     //[ dfn[u], dfn[u]+sz[u]-1 ]代表u的子树
65     if(l == r) {
66         tr[p] = a[rk[l]]; //rk代表dfn序对应哪个点
67         return;
68     }
69     int mid = (l + r) >> 1;
70     build(p * 2, l, mid);
71     build(p * 2 + 1, mid + 1, r);
72     pushup(p);
73 }
74 void Lazy(int p, int l, int r, int t) {
75     tr[p] += (r - l + 1) * t;
76     tr[p] %= mod;
77     tag[p] += t;

```

```

78     tag[p] %= mod;
79 }
80 void pushdown(int p, int l, int r) {
81     int mid = (l + r) >> 1;
82     Lazy(p * 2, l, mid, tag[p]);
83     Lazy(p * 2 + 1, mid + 1, r, tag[p]);
84     tag[p] = 0;
85 }
86 void update(int p, int l, int r, int x, int y, int t) {
87     if(y < l || r < x) return;
88     pushdown(p, l, r);
89     if(x ≤ l && r ≤ y) {
90         Lazy(p, l, r, t);
91         return;
92     }
93     int mid = (l + r) >> 1;
94     update(p * 2, l, mid, x, y, t);
95     update(p * 2 + 1, mid + 1, r, x, y, t);
96     pushup(p);
97 }
98 int query(int p, int l, int r, int x, int y) {
99     if(y < l || r < x) return 0;
100    pushdown(p, l, r);
101    if(x ≤ l && r ≤ y) return tr[p];
102    int mid = (l + r) >> 1;
103    int ans = query(p * 2, l, mid, x, y);
104    ans = (ans + query(p * 2 + 1, mid + 1, r, x, y)) % mod;
105    return ans;
106 }
107 void op1(int x, int y, int t) {
108     //x-y链上所有点权值+t
109     while(top[x] ≠ top[y]) {
110         if(dep[top[x]] < dep[top[y]]) swap(x, y);
111         //把x换成 重链根节点 更深的点
112         update(1, 1, n, dfn[top[x]], dfn[x], t);
113         //线段树以dfn序建立, 所以更新时用dfn序
114         x = fa[top[x]]; //每次 让 x=fa[top[x]], 跳到下一条链
115     } //每次跳完之后, 所在子树大小至少减半, 所以最多logn次跳完
116     //当前x, y在同一条重链上
117     if(dep[x] > dep[y]) swap(x, y);
118     //dep[x] < dep[y]且x, y在一条重链的时候, dfn[x]<dfn[y]
119     update(1, 1, n, dfn[x], dfn[y], t);
120 }
121 int op2(int x, int y) {
122     //求x-y链上所有点权值和
123     int ans = 0;
124     while(top[x] ≠ top[y]) {
125         if(dep[top[x]] < dep[top[y]]) swap(x, y);
126         ans += query(1, 1, n, dfn[top[x]], dfn[x]);
127         ans %= mod;
128         x = fa[top[x]];
129     }
130     if(dep[x] > dep[y]) swap(x, y);
131     ans += query(1, 1, n, dfn[x], dfn[y]);
132     ans %= mod;
133     return ans;
134 }
135 //注意, dfn在每颗子树上也是连续的

```

```

136 // [ dfn[u], dfn[u]+sz[u]-1 ] 代表u的子树
137 void op3(int x, int t) {
138     update(1, 1, n, dfn[x], dfn[x] + sz[x] - 1, t);
139     // x的子树所有点权值+t
140 }
141 int op4(int x) {
142     // 求x子树所有点权值
143     int ans = 0;
144     ans = query(1, 1, n, dfn[x], dfn[x] + sz[x] - 1);
145     return ans;
146 }
147 void solve() {
148     n = read(), m = read(), r = read(), mod = read();
149     for(int i = 1; i ≤ n; i++) {
150         a[i] = read();
151         a[i] %= mod;
152     }
153     for(int i = 1; i < n; i++) {
154         int u = read(), v = read();
155         add(u, v); add(v, u);
156     }
157     dfs1(r, 0);
158     dfs2(r, r); // 注意这里是r, r !
159     build(1, 1, n);
160     for(int i = 1; i ≤ m; i++) {
161         int op = read();
162         if(op == 1) {
163             int x = read(), y = read(), z = read();
164             op1(x, y, z);
165         }
166         else if(op == 2) {
167             int x = read(), y = read();
168             cout << op2(x, y) << '\n';
169         }
170         else if(op == 3) {
171             int x = read(), y = read();
172             op3(x, y);
173         }
174         else {
175             int x = read();
176             cout << op4(x) << '\n';
177         }
178     }
179 }
180 signed main() {
181     int T = 1;
182     while(T--) solve();
183     return 0;
184 }
185
186 /*
187
188 */

```


5. 主席树（可持久化线段树，区间第k大）

主席树：静态区间前 k 大

平衡树：动态维护前 k 大

6. 平衡树（第k大）

主席树：静态区间前 k 大

平衡树：动态维护前 k 大

7. 单调栈

求一个数组中，左边和右边最靠近每个 a_i 的 $\leq a_i / < a_i / \geq a_i / a_i$ 的数的位置在哪

```
1  for(int i = 1; i ≤ n; i++) {
2      while(!st.empty() && a[st.top()] ≥ a[i]) { // 求的是最靠左的 < a[i]的
3          // 如果a[st.top()] ≥ a[i]，那么对于j=i+1~n这些位置，如果a[j] ≥ a[i]，那么答案就该
          更新为i，而不是st.top()
4          // 如果a[j] ≤ a[i]，那更不可能了，因为求的是<a[j]的，而a[st.top()] ≥ a[i] ≥ a[j]
5          // 所以综上，无论如何st.top()在这种情况下都该被弹出
6          st.pop();
7      }
8      if(!st.empty()) l[i] = st.top();
9      st.push(i);
10 }
11 while(!st.empty()) st.pop(); // 清空栈
12 for(int i = n; i ≥ 1; i--) {
13     while(!st.empty() && a[st.top()] ≥ a[i]) { // 求的是最靠右的 < a[i]的
14         // 其余同理
15         st.pop();
16     }
17     if(!st.empty()) r[i] = st.top(); // 记得这里改成 r
18     st.push(i);
19 }
```

C++

六. 数学相关

1. 一些常用结论

$$1. \sum_{i=1}^N \frac{1}{i} \approx \ln N \rightarrow \sum_{i=1}^N \frac{N}{i} \approx N * \ln N$$

$$2. \sum_{i \in P} \frac{1}{i} \approx \ln(\ln(N)) \rightarrow \sum_{i \in P} \frac{N}{i} \approx N * \ln(\ln(N)) \quad (P \text{ 是质数集, 此条结论可以用于筛素数, 时间复杂度略次于线性筛})$$

$$3. [1, N] \text{ 的质数个数大约为 } \frac{N}{\ln(N)}$$

$$4. n \text{ 个盒子放 } k \text{ 个球, 方案数为 } C_{n+k-1}^k. \text{ 如果不允许空盒, 方案数是 } C_{k-1}^{n-1} \text{ (每个盒子都先放了1个球)}$$

$$5. n \text{ 个盒子, 放的球的数量单调不减 (或者单调不减), 每个盒子能放 } [0, k] \text{ 个, 方案数为 } C_{n+k}^n$$

$$6. \sum_{m=0}^k C_{n+m-1}^m = C_{n+k}^k, \text{ 即分别能放 } 0 \sim k \text{ 个球的方案}$$

$$7. \text{ 假设有 } a < b < c, \text{ 那么, } \gcd(a, b, c) = \gcd(a, b-a, c-b)$$

$$8. \sum_{i=1}^n \frac{n}{i^2} \approx \frac{\pi^2 \cdot n}{6}$$

2. 线性基(异或相关问题, GF(2)和GF(3))

重要!!!! 特殊判断, 若线性基经过高斯消元结束后秩 $r < n$ 则能找到一个子集 $XOR a_i = 0$

通常在做异或相关问题的时候 (异或和最大, 异或和第k大, 图上异或和等等)

会用到异或线性基

异或线性基是一个长度为 $\log(W)$ 的数组 $p = p_1, \dots, p_n$

假设原数组为 $a = a_1, \dots, a_n$

有以下特性:

1. 任意子集的异或和不为0。在下述构造中, 第 i 位线性基在二进制中为1的最高位为第 i 位, 这样就保证了任意子集异或和一定不为0
2. $a_i = p_{k_1} \oplus p_{k_2} \dots \oplus p_{k_m}$ 恒成立 (a_i 能由线性基某个子集表示)
3. $p_i = a_{k_1} \oplus a_{k_2} \dots \oplus a_{k_m}$ 恒成立 (p_i 能由原数组某个子集表示)

2&3特性表示:

原数组的任意子集的异或和等于线性基某个子集异或和

同时, 线性基的任意一个子集的异或和也等于原数组某个子集的异或和

总结: 线性基异或和集合 与 原数组异或和集合 完全相等

2.1 构造方法

对于 a_i , 从最高位找到最低位

若二进制下第 x 位为0, 则无影响

若二进制下第 x 位为1, 那么判断, 如果 $p_x = 0$, 则令 $p_x = a_i$

如果 $p_x \neq 0$, 那么令 $a_i = a_i \oplus p_x$

如果最后 $a_i = 0$ 说明 a_i 能通过现在的线性基构造出来

反之就一定能找到一位 $p_x = a_i$

最后反推回去一定能使得满足上述构造条件: 异或和的集合相同, 能相互构造出来

简单版本

```
1 void insert(int x) {
2     for(int i = 63; i ≥ 0; i--) {
3         int f = (x >> i) & 1;
4         if(!f) continue;
5         if(!p[i]) {
6             p[i] = x;
7             return;
8         }
9         x = x ^ p[i];
10    }
11 }
12 int main() {
13     n = read();
14     for(int i = 1; i ≤ n; i++) {
15         a[i] = read();
16         insert(a[i]);
17     }
18 }
```

C++

记录每个线性基来自于原数组哪几个 a_i 异或而来的版本

```

1  bitset<N> bs[70]; //bs[i][j]=1代表线性基i是由a[j1]^a[j2]^...^a[jx]异或而来
2  void insert(int x, int id) {
3      int mask = 0;
4      for(int i = 63; i ≥ 0; i--) {
5          int f = (x >> i) & 1;
6          if(!f) continue;
7          if(!p[i]) {
8              p[i] = x;
9              from[i] = mask;
10             //from[i]: 记录基向量 i是由哪些基向量异或而来
11             //用的二进制保存
12             bs[i].set(id);
13             //bs[i]: 记录用了原数组中哪些a[i]异或而成
14             //当前只记录a[id], 后续再GetSource()
15             //不然时间复杂度很大
16             return;
17         }
18         x = x ^ p[i];
19         mask |= (1ll << i);
20     }
21 }
22 void GetSource() {
23     for(int i = 62; i ≥ 0; i--) {
24         for(int j = i + 1; j ≤ 63; j++) {
25             if((from[i] >> j) & 1) {
26                 bs[i] ^= bs[j];
27             }
28         }
29     }
30 }
31 signed main() {
32     n = read();
33     for(int i = 1; i ≤ n; i++) {
34         a[i] = read();
35         insert(a[i], i);
36     }
37     GetSource();
38     return 0;
39 }

```

C++

时间复杂度: $O(N \log W)$, 常数会比较大

2.2 线性基合并

如果知道数组 a 的线性基为 $p1$

数组 b 的线性基为 $p2$

这个时候数组 $c = a + b$, 那么 c 的线性基可以直接由 $p1$ 和 $p2$ 合并

把 $p1$ 和 $p2$ 的数再做一次 $insert$ 即可

时间复杂度 $O(\log^2 W)$

```
1 void merge(int *c, int *a, int *b) { // c是合并后的
2     for(int i = 0; i ≤ 63; i++) c[i] = a[i]; // 先把a的值传给c
3     for(int i = 0; i ≤ 63; i++) { // 枚举b
4         int now = b[i];
5         for(int j = 63; j ≥ 0; j--) {
6             if(!(now >> j & 1)) continue;
7             if(c[j] now ^ c[j];
8             else {
9                 c[j] = now;
10                break;
11            }
12        }
13    }
14 }
15 merge(p, p1, p2);
```

C++

2.3 常用方法

2.3.-1 查找是否能 $XOR = k$

📢 Important

最重要的功能!!!

若未进行高斯消元，那么**必须从高位到低位进行判断**，每次都需要重新XOR再进行下一步

```
1 k = read();
2 for(int i = 63; i ≥ 0; i--) {
3     if((k >> i) & 1) {
4         if(!p[i]) {
5             std::cout << -1 << '\n';
6             return;
7         }
8         ans.push_back(i); // 答案
9         k ^= p[i]; // 重要!! 因为没有进行高斯消元，所有都是乱的，只能保证第i个线性基最高位是
            (1ll<<i)
10    }
11 }
```

C++

2.3.0 高斯消元

```
1 void rebuild() { // 高斯消元重构线性基
2     for(int i = 63; i ≥ 0; i--) {
3         for(int j = i - 1; j ≥ 0; j--) {
4             if((p[i] >> j) & 1) p[i] ^= p[j];
5         }
6     }
7     for(int i = 0; i ≤ 63; i++)
8         if(p[i]) b[cnt++] = p[i]; // 重构新的没有0的线性基
9     // 注意, 是cnt++, 因为要从第0位开始
10 }
```

C++

2.3.1 查询原数组子集的最大异或和

从线性基最高位开始找, 初始化为 $ans = 0$

如果当前位为0, 并且线性基这一位存在值, 那么令 $ans = ans \oplus p_x$

反之继续往下找

如果求的是原数组子集的异或和与某个数 c 异或的最大值, 那么 $ans = c$ 即可

2.3.2 查询原数组子集的最小异或和

如果初始值不为零, 即选一些数和某个固定的数异或求最小, 那就要从高位到低位进行不断xor, 比如 now 在某一一位为1且这一位 $p[i]$ 不为零, 那么 $now = now \oplus p[i]$ 即可

```
1 now = 0; // 一般now初始都是0
2 for(int i = MaxP; i ≥ 0; i--) {
3     if(p[i] && (now >> i & 1)) {
4         now = now ^ p[i];
5     }
6 }
```

C++

2.3.3 寻找第 k 小值和第 k 大值 (从小到大第 k 个)

把原线性基经过高斯消元后得到新的“阶梯状”线性基

比如原线性基为: (不是阶梯状)

```

1 1001111001
2 0100110000
3 0011010011
4 0001111010
5 0000000000
6 0000010000

```

C++

高斯消元后为：（是阶梯状，并且把为0的去掉）

```

1 1000000011
2 0100100000
3 0010101001
4 0001101010
5 0000010000

```

C++

这样一来如果 $res = p_x \oplus p_y \dots$

那么 res 就是从小到大第 $2^x + 2^y \dots$ 个

那么就显而易见该怎么做了

Code

```

1 void rebuild() { // 高斯消元重构线性基
2     for(int i = 63; i ≥ 0; i--) {
3         for(int j = i - 1; j ≥ 0; j--) {
4             if((p[i] >> j) & 1) p[i] ^= p[j];
5         }
6     }
7     for(int i = 0; i ≤ 63; i++)
8         if(p[i]) b[cnt++] = p[i]; // 重构新的没有0的线性基
9     // 注意，是cnt++，因为要从第0位开始
10 }
11 int find_kmin(int k) { // 找的是第k小
12     // 最多只能构造2^{cnt}-1个（包括0（空集）的话就是2^{cnt}个）
13     if(k ≥ (1 << cnt)) return -1;
14     int ans1 = 0, ans2 = 0;
15     for(int i = 0; i < cnt; i++) { // 只有cnt个所以是小于
16         if((k >> i) & 1) ans1 ^= b[i];
17     }
18     return ans;
19 }
20 int find_kmax(int k) { // 找的是第k大
21     // 最多只能构造2^{cnt}-1个（包括0（空集）的话就是2^{cnt}个）
22     k = (1 << cnt) - 1 - k + 1; // 第k大则为2^{cnt}-1-k+1小
23     // 如果空集要算入的话，k=(1<<cnt)-k+1;
24     if(k ≥ (1 << cnt)) return -1;
25     int ans1 = 0, ans2 = 0;
26     for(int i = 0; i < cnt; i++) { // 只有cnt个所以是小于
27         if((k >> i) & 1) ans1 ^= b[i];
28     }
29     return ans;
30 }
31 void insert(int x) { // 线性基构造
32     for(int i = 63; i ≥ 0; i--) {

```

```

33         int f = (x >> i) & 1;
34         if(!f) continue;
35         if(!p[i]) {
36             p[i] = x;
37             return;
38         }
39         x = x ^ p[i];
40     }
41 }
42 insert→rebuild→find_k

```

C++

2.3.4 求XOR和为0的子集个数

子集个数为 2^{n-r} (包含空集)

其中 r 是高斯消元结束后矩阵的秩，源代码中 $r = cnt$ (注意不是 $cnt - 1$ ，因为base 0)

很神奇的定理！！

注意，答案包括了空集！！！！！！

2.3.5 图上最大异或和

求带权无向图从 s 到 t 所有路径中最大的异或和 (可以绕环走)

首先，路径可以拆分为两部分，一条链和 m 个环，再加上 m 条过渡链，即把链和环连接起来的一条链

假设某条路径如下：

```

1  s→s1(链)
2  s1→s2(过渡链)
3  s2→s2(环)
4  s2→s1(过渡链)
5  s1→t(链)

```

C++

那么可以发现过渡链贡献为0，因为经过了两次

所以只需要考虑任意一条链的值为 A ，异或上任意个环，使得其值最大 (过渡链不用考虑)

又发现假如从 s 到 t 有多条链 (A 的值有很多选择)，那么同样这些链也可以构成很多环，最后可以发现**无论选哪条链作为 s 到 t 的链都一样**，因为起点到终点也是环最后可以异或回来

那么把所有链的异或和存入线性基，再随便找一条从 s 到 t 的链的值为 A

接下来就是求最简单的异或和最大问题了

Code

```
1 void dfs(int u, int res) {
2     d[u] = res;
3     vis[u] = true;
4     for(int i = fi[u]; i; i = ne[i]) {
5         int v = to[i];
6         if(!vis[v]) dfs(v, res ^ w[i]);
7         else insert(res ^ w[i] ^ d[v]); // 存入线性基
8         // 如果已经找过了说明存在环
9         // res^d[v]意思是把共同走过的路消掉,剩下的就是环上的路了,再加上(u,v)这条边,即
        res^d[v]^w[i]
10    }
11 }
```

C++

2.3.6 树上最大异或和

如果有 q 条路径,每次询问从某条路径上选一些点权,使得异或和最大,如何做?

首先一遇到树上问题,十有八九需要用倍增

$g[u][i][0 \rightarrow 63]$ 表示从 u 的父亲节点开始到 u 往上 2^i 节点这条路径上点权的线性基

(注意,是从父亲节点开始,因为 $f[i][0]$ 为父亲节点,所以后续使用要注意本身的点权没有算入)

2.4 GF(3)意义下的异或线性基 (三进制)

数组中每个数能用多次

基本代码:

```
1 int Bit(int x, int t){
2     return (x / p[t]) % 3;
3 }
4 int Xor(int x, int y) {
5     int ans = 0;
6     for(int i = 0; i ≤ 37; i++) {
7         ans += p[i] * ((x + y) % 3);
8         x /= 3;
9         y /= 3;
10        if(!x && !y) break;
11    }
12    return ans;
13 }
14 void insert(int x){
15     for(int i = 37; i ≥ 0; i--) {
16         int v = Bit(x, i);
17         if(!v) continue;
18         if(!vis[i]){
19             vis[i] = 1;
20             if(v == 2) x = Xor(x, x);
21             num[i] = x;
22             return;
23         }
```

```

23     }
24     if(v == 1) x = Xor(x,x);
25     x = Xor(x, num[i]);
26 }
27 return;
28 }
29 bool query(int x){//查询某个数是否能被组合出来
30     for(int i = 37; i ≥ 0; i--) {
31         if(!Bit(x, i)) continue;
32         if(!vis[i]) return false;
33         while(Bit(x, i)) x = Xor(x, num[i]);
34     }
35     if(!x) return true;
36     return false;
37 }
38 void solve() {
39     for(int i = 1; i ≤ n; i++) {
40         x = read();
41         insert(x);
42     }
43 }

```

C++

3. 求质因数

3.1 单个数求质因数

假设一个数 $X \in [1, 10^{14}]$ ，如何求它的质因数？

把所有质数保存下来显然是不可能

可以发现， X 必定不可能是两个 $> 10^7$ 的**因数**相乘

否则就会超过范围

那么我们只需要找到 $[1, 10^7]$ 范围内的质因数即可

每找到一个质因数我们就除干净

最后剩的那一部分在单独判断是否为质数即可

时间复杂度 $O(\sqrt{N})$

Code

```

1 void is_prime(int x) {
2     if(x == 1) return false; // 注意为1的不是质数，要特判
3     for(int i = 2; i ≤ sqrt(x); i++)

```

```

4     if(x % i == 0) return false;
5     return true;
6 }
7 ll nx = n;
8 for(int i = 2; i ≤ sqrt(n); i++) {
9     if(n % i ≠ 0) continue;
10    if(vis[i]) continue; //不是质数
11    d[++cnt] = i; //记录质因数
12    while(true) {
13        if(nx % i ≠ 0) break;
14        nx /= i;
15    }
16 }
17 if(is_prime(nx)) d[++cnt] = nx; //判断最后一部分是否为质数，暴力判断即可
18 //注意判断的时候，1要特判！！

```

C++

3.2 多个数求质因数

假设现在有 $n = 1e6$ 个数，每个数 $a_i \leq 5e6(V)$

如何求这些数的质因数？

首先预处理，用 $sp[i]$ 表示数 i 的某一个质因数（为了方便这里取最小或者最大的质因数）

对于需要分解的数 x ，不断递归分解它的质因数即可

时间复杂度： $O((N + V) * \log(V))$

Code

```

1  int N = 5e6;
2  /*预处理*/
3  void init() {
4      sp[1] = 0;
5      for(int i = 2; i ≤ N; i++) {
6          if(!sp[i]) { //i为质数
7              for(int j = 1; ; j++) {
8                  if(i * j > N) break;
9                  sp[i * j] = i; //最大质因数
10             }
11         }
12     }
13 }
14 /*分解*/
15 void factorize(int x) {
16     while(true) {
17         if(x == 1) break;
18         int nowp = sp[x], numx = 0;
19         while(true) {
20             if(x % nowp == 0) {
21                 x /= nowp;
22                 numx++;
23             }

```

```

24         else break;
25     }
26     //nowp是一个质因数，且不会重复
27     //numx是这个质因数的个数
28 }
29 }
30 n = read();
31 for(int i = 1; i ≤ n; i++) {
32     a[i] = read();
33     factorize(a[i]);
34 }

```

C++

4. 线性筛&埃氏筛

4.1 线性筛（欧拉筛）

时间复杂度 $O(N)$

```

1  for(int i = 2; i ≤ 1e7; i++) {
2      if(!vis[i]) p[++num] = i;
3      for(int j = 1; j ≤ num; j++) {
4          if(i * p[j] > 1e7) break;
5          vis[i * p[j]] = true;
6          if(i % p[j] == 0) break;
7      }
8  }

```

C++

实际上对于 $N < 10^8$ 而言，线性筛和埃氏筛在性能上表现接近，都可以用

4.2 埃氏筛

时间复杂度 $O(N * \ln(\ln N))$ （时间复杂度证明见[一些结论](#)）

```

1  int N0 = 1e7 + 100;
2  for(int i = 2; i ≤ N0; i++) {
3      if(!vis[i]) {
4          p[++cnt] = i;
5          for(int j = 2; j ≤ N0; j++) {
6              if(i * j > N0) break;
7              vis[i * j] = i;
8          }
9      }
10 }

```

C++

5. 欧拉函数&欧拉降幂

5.1 欧拉函数

欧拉函数 $\varphi(n)$ 为 $[1, n-1]$ 中与 n 互质的数的个数

比如 $\varphi(6) = 2(\gcd(1, 6) = \gcd(5, 6) = 1)$

特别的, 若 n 为质数, 则 $\varphi(n) = n - 1$

5.2 前置: 欧拉定理

伟大的公式: $a^{\varphi(M)} \equiv 1 \pmod{M}$

(当 $\gcd(a, M) = 1$ 的时候成立)

5.3 欧拉降幂

众所周知, 当 a^b 非常大的时候, 可以运用快速幂计算

但若是 b 也非常大呢?

假设我们需要求一个式子: $a^b \bmod M = ?$

假设 $\gcd(a, M) = 1$, 那么显然 $a^b \bmod M = a^{b \% \varphi(M)} \bmod M$

若 $\gcd(a, M) \neq 1$, 那么直接给出答案 (我也不会推)

通项公式: $a^b \bmod M = a^{b \% \varphi(M) + \varphi(M)} \bmod M$

特殊情况 (一般都是特殊情况): 当 M 为质数的时候, $a^b \bmod M = a^{b \% (M-1)} \bmod M$

```
1 \varphi 欧拉函数
```

C++

5.4 例题

[\[蓝桥杯 2025 国 A\] 斐波那契数列](#)

思路: 矩阵快速幂+欧拉降幂

6. 逆元的多种求法

6.6 进阶版可以处理几乎所有情况!

6.1 Method 1

正序线性求逆元，时间复杂度 $O(n)$

适用：预处理， n 不是很大的时候

```
1 inv[1] = 1;
2 for (int i = 2; i ≤ n; i++) {
3     inv[i] = (Mod - Mod / i) * inv[Mod % i] % Mod;
4 }
```

C++

6.2 Method 2

逆序线性求逆元

但是注意，此处逆元求的是 阶乘 的逆元!!

只有在求组合数才会用到

时间复杂度 $O(n)$

```
1 fac[0] = 1;
2 for (int i = 1; i ≤ n; i++) fac[i] = (fac[i - 1] * i) % mod;
3 inv_fac[n] = ksm(fac[n], Mod - 2); //费马小定理求逆元
4 for (int i = n - 1; i ≥ 0; i--) {
5     inv_fac[i] = inv_fac[i + 1] * (i + 1) % Mod;
6 }
```

C++

6.3 Method 3

求单个逆元，时间复杂度 $O(\log(n))$

条件：Mod为质数

```
1 inv = ksm(a, Mod - 2); //a的逆元
```

C++

$$a^{\varphi(M)} \equiv 1 \pmod{M}$$

所以通项为： $inv = ksm(a, \varphi(M) - 1)$ ，条件： $gcd(a, M) = 1$

6.4 Method 4

如果模数不为质数，那只能用`method4`或者`method3`的通项公式！

求单个逆元，时间复杂度 $O(\log(n))$

适用于所有情况

求 a 在 Mod 意义下的逆元

用`Exgcd`求： $a * x + Mod * y = gcd(a, Mod)$ 令 $t = gcd(a, Mod)$ ， x 即为逆元

注意，若 t 不等于1则 a 在 Mod 意义下无逆元

```
1 void ExGcd(ll a, ll b, ll &x, ll &y) {
2     if (b == 0) {
3         x = 1;
4         y = 0;
5         return;
6     }
7     ll x1, y1;
8     ExGcd(b, a % b, x1, y1);
9     x = y1;
10    y = x1 - (a / b) * y1;
11 }
12 ll GetInv(ll a, ll Mod) {
13     ll x, y;
14     ExGcd(a, Mod, x, y);
15     if(x < 0) x += Mod;
16     return x;
17 }
```

C++

6.5 Method 5

如果要求 $a^0, a^1 \dots a^n$ 对于 M (质数)的逆元

```
1 inv[0] = 1;
2 inv[1] = ksm(a, M - 2);
3 for(int i = 2; i ≤ n; i++) {
4     inv[i] = inv[i - 1] * inv[1];
5     inv[i] %= M;
6     // 逆元可以相乘
7 }
```

C++

6.6 进阶版： $O(n)$ 求 n 个数的逆元

求 n 个数在 mod 意义下的逆元 ($n = 1e7$)

现在计算前 n 个数的前缀积，记为 $s_i = \prod_{j=1}^i a_j = s_{i-1} * a_i$

然后使用快速幂或扩展欧几里得法计算求 s_n 的逆元，记为 sv_n

因为 sv_n 是 n 个数的积的逆元，所以当我们把它乘上 a_n 时，就会和 a_n 的逆元抵消，于是就得到了 a_i 到 a_{n-1} 的积的逆元，记为 sv_{n-1}

同理我们可以依次计算出所有的 sv_i 于是 a_i^{-1} 就可以用 $s_{i-1} \times sv_i$ 求得

时间复杂度: $O(n + \log(mod))$

```
1 for(int i = 1; i <= n; i++) cin >> a[i];
2 s[0] = 1;
3 for (int i = 1; i <= n; ++i) s[i] = s[i - 1] * a[i] % p;
4 sv[n] = ksm(s[n], p - 2);
5 for (int i = n; i >= 1; --i) sv[i - 1] = sv[i] * a[i] % p;
6 for (int i = 1; i <= n; ++i) inva[i] = sv[i] * s[i - 1] % p;
```

c++

7. 快速乘&快速幂

7.1 快速乘

又称：龟速乘

7.1.1 用途

- 大数乘法
- 模运算优化
- 避免数值溢出

7.1.2 代码

```
1 long long fast_multiply(long long a, long long b, long long p) {
2     long long res = 0;
3     a = (a + p) % p; // 十分重要!!!
4     while (b > 0) {
5         if (b & 1) res = (res + a) % p;
6         a = (a + a) % p;
7         b >>= 1;
8     }
9     return res;
10 }
```

cpp

7.2快速幂

7.2.1 用途

- 大数幂运算
- 模幂运算优化
- 提高幂运算效率

7.2.2 代码

```
1 ll ksm(ll x, ll y) {
2     ll ans = 1;
3     x = (x + M) % M;
4     while(y) {
5         if(y & 1) ans = (ans * x) % M;
6         x = (x * x) % M;
7         y >>= 1;
8     }
9     return ans;
10 }
```

cpp

8. 矩阵快速幂（矩阵加速）

如何快速的求一个矩阵的 y 次方？

如：

$$\begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}^y$$

其中 $\begin{bmatrix} 1 & 2 \end{bmatrix}$ 是初始答案矩阵，即答案矩阵初始值为 $\begin{bmatrix} 1 & 2 \end{bmatrix}$

其实和快速幂是一个道理，只有在二进制下为1的才需要操作

$A^0 = I$ (单位矩阵)

8.1 Code

```
1 void getf() {
2     for(int i = 1; i <= nf; i++)
3         for(int j = 1; j <= mf; j++)
4             now[i][j] = 0; // 重置过渡矩阵
5     for(int i = 1; i <= nf; i++)
6         for(int j = 1; j <= mf; j++)
7             for(int k = 1; k <= nf; k++) {
8                 now[i][j] += f[i][k] * f[k][j];
9             }
```

```

9         now[i][j] %= M;
10        now[i][j] = (now[i][j] + M) % M;
11    }
12    for(int i = 1; i ≤ nf; i++)
13        for(int j = 1; j ≤ mf; j++)
14            f[i][j] = now[i][j]; // 重新赋值
15 }
16 void getans() {
17     for(int i = 1; i ≤ na; i++)
18         for(int j = 1; j ≤ ma; j++)
19             now[i][j] = 0; // 重置过渡矩阵
20     for(int i = 1; i ≤ na; i++)
21         for(int j = 1; j ≤ mf; j++)
22             for(int k = 1; k ≤ ma; k++) {
23                 now[i][j] += ans[i][k] * f[k][j];
24                 now[i][j] %= M;
25                 now[i][j] = (now[i][j] + M) % M;
26             }
27     for(int i = 1; i ≤ na; i++)
28         for(int j = 1; j ≤ ma; j++)
29             ans[i][j] = now[i][j]; // 重新赋值
30 }
31 void ksm_matrix(int y) { // y次幂
32     while(y) {
33         if(y & 1) getans(); // 更新ans矩阵
34         getf(); // 更新f矩阵
35         y >>= 1;
36     }
37 }
38 signed main() {
39     int y = read();
40     na = read(), ma = read();
41     nf = read(), mf = read();
42     // ma=nf=mf
43     for(int i = 1; i ≤ nf; i++)
44         for(int j = 1; j ≤ mf; j++)
45             f[i][j] = read(); // 初始矩阵f, 求的也是这个矩阵的y次方再乘ans
46     for(int i = 1; i ≤ na; i++)
47         for(int j = 1; j ≤ ma; j++)
48             ans[i][j] = read(); // 初始答案矩阵
49     ksm_matrix(y);
50     return 0;
51 }

```

C++

8.2 特殊矩阵快速幂

如果遇到类似于下述情况，可能就需要用到**特殊矩阵快速幂**

例如，推出来 dp 转移方程为 $(i \leq R)$ ：

$$dp_{i,s} = \max(dp_{i-1,t} + f_s)$$

或

$$dp_{i,s} = \max(dp_{i-1,t} + f_{t,s})$$

(可以把 f_s 写成 $f_{t,s}$)

即：转移方程至于上一次有关，且 R 非常大

那么，定义矩阵乘法为： $(A \cdot B)_{i,j} = \max(A_{i,k} + B_{k,j})$

然后求出 $f_{t,s}^{R-2}$ (特判 $R = 1$)

初始化： $ans_{i,j} = f_{i,j}$ ，求的是 $ans \cdot f^{R-2}$

最后 $dp_{R,i} = \max(dp_{1,j} + ans_{j,i})$

注意，如果用的是状压 dp ，那矩阵乘法是从0开始的！

Code

```
1 void getf() {
2     //注意，如果用的是状压dp，那矩阵乘法是从0开始的！
3     for(int i = 1; i ≤ nf; i++)
4         for(int j = 1; j ≤ mf; j++)
5             now[i][j] = -INF; //重置过渡矩阵，记得重置为-INF
6     //求min的话重置为INF
7     for(int i = 1; i ≤ nf; i++)
8         for(int j = 1; j ≤ mf; j++)
9             for(int k = 0; k ≤ nf; k++)
10                now[i][j] = max(now[i][j], f[i][k] + f[k][j]);
11    //新定义下的矩阵乘法
12    for(int i = 1; i ≤ nf; i++)
13        for(int j = 1; j ≤ mf; j++)
14            f[i][j] = now[i][j]; //重新赋值
15 }
16 void getans() {
17     for(int i = 1; i ≤ na; i++)
18         for(int j = 1; j ≤ ma; j++)
19             now[i][j] = -INF; //重置过渡矩阵，记得重置为-INF
20    //求min的话重置为INF
21    for(int i = 1; i ≤ na; i++)
22        for(int j = 1; j ≤ mf; j++)
23            for(int k = 1; k ≤ ma; k++)
24                now[i][j] = max(now[i][j], ans[i][k] + f[k][j]);
25    //新定义下的矩阵乘法
26    for(int i = 1; i ≤ na; i++)
27        for(int j = 1; j ≤ ma; j++)
28            ans[i][j] = now[i][j]; //重新赋值
29 }
30 void ksm_matrix(int y) { //y次幂
31     while(y) {
32         if(y & 1) getans(); //更新ans矩阵
33         getf(); //更新f矩阵
34         y >>= 1;
35     }
36 }
37 void solve() {
38     na = ma = nf = mf = read();
39     R = read();
40     for(int i = 1; i ≤ na; i++) {
```

```

41     for(int j = 1; j ≤ na; j++) {
42         ans[i][j] = f[i][j] = read();
43     }
44 }
45 if(R ≥ 2) {
46     ksm_matrix(R - 2);
47 }
48 }

```

C++

9. 矩阵的积和式（二分图完美匹配方案数）

已知行列式 $\det(A) = \sum_{i=1}^n (-1)^{i+j} a_{i,j} M_{i,j}$ ，对于行列式，有符号变换 $(-1)^{i+j}$

对于积和式， $\text{perm}(A) = \sum_{i=1}^n a_{i,j} * \text{perm}(A_{i,j})$ （ $A_{i,j}$ 是删掉*i*行*j*列的子矩阵），积和式没有符号变化

特别的，在 $\text{mod} = 2$ 的情况下， $\det(A) = \text{perm}(A)$

9.1 二分图完美匹配

假设 Alice 有 n 个物品要和 Bob 的 n 个物品一一对应

对应关系为一个矩阵 A ， $a_{i,j} = 1$ 表示 Alice 的第 i 个物品能和 Bob 的第 j 个对应，反之代表不能与之对应

那么 $\text{perm}(A)$ 即为答案数

9.2 求解方法

对于 $\text{perm}(A)$ ，目前没有好的方法，最佳方案时间复杂度为 $O(n * 2^n)$

Code(在 $\text{mod} = 1e9 + 7$ 的意义下)

```

1  const int MOD = 1e9 + 7;
2
3  int permanent_mod(const vector<vector<int>>& A) { // 矩阵A
4      int n = A.size(); // A的大小
5      long long res = 0;
6
7      for (int mask = 0; mask < (1 << n); ++mask) {
8          long long prod = 1;
9          int bits = __builtin_popcount(mask);
10         for (int i = 0; i < n; ++i) {
11             long long sum = 0;
12             for (int j = 0; j < n; ++j) {

```

```

13         if (mask & (1 << j))
14             sum = (sum + A[i][j]) % MOD;
15     }
16     prod = prod * sum % MOD;
17 }
18 if (bits % 2 == 0)
19     res = (res + prod) % MOD;
20 else
21     res = (res - prod + MOD) % MOD;
22 }
23
24 return res;
25 }

```

C++

其他适用情况（实际例子）

假如 *Alice* 第 i 个物品能对应 *Bob* 第 $[l_i, r_i]$ 个物品，在 $\text{mod} = 2$ 的意义下计算 $\text{perm}(A)$ 的奇偶性

那么只需要建一张图， $(l_i - 1)$ 向 (r_i) 连一条边， $n + 1$ 个点 $(0 \rightarrow n)$ ， n 条边

如果构成一棵树，说明线性无关， $\text{perm}(A) = \det(A) = 1 (\text{mod} = 2)$

反之，线性相关， $\text{perm}(A) = 0$

10. 超高效求质因数：Pollard-Rho 算法

通过随机化能在 $O(N^{\frac{1}{4}} * \log N)$ 时间内求出数 N 的所有质因数

此为最坏时间复杂度，实际情况可能更快

并不用记住

10.1 Code

```

1  #include<bits/stdc++.h>
2  #define ll long long
3  using namespace std;
4  ll read() {
5      ll x = 0, f = 1; char ch = getchar();
6      while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
7      while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
8      return x * f;
9  }
10
11 // 大数乘法取模（防止溢出）
12 // 用龟速乘的话时间复杂度可能不行，但是下面这个函数精度可能不够
13 // 视情况而定用哪个，或者直接开__int128

```

```

14 ll multiply_mod(ll a, ll b, ll mod) {
15     return (a * b - (ll)((long double)a / mod * b + 0.5) * mod + mod) % mod;
16 }
17
18 // 快速幂取模
19 ll ksm(ll x, ll y, ll mod) {
20     ll ans = 1 % mod;
21     x %= mod;
22     while (y > 0) {
23         if (y & 1) {
24             ans = multiply_mod(ans, x, mod);
25         }
26         x = multiply_mod(x, x, mod);
27         y >>= 1;
28     }
29     return ans;
30 }
31
32 // Miller-Rabin 素性测试使用的基
33 const int test_bases[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 31};
34 const int test_bases_count = 9;
35 // 如果答案不对, 这里可以再扩张一点
36
37 // Miller-Rabin 素性测试
38 bool is_prime(ll n) {
39     if (n == 1) return false;
40     if (n == 2) return true;
41     if (n % 2 == 0) return false;
42
43     // 检查小素数
44     for (int i = 0; i < test_bases_count; i++) {
45         if (n == test_bases[i]) return true;
46         if (n % test_bases[i] == 0) return false;
47     }
48
49     // 分解  $n-1 = d * 2^s$ 
50     ll d = n - 1;
51     int s = 0;
52     while (d % 2 == 0) {
53         d /= 2;
54         s++;
55     }
56
57     // 对每个基进行测试
58     for (int i = 0; i < test_bases_count; i++) {
59         ll a = test_bases[i];
60         ll x = ksm(a, d, n);
61         if (x == 1 || x == n - 1) continue;
62
63         bool composite = true;
64         for (int j = 0; j < s - 1; j++) {
65             x = multiply_mod(x, x, n);
66             if (x == n - 1) {
67                 composite = false;
68                 break;
69             }
70         }
71         if (composite) return false;

```

```

72     }
73     return true;
74 }
75
76 // 随机数生成
77 ll get_random(ll n) {
78     return (ll)rand() << 32 | rand();
79 }
80
81 // Pollard-Rho 算法的迭代函数
82 ll pollard_rho_function(ll x, ll c, ll mod) {
83     return (multiply_mod(x, x, mod) + c) % mod;
84 }
85
86 // Pollard-Rho 算法实现
87 ll pollard_rho(ll n) {
88     if (n % 2 == 0) return 2;
89     if (n % 3 == 0) return 3;
90     if (n % 5 == 0) return 5;
91
92     while (true) {
93         ll c = get_random(n - 1) % (n - 1) + 1;
94         ll x = get_random(n) % n;
95         ll y = x;
96         ll d = 1;
97
98         for (int cycle_size = 1; d == 1; cycle_size *= 2) {
99             ll x_saved = x;
100             for (int i = 0; i < cycle_size && d == 1; i++) {
101                 x = pollard_rho_function(x, c, n);
102                 y = pollard_rho_function(pollard_rho_function(y, c, n), c, n);
103                 d = __gcd(abs(x - y), n);
104             }
105             if (d != 1 && d != n) return d;
106             x = x_saved;
107         }
108     }
109 }
110
111 vector<ll> factors;
112
113 // 分解质因数
114 void factorize(ll n) {
115     if (n == 1) return;
116     if (is_prime(n)) {
117         factors.push_back(n);
118         return;
119     }
120
121     ll divisor = pollard_rho(n);
122     factorize(divisor);
123     factorize(n / divisor);
124 }
125
126 int main() {
127     srand(time(0));
128     int T = read();
129     while (T--) {

```

```

130         ll n;
131         cin >> n;
132         if (is_prime(n)) {
133             cout << "Prime" << endl;
134             continue;
135         }
136         factors.clear();
137         factorize(n);
138
139         // 排序并输出质因数
140         sort(factors.begin(), factors.end());
141         for (size_t i = 0; i < factors.size(); i++) {
142             if (i > 0) cout << " ";
143             cout << factors[i];
144         }
145         cout << endl;
146     }
147
148     return 0;
149 }

```

C++

11. 多项式乘法FFT

问题：求 $A(x) * B(x) = ?$

$A(x), B(x)$ 都是多项式，一个是 n 阶，一个是 m 阶

如果正常求肯定是 $O(n * m)$ 时间复杂度

如何优化？

11.1 Solution

已知：

1. 一个 n 阶多项式可以被 $n + 1$ 个点确定，即如果知道 $n + 1$ 个点可以推出多项式表达式（正常推时间复杂度也是 $O(n^2)$ ）
2. $C(x_i) = A(x_i) * B(x_i)$ ，即点 $(x_i, A(x_i)) * (x_i, B(x_i)) = (x_i, C(x_i))$

那么我们可以高效预处理 $\geq n + m + 1$ 个点，求出 $A(x_i)$ 以及 $B(x_i)$

再高效转换为 $C(x_i)$ 即可

如何做到？这里就不证明了，，，

a. 通过傅里叶变化可以变成分治求答案（需要运用到复数，即把坐标用复数表示，`complex` 类）

b. 再加上一个优化过程即可做到 $n \log(n)$ 求答案

横坐标为 $N = 2^M$ 个 $x_0 \dots x_{N-1}$, 其中 $x_i = (\cos(\pi/(2 * len)), \sin(\pi/(2 * len)))$ (用复数表示)

(分为 2^M 个点原因是这样能更好的分治, 使得每次都能完美分治)

逆转换时, $x_i = (\cos(\pi/(2 * len)), -\sin(\pi/(2 * len)))$

直接给代码

11.2 Code

非必要不要手写STL, complex类, 除非卡你STL

```
1  #include<bits/stdc++.h>
2  #include<cmath>
3  #define ll long long
4  using namespace std;
5  const ll N = 4e6 + 100, M = 1e9 + 7, INF = 1e13;
6  int f[N], Max = 1, cnt;
7  const double pi = acos(-1.0); // 计算 π 的值
8  complex<double> a[N], b[N]; // stl的complex类
9  ll read() {
10     ll x = 0, f = 1; char ch = getchar();
11     while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
12     while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
13     return x * f;
14 }
15 void FFT(complex<double> *a, int type) { // type 代表类型, 同时也是系数
16     for (int i = 0; i < Max; i++) {
17         if(i < f[i]) swap(a[i], a[f[i]]);
18         // 提前变换区间简化操作
19     }
20     for(int mid = 1; mid < Max; mid <= 1) { // mid代表长度len的一半
21         double wx = pi / (double)mid;
22         // 正常来说wx=pi/(len/2), len=mid*2, 这里简化了
23         complex<double> W(cos(wx), type * sin(wx));
24         int len = mid << 1; // 区间长度
25         for(int j = 0; j < Max; j += len) { // 枚举每个区间起点
26             complex<double> w(1, 0);
27             for(int k = 0; k < mid; k++, w = w * W) { // 每个区间枚举前半
28                 complex<double> u = a[j + k];
29                 complex<double> v = w * a[j + k + mid];
30                 a[j + k] = u + v;
31                 a[j + k + mid] = u - v;
32             }
33         }
34     }
35 }
36 int main() {
37     int n = read(), m = read();
38     for(int i = 0; i ≤ n; i++) a[i] = read();
39     for(int i = 0; i ≤ m; i++) b[i] = read();
40     Max = 1, cnt = 0; // 恢复初始值
41     while(Max ≤ (n + m)) Max <= 1, cnt++;
42     for(int i = 0; i < Max; i++)
43         f[i] = (f[i >> 1] >> 1) | ((i & 1) << (cnt - 1));
```

```

44     FFT(a, 1); // 正运算求点坐标
45     FFT(b, 1); // 对b正运算换成坐标
46     for(int i = 0; i ≤ Max; i++) a[i] = a[i] * b[i];
47     FFT(a, -1); // 逆运算 计算系数
48     for(int i = 0; i ≤ n + m; i++) {
49         printf("%d ", (int)(a[i].real() / Max + 0.5));
50         // 最后答案记得除以 2^M
51         // 以防精度不够, 最后 + 0.5
52     }
53     return 0;
54 }
55 // 月雪·薇婷

```

C++

讲一下 f_i 如何 $swap$ 来做到优化的

首先 FFT 通过分治来求解, 通过区分奇偶幂来分治 (即下标区分, i 代表 x^i)

其次 FFT 需要 $2^M > (n + m + 1)$ 个点, 求出最小的 M

比如说最开始下标分别为0, 1, 2, 3, 4, 5, 6, 7

那么就分为两组, 0, 2, 4, 6和1, 3, 5, 7 (共8个点)

下标都重新排为0, 1, 2, 3

由此推下去, 顺序如果变为0, 4, 1, 3, 1, 5, 3, 7

这样不用排序, 直接从左往右扫一遍就行

12. 多项式乘法NTT

12.1 Solution

假设模数满足生成原根的条件

那么我们可以用 $g_1, g_2 \dots g_{2^M}$ 来替代 FFT 中的 $w_n^1, w_n^2 \dots w_n^{2^M}$

需特别注意, $(p-1) \% 2^M$ 必须为1

例如, 对于常见模数 $998244353 = 119 * 2^{23} + 1$

所以长度 $2^M \leq 2^{23}$

其中 $g_i = g^{(p-1)/2^i}$

(g 是原根, w 是复数)

相比于 FFT , NTT 的无须担心精度问题, 但前提条件是模数能生成原根

具体步骤：

1. 生成原根（参见原根知识点，可以提前求出来）及其单位根和它们的逆元
2. 正运算
3. $C(x) = A(x) * B(x)$
4. 逆运算
5. 归一化：除以 2^M 得到实际答案（实际上乘上 2^M 的逆元）

注意！NTT的逆运算用的是乘上 g^i 的逆元，正运算直接乘 g^i

注意！NTT的模数最好只能是998244353，且 $\text{mod} \% D = 1$ （ D 含义见代码）， g 是原根

只要存在这样一个 D ，且有原根，那么这个模数就是友好模数

如果不是友好模数，则需要用任意模数FFT

12.2 Code

```
1  #include<bits/stdc++.h>
2  #include<cmath>
3  #define ll long long
4  using namespace std;
5  const ll N = 4e6 + 100, M = 1e9 + 7, INF = 1e13;
6  const ll D = pow(2, 15);
7
8  ll mod, f[N], wx[N], inv_w[N], g = 3, a[N], b[N];
9  ll n, m, Max, cnt;
10
11 ll read() {
12     ll x = 0, f = 1; char ch = getchar();
13     while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
14     while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
15     return x * f;
16 }
17 ll ksm(ll x, ll y) {
18     ll ans = 1;
19     while(y) {
20         if(y & 1) ans = (ans * x) % mod;
21         x = (x * x) % mod;
22         y >>= 1;
23     }
24     return ans;
25 }
26 void init() {
27     for(int i = 1; i ≤ 40; i++) {
28         wx[i] = ksm(g, (mod - 1) / (1 << i));
29         inv_w[i] = ksm(wx[i], mod - 2);
30     }
31 }
32
33 void NTT(ll *a, int type) {
34     for (int i = 0; i < Max; ++i) {
```

```

35     if(i < f[i]) swap(a[i], a[f[i]]);
36 }
37 for(int mid = 1, tx = 1; mid < Max; mid <= 1, tx++) {
38     ll W = 0;
39     if(type == 1) W = wx[tx];
40     else W = inv_w[tx];
41     ll len = mid << 1; // 区间长度
42     for(int j = 0; j < Max; j += len) {
43         ll w = 1;
44         for(int k = 0; k < mid; k++) { // 每个区间枚举前一半
45             ll u = a[j + k];
46             ll v = (w * a[j + k + mid]) % mod;
47             a[j + k] = (u + v) % mod;
48             a[j + k + mid] = (u - v + mod) % mod;
49             w = (w * W) % mod;
50         }
51     }
52 }
53 if(type == -1) {
54     ll inv_n = ksm(Max, mod - 2);
55     for(int i = 0; i <= n + m; i++) a[i] = (a[i] * inv_n) % mod;
56 }
57 }
58 int main() {
59     n = read(), m = read(); mod = read();
60     for(int i = 0; i <= n; i++) a[i] = read();
61     for(int i = 0; i <= m; i++) b[i] = read();
62     Max = 1;
63     while(Max <= (n + m)) Max <= 1, cnt++;
64     for(int i = 0; i < Max; i++)
65         f[i] = (f[i >> 1] >> 1) | ((i & 1) << (cnt - 1));
66
67     // g = Root(mod); // 求原根, 也可以提前求出来, 略过这一步骤
68     // 详情参见 原根知识点
69     g=xxxxxxx; // 原根 g 需要初始化, 通过Root(mod)求
70     // 如果mod=998244353, g=3
71     init(); // 初始化单位根
72
73     NTT(a, 1); NTT(b, 1);
74     for(int i = 0; i <= Max; i++) a[i] = (a[i] * b[i]) % mod;
75     NTT(a, -1);
76     ll inv_M = ksm(Max, mod - 2);
77     for(int i = 0; i <= n + m; i++) {
78         cout << a[i] << " ";
79     }
80     return 0;
81 }
82 // 月雪·薇婷

```

C++

一定要保证 mod 满足以上条件

13. 多项式乘法逆

13.1 Solution

已知 $F(x)$ ，找到一个函数使得 $F(x) * G(x) \equiv 1 \pmod{x^n}$

经过一系列推算，得到

$$G(x) \equiv 2G'(x) - F(x)G'^2(x) \pmod{x^n}$$

其中， $F(x) * G'(x) \equiv 1 \pmod{x^{\frac{n}{2}}}$

那么我们从最下面递推上来即可

其中 $G(x)$ 中常数系数为 $F(x)$ 常数系数的逆元

13.2 Code

一定要注意本篇代码里面原函数 n 代表的是阶数，总体输入数量为 $n + 1$

```
1  #include<bits/stdc++.h>
2  #include<cmath>
3  #define ll long long
4  using namespace std;
5  const ll N = 4e6 + 100, mod = 998244353, INF = 1e13;
6  const ll D = pow(2, 15);
7  ll f[N], wx[N], inv_w[N], g = 3, a[N], b[N];
8  ll n, m, ans[N], now[N];
9  ll c[N], d[N], e[N];
10 ll read() {
11     ll x = 0, f = 1; char ch = getchar();
12     while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
13     while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
14     return x * f;
15 }
16 ll ksm(ll x, ll y) {
17     ll ans = 1;
18     while(y) {
19         if(y & 1) ans = (ans * x) % mod;
20         x = (x * x) % mod;
21         y >>= 1;
22     }
23     return ans;
24 }
25 void init() {
26     for(int i = 1; i ≤ 25; i++) {//最多23个
27         wx[i] = ksm(g, (mod - 1) / (1 << i));
28         inv_w[i] = ksm(wx[i], mod - 2);
29     }
30 }
31 void initx(int Max) {
```

```

32     int cnt = log2(Max);
33     for(int i = 0; i < Max; i++)
34         f[i] = (f[i >> 1] >> 1) | ((i & 1) << (cnt - 1));
35 }
36
37 void NTT(ll Max, ll *b, ll *a, int type) {
38     for(int i = 0; i < Max; i++) a[i] = b[i];
39     for (int i = 0; i < Max; ++i) {
40         if(i < f[i]) swap(a[i], a[f[i]]);
41     }
42     for(int mid = 1, tx = 1; mid < Max; mid <= 1, tx++) {
43         ll W = 0;
44         if(type == 1) W = wx[tx];
45         else W = inv_w[tx];
46         ll len = mid << 1; // 区间长度
47         for(int j = 0; j < Max; j += len) {
48             ll w = 1;
49             for(int k = 0; k < mid; k++) { // 每个区间枚举前半
50                 ll u = a[j + k];
51                 ll v = (w * a[j + k + mid]) % mod;
52                 a[j + k] = (u + v) % mod;
53                 a[j + k + mid] = (u - v + mod) % mod;
54                 w = (w * W) % mod;
55             }
56         }
57     }
58     if(type == -1) {
59         ll inv_M = ksm(Max, mod - 2);
60         for(int i = 0; i < Max; i++) a[i] = (a[i] * inv_M) % mod;
61     }
62 }
63 void Multi(int Max, ll *a, ll *b, ll *c) {
64     initx(Max); // 重新计算交换数组f
65     NTT(Max, a, d, 1); NTT(Max, b, e, 1);
66     // 分别令d=a正运算, e=b正运算
67     for(int i = 0; i < Max; i++) {
68         d[i] = (d[i] * e[i]) % mod;
69     }
70     NTT(Max, d, c, -1); // 用c保存d的逆运算
71     for(int i = 0; i < Max; i++) d[i] = e[i] = 0; // 置零
72 }
73 void Solve(int x) {
74     int len = 1, Max = 4;
75     b[0] = a[0]; // 用b表示a, 代表F系数
76     ans[0] = ksm(a[0], mod - 2); // 常数系数
77     while(len <= n) {
78         for(int i = 0; i < Max; i++) now[i] = ans[i];
79         Multi(Max, ans, ans, c); // 求G(x)*G(x), 用c存下来
80         for(int i = len; i < (len << 1); i++) b[i] = a[i];
81         // 每次区域扩张, b也更新
82         Multi(Max, c, b, c); // 求F(x)*(G(x)*G(x)), 用c存下来
83         for(int i = 0; i < (len << 1); i++) {
84             ans[i] = (2ll * ans[i] - c[i] + mod) % mod;
85             // 整体计算
86         }
87         len <= 1; Max <= 1;
88     }
89 }

```

```

90 }
91 int main() {
92     n = read(); n--;
93     //注意此处代表的为 多少阶的多项式 !!!
94     for(int i = 0; i ≤ n; i++) a[i] = read();
95     g = 3;
96     init();
97     Solve(n);
98     for(int i = 0; i ≤ n; i++) cout << ans[i] << " ";
99     return 0;
100 }
101 //月雪·薇婷

```

C++

14. 分治FFT/NTT

1. $P(x) = \prod_{i=1}^n P_i(x)$, 其中 $P_i(x)$ 阶数很小 (最多阶数不超过10), 如何求? (计算 n 个选 i 个的概率)
2. 已知序列 $g = g_1, g_2 \dots g_n$ 以及 f_0 , 求序列 f , 其中 $f_i = \sum_{j=1}^i f_{i-j} g_j$

14.1 Solution-1

对于第一个问题, 很好想到

直接分治来求, 假设我们已知 $\prod_l^{mid} P_i(x)$ 以及 $\prod_{mid+1}^r P_i(x)$

那么直接相乘便能得到 $\prod_l^r P_i(x)$ 。用 FFT 或者 NTT 加速

注意! 如果模数 $mod = 1e9 + 7$, 那么需要把中间 NTT 过程改成任意模数 FFT ! (去抄模板即可, 稍微理解一下任意模数 FFT)

Code

例题: [百度之星2023·课件设计](#)

Multi() 的 `len` 一定不要忘了, 否则可能会超时

```

1  #include<bits/stdc++.h>
2  #include<cmath>
3  #define ll long long
4  #define int long long
5  using namespace std;
6  const ll N = 4e6 + 100, mod = 998244353, INF = 1e13;
7  const ll D = 1ll << 15; //pow(2,15)
8  ll f[N], wx[N], inv_w[N], g = 3;
9  ll n, m;
10 vector<vector<int>>> fx;
11 ll read() {
12     ll x = 0, f = 1; char ch = getchar();
13     while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}

```

```

14     while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
15     return x * f;
16 }
17 ll ksm(ll x, ll y) {
18     ll ans = 1;
19     while(y) {
20         if(y & 1) ans = (ans * x) % mod;
21         x = (x * x) % mod;
22         y >>= 1;
23     }
24     return ans;
25 }
26 void init() {
27     for(int i = 1; i ≤ 40; i++) {
28         wx[i] = ksm(g, (mod - 1) / (1 << i));
29         inv_w[i] = ksm(wx[i], mod - 2);
30     }
31 }
32 void NTT(vector<ll> &a, int type, int Max) {
33     for (int i = 0; i < Max; ++i) {
34         if(i < f[i]) swap(a[i], a[f[i]]);
35     }
36     for(int mid = 1, tx = 1; mid < Max; mid <= 1, tx++) {
37         ll W = 0;
38         if(type == 1) W = wx[tx];
39         else W = inv_w[tx];
40         ll len = mid << 1; // 区间长度
41         for(int j = 0; j < Max; j += len) {
42             ll w = 1;
43             for(int k = 0; k < mid; k++) { // 每个区间枚举前半
44                 ll u = a[j + k];
45                 ll v = (w * a[j + k + mid]) % mod;
46                 a[j + k] = (u + v) % mod;
47                 a[j + k + mid] = (u - v + mod) % mod;
48                 w = (w * W) % mod;
49             }
50         }
51     }
52     if(type == -1) {
53         ll inv_M = ksm(Max, mod - 2);
54         for(int i = 0; i < Max; i++) {
55             a[i] = (a[i] * inv_M) % mod;
56         }
57     }
58 }
59 vector<ll> Multi(vector<ll> a, vector<ll> b, int len) {
60     int Max = 1;
61     int cnt = 0;
62     while(Max ≤ len) Max <= 1, cnt++;
63     a.resize(Max + 3); b.resize(Max + 3);
64     for(int i = 0; i < Max; i++)
65         f[i] = (f[i >> 1] >> 1) | ((i & 1) << (cnt - 1));
66     NTT(a, 1, Max); NTT(b, 1, Max);
67     for(int i = 0; i < Max; i++) a[i] = (a[i] * b[i]) % mod;
68     NTT(a, -1, Max);
69     return a;
70 }
71 pair<int, vector<ll>> Solve(int l, int r) {

```



```

72 //注意! 如果模数mod=1e9+7$, 那么需要把中间$NTT$过程改成任意模数$FFT$!
73 if(l == r) return {fx[l].size() - 1, fx[l]}; //这里注意可能会改成别的值
74 int mid = (l + r) >> 1;
75 pair<int, vector<int>> ans1 = Solve(l, mid);
76 pair<int, vector<int>> ans2 = Solve(mid + 1, r);
77 int len = ans1.first + ans2.first;
78 vector<int> ansx = Multi(ans1.second, ans2.second, len);
79 ansx.resize(len + 1);
80 return {len, ansx};
81 //Multi的len一定不要忘了, 否则可能会超时
82 }
83 signed main() {
84     g = 3; init();
85     n = read();
86     fx.resize(n + 5);
87     for(int i = 1; i ≤ n; i++) {
88         int sz = read();
89         for(int j = 1; j ≤ sz; j++) {
90             int x = read();
91             fx[i].push_back(x); //第i个多项式是f[i]=\sum fx[i][0]*x^i
92             //顺序别弄反了!!!, 从小到大项数增大
93             //可能不是直接输入, 可能是需要自己构造
94         }
95     }
96     vector<int> ans = Solve(1, n).second;
97     for(int i = 0; i < ans.size(); i++) cout << ans[i] << " ";
98     return 0;
99 }
100 //月雪·薇婷

```

C++

14.2 Solution-2

假设一个区间 $[l, r]$ 我们已经知道了 $f_{l \rightarrow mid}$

那么这一部分对于 $f_{mid+1 \rightarrow r}$ 中每一个系数的贡献为

$$\Delta f_i = \sum_{j=l}^{mid} f_j * g_{i-j}, \text{ 其中 } i \in [mid + 1, r]$$

$g: [1, r - l]$ (g 的范围)

所以求出来区间 $[l, mid]$ 之后可以更新 $[mid + 1, r]$ 中 f 的值

列两个多项式: $A(x)$ 系数为 $f_l, f_{l+1} \dots f_{mid}$

$B(x)$ 系数为 $g_1, g_2 \dots g_{r-l}$

假设 $A(x) * B(x) = \sum_{i=0}^{r-l+mid} c_i x^i$ (c_i 为系数)

那么最后 $f_i = c_i$ ($i \in [mid + 1, r]$)

用NTT实现多项式乘法

然后依次分治求即可

优化方法: $Multi$ 函数里面长度不用 $r - l + mid = 1.5(r - l)$, 只需要 $0.66(r - l)$ 即可, 因为本来NTT或者FFT就会把长度扩大到两倍左右 (我没验证过, 谨慎使用)

伪代码:

```
1 void Solve(int l, int r) {
2     if(l == r) return {
3         if(!l) f[l] = 1; // 规定的f[0]=1
4         return;
5     }
6     int mid = (l + r) >> 1;
7     Solve(l, mid); // 先预处理[l, mid]
8     Multi(f: [l→mid], g: [1→r-l], r-l+1+mid); // 伪代码, []代表计算区间, 需要用一个新的
    vector存下来再计算
9     merge(); // 合并新增值
10    Solve(mid + 1, r); // 再计算[mid+1, r]
11    return;
12 }
```

C++

15. 任意模数多项式乘法(高精度FFT)

假设多项式系数特别大 (最后对其取模), 并且模数不固定 (即无法确定是否能生成原根, 所以不能用NTT), 直接用FFT必定掉精度, 所以需要用特别的方法

15.1 Solution

假设 $F(x) = A(x) + d * B(x)$, $G(x) = C(x) + d * D(x)$

(即 $A(x)$ 系数为 $F(x) \% d$, $B(x)$ 系数为 $F(x) / d$)

那么 $F(x) * G(x) = A(x)C(x) + d * (A(x)D(x) + B(x)C(x)) + d^2 * B(x)D(x)$

若令 $T(x) = C(x) + i * D(x)$

那么通过 $A(x) * T(x)$, $B(x) * T(x)$ 便是能得到上述式子需要的四个表达式

那么就很简单了

1. 对 $A(x)$, $B(x)$, $T(x)$ 各做一次DFT (即FFT正运算)
2. $A(x) * T(x)$, $B(x) * T(x)$
3. 对 $A(x)$, $B(x)$ 各做一次IDFT (即FFT逆运算)
4. 归一化, $A[i].real() / 2^M + 0.5$, 其他三个亦是如此

5. $ans[i] = ar + (ai + br) * d + bi * d^2$, 其中 ar, ai, br, bi 含义见code

注意!! 记得用longdouble, 否则精度仍然有问题

d 的取值, 一般取到 2^{15} , 再大精度可能不准, 非必要不取大

15.2 Code

```
1  #include<bits/stdc++.h>
2  #include<cmath>
3  #define ll long long
4  #define int long long
5  using namespace std;
6  const ll N = 4e6 + 100, M = 1e9 + 7, INF = 1e13;
7  const ll D = pow(2, 15);
8
9  ll mod, f[N], a[N];
10 int n, m, Max, cnt;
11 const long double pi = acos(-1.0); // 计算 π 的值
12 // 记得用long double
13 ll read() {
14     ll x = 0, f = 1; char ch = getchar();
15     while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
16     while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
17     return x * f;
18 }
19
20 complex<long double> A[N], B[N], T[N];
21 void FFT(complex<long double> *a, int type) { // FFT模板
22     for (int i = 0; i < Max; ++i) {
23         if(i < f[i]) swap(a[i], a[f[i]]);
24     }
25     for(int mid = 1; mid < Max; mid <= 1) {
26         complex<long double> W(cos(pi / mid), type * sin(pi / mid));
27         int len = mid << 1;
28         for(int j = 0; j < Max; j += len) {
29             complex<long double> w(1, 0);
30             for(int k = 0; k < mid; k++, w = w * W) {
31                 complex<long double> u = a[j + k];
32                 complex<long double> v = w * a[j + k + mid];
33                 a[j + k] = u + v;
34                 a[j + k + mid] = u - v;
35             }
36         }
37     }
38 }
39 complex<long double> Get(complex<long double> x) {
40     long double xr = x.real(), xi = x.imag();
41     xr = xr / Max + 0.5;
42     xi = xi / Max + 0.5;
43     // 先用long double计算, 以免掉精度
44
45     ll xrr = xr, xii = xi;
46     xrr %= mod, xii %= mod;
47     // +0.5之后能转换为整型, 再取模
```

```

48
49     x.real(xrr); x.imag(xii); //重新赋值
50     return x;
51 }
52 signed main() {
53     n = read(), m = read(), mod = read();
54     for(int i = 0; i ≤ n; i++) {
55         int x = read();
56         A[i].real((x % D));
57         B[i].real((x / D));
58         //初始化
59     }
60     for(int i = 0; i ≤ m; i++) {
61         int x = read();
62         T[i].real((x % D));
63         T[i].imag((x / D));
64     }
65     Max = 1;
66     while(Max ≤ (n + m)) Max <<= 1, cnt++;
67     for(int i = 0; i < Max; i++)
68         f[i] = (f[i >> 1] >> 1) | ((i & 1) << (cnt - 1));
69     FFT(A, 1); FFT(B, 1); FFT(T, 1); //正运算
70
71     for(int i = 0; i < Max; i++) {
72         A[i] = A[i] * T[i];
73         B[i] = B[i] * T[i];
74     }
75     FFT(A, -1); FFT(B, -1); //逆运算
76     for(int i = 0; i ≤ n + m; i++) {
77         A[i] = Get(A[i]); //归一化
78         B[i] = Get(B[i]);
79         ll ar = A[i].real(), ai = A[i].imag();
80         ll br = B[i].real(), bi = B[i].imag();
81         ll ans = (ai + br) * D; ans %= mod;
82         ans = (ans + ar) % mod;
83         ans = (ans + (bi * D * D)) % mod; //计算答案(ans即当前^i的系数)
84         a[i] = ans; //记录答案
85         cout << ans << " "; //这里不一定需要,视情况而定
86     }
87     return 0;
88 }
89 //月雪·薇婷

```

C++

16. 多项式中的一些概念

DFT: 多项式由系数表示法转为点值表示法的过程 (正运算)

IDFT: 把一个多项式的点值表示法转化为系数表示法的过程 (逆运算)

FFT: 就是通过取某些特殊的x的点值来加速DFT和IDFT的过程 (DFT + IDFT结合以及加速运算)

NTT：和*FFT*类似。也是通过*DFT + IDFT*来求答案。区别在于*NTT*通过原根等操作可以避免*FFT*复数出现的精度问题，不过求原根的限制性比较大，时间复杂度也无法确定。一般来说如果模数不固定就只能用*FFT*。

(原根相关可以看一下相关知识以及局限性)

17. 原根

假设有一个模数 p

- $g^{p-1} \equiv 1 \pmod{p}$ (即 g 一定是和 p 互质, 这里就假设 g 为质数)
- $g^1, g^2 \dots g^{p-1} \pmod{p}$ 的集合为 $1, 2, 3 \dots p-1$
- $p = a * P_0^k$, 其中 $a = 1$ 或 2 , P_0 为质数, $k \in \mathbb{N}^*$, 这里假设 p 就是质数

那么称 g 为 p 的原根

17.1 求解方法

- 求解 $p-1$ 的所有质因数 $q_1, q_2 \dots q_k$
- 令 g 从 $2 \rightarrow p-1$, 若有 $j = 1 \rightarrow k$ 全都有 $g^{(p-1)/q_j} \% p \neq 1$, 那么 $g = q_i$ 就是原根

时间复杂度 $O(N \log^2 N)$

注意, 对于固定的模数, 可以提前求出来原根直接用而不用每次都单独求

17.2 Code

```
1  #include<bits/stdc++.h>
2  #include<cmath>
3  #define ll long long
4  using namespace std;
5  const ll N = 4e6 + 100, M = 1e9 + 7, INF = 1e13;
6  ll d[N], P[N];
7  ll read() {
8      ll x = 0, f = 1; char ch = getchar();
9      while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
10     while(ch >= '0' && ch <= '9') {x = x * 10 + ch - '0'; ch = getchar();}
11     return x * f;
12 }
13 void init() { // 筛质数
14     int Nx = 1e6;
15     for(int i = 2; i <= Nx; i++) {
16         if(!vis[i]) {
17             for(int j = 1; j <= tot; j++) {
18                 if(i * P[j] > Nx) break;
```

```

19         vis[i * P[j]] = true;
20         if(i % P[j] == 0) break;
21     }
22 }
23 }
24 bool is_prime(ll x) {
25     if(x == 1) return false; // 特判 1 不是质数!!
26     for(int i = 2; i ≤ sqrt(x); i++) {
27         if(x % i == 0) return false;
28     }
29     return true;
30 }
31 ll ksm(ll x, ll y, ll mod) {
32     ll ans = 1;
33     while(y) {
34         if(y & 1) ans = (ans * x) % mod;
35         x = (x * x) % mod;
36         y >>= 1;
37     }
38     return ans;
39 }
40 void GetRoot(ll n) { // 求质因数以及原根
41     n--; // 求的是n-1的质因数
42     ll nx = n, cnt = 0;
43     for(int i = 2; i ≤ sqrt(n); i++) {
44         if(vis[i]) continue; // 保证是质数
45         if(n % i != 0) continue;
46         d[++cnt] = i; // 记录质因数
47         while(true) {
48             if(nx % i != 0) break;
49             nx /= i;
50         }
51     }
52     if(is_prime(nx)) d[++cnt] = nx; // 最后剩下的暴力判断即可
53     // 上面为求质因数
54     for(int i = 2; i ≤ n; i++) {
55         bool F = true;
56         for(int j = 1; j ≤ cnt; j++) {
57             if(ksm(i, n / d[j], n + 1) == 1) {
58                 F = false;
59                 break;
60             }
61         }
62         if(F) {
63             cout << i << " is a YuanGen\n";
64             break; // 找到一个就退出, 只需要一个
65         }
66     }
67 }
68 int main() {
69     init();
70     ll n = read();
71     GetRoot(n);
72     return 0;
73 }
74 // 月雪·薇婷

```

18. 拉格朗日插值（多项式求值）

假设已知多项式函数 $f(x)$ 的 n 个点 (x_i, y_i) ， $f(x)$ 最高项为 x^n ，且对于 $i \neq j$ ， $x_i \neq x_j$ ，那 $f(x)$ 表达式唯一确定，我们需要求的是 $f(k) = ?$

使用高斯消元时间复杂度为 $O(n^3)$ 且有精度问题，而拉格朗日插值可以把时间复杂度降到 $O(n^2)$ 甚至是 $O(n)$ ，因为求 $f(k)$ 并不用求出系数

Solution

首先，已知 n 个点一定能推出这个多项式，那么我们只需要构造一个式子，使得 $k = x_i$ 的时候式子恰好等于 y_i ，那这个式子就是 $f(x)$

很容易想到 $f(x) = \sum_{i=1}^n y_i \cdot \prod_{j=1, j \neq i}^n \frac{(x-x_j)}{x_i-x_j}$

当 $x = x_c$ 的时候，除了 $i = c$ 的时候，其它项都包含 $x - x_c$ ，所以其它项为0

那么 $f(x_c) = y_c \cdot \prod_{j=1, j \neq c}^n \frac{(x_c-x_j)}{x_c-x_j} = y_c$

所以这个式子即为 $f(x)$ ，即 $f(k) = \sum_{i=1}^n y_i \cdot \prod_{j=1, j \neq i}^n \frac{(k-x_j)}{x_i-x_j}$

直接求的时间复杂度为 $O(n^2)$ （应该不需要给代码了吧，够简洁了，多算一次 Inv 就行了）

特殊优化 $O(n)$

注意，当且仅当 $x_i = i$ 时可以优化

当满足上述条件的时候，原式变为 $\prod_{j=1, j \neq i}^n \frac{k-j}{i-j}$ ，很显然可以发现是一个关于前缀积，后缀积以及阶乘有关问题，需要注意的是要考虑 $i - j \leq 0$ 的时候的奇偶性，需不需要乘以 -1

快速拉格朗日插值·一般优化 $O(n \log^2 n)$

考虑另一个多项式 $P(x) = \prod_{i=1}^n (x - x_i)$

那么显然有 $P'(x_k) = \prod_{i=1, i \neq k}^n (x_k - x_i)$

对于 $P(x)$ 显然可以用分治FFT来做，时间复杂度 $O(n \log n)$

后面需要用到多项式多点求值，暂时不会，以后补充

Code

```
1 #include<bits/stdc++.h>
```

```

2  #include<cmath>
3  #define ll long long
4  #define int long long
5  using namespace std;
6  const ll N = 4e6 + 100, mod = 998244353, INF = 1e13;
7  ll read() {
8      ll x = 0, f = 1; char ch = getchar();
9      while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
10     while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
11     return x * f;
12 }
13 int ksm(int x, int y) {
14     int ans = 1;
15     x = (x + mod) % mod;
16     while(y) {
17         if(y & 1) ans = (ans * x) % mod;
18         x = (x * x) % mod;
19         y >>= 1;
20     }
21     return ans;
22 }
23 struct DC_FFT {
24     static const ll mod = 998244353, g = 3, D = (1ll << 15);
25     static vector<int> work(int n, const vector<vector<int>>> &fx) {
26         DC_FFT sl;
27         fx = _fx;
28         wx.resize(42);
29         inv_w.resize(42);
30         sl.init();
31         vector<int> ans = sl.Solve(1, n).second;
32         return ans;
33     }
34     private:
35         static vector<int> wx, inv_w;
36         static vector<vector<int>>> fx;
37         static ll ksm(ll x, ll y) {
38             ll ans = 1;
39             while(y) {
40                 if(y & 1) ans = (ans * x) % mod;
41                 x = (x * x) % mod;
42                 y >>= 1;
43             }
44             return ans;
45         }
46         static void init() {
47             for(int i = 1; i ≤ 40; i++) {
48                 wx[i] = ksm(g, (mod - 1) / (1 << i));
49                 inv_w[i] = ksm(wx[i], mod - 2);
50             }
51         }
52         static void NTT(vector<ll> &a, int type, int Max, vector<int> &f) {
53             for (int i = 0; i < Max; ++i) {
54                 if(i < f[i]) swap(a[i], a[f[i]]);
55             }
56             for(int mid = 1, tx = 1; mid < Max; mid <= 1, tx++) {
57                 ll W = 0;
58                 if(type == 1) W = wx[tx];
59                 else W = inv_w[tx];

```



```

60         ll len = mid << 1; // 区间长度
61         for(int j = 0; j < Max; j += len) {
62             ll w = 1;
63             for(int k = 0; k < mid; k++) { // 每个区间枚举前半
64                 ll u = a[j + k];
65                 ll v = (w * a[j + k + mid]) % mod;
66                 a[j + k] = (u + v) % mod;
67                 a[j + k + mid] = (u - v + mod) % mod;
68                 w = (w * W) % mod;
69             }
70         }
71     }
72     if(type == -1) {
73         ll inv_M = ksm(Max, mod - 2);
74         for(int i = 0; i < Max; i++) {
75             a[i] = (a[i] * inv_M) % mod;
76         }
77     }
78 }
79 static vector<ll> Multi(vector<ll> a, vector<ll> b, int len) {
80     int Max = 1;
81     int cnt = 0;
82     while(Max ≤ len) Max <=< 1, cnt++;
83     a.resize(Max + 3); b.resize(Max + 3);
84     vector<int> f;
85     f.resize(Max + 1);
86     for(int i = 0; i < Max; i++)
87         f[i] = (f[i >> 1] >> 1) | ((i & 1) << (cnt - 1));
88     NTT(a, 1, Max, f); NTT(b, 1, Max, f);
89     for(int i = 0; i < Max; i++) a[i] = (a[i] * b[i]) % mod;
90     NTT(a, -1, Max, f);
91     return a;
92 }
93 static pair<int, vector<ll>> Solve(int l, int r) {
94     if(l == r) return {fx[l].size() - 1, fx[l]}; // 这里注意可能会改成别的值
95     int mid = (l + r) >> 1;
96     pair<int, vector<int>> ans1 = Solve(l, mid);
97     pair<int, vector<int>> ans2 = Solve(mid + 1, r);
98     int len = ans1.first + ans2.first;
99     vector<int> ansx = Multi(ans1.second, ans2.second, len);
100    ansx.resize(len + 1);
101    return {len, ansx};
102    // Multi的len一定不要忘了, 否则可能会超时
103 }
104 };
105 vector<int> DC_FFT::wx;
106 vector<int> DC_FFT::inv_w;
107 vector<vector<int>> DC_FFT::fx;
108
109 // Poly begin
110 int rev[N], lim, g[20][N], Max, vv;
111 void upd(int& a){a += a >> 31 & mod;}
112 void init(int n){
113     int l = -1;
114     for(lim = 1; lim < n; lim <=< 1) ++l; Max = l + 1;
115     for(int i = 1; i < lim; ++i)
116         rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << l);
117     vv = ksm(lim, mod - 2);

```

```

118 }
119 void NTT(int*a,int f){
120     for(int i=1;i<lim;++i)if(i<rev[i])std::swap(a[i],a[rev[i]]);
121     for(int i=0;i<Max;++i){
122         const int*G=g[i],c=1<<i;
123         for(int j=0;j<lim;j+=c<<1)
124             for(int k=0;k<c;++k){
125                 const int x=a[j+k],y=a[j+k+c]*(ll)G[k]%mod;
126                 upd(a[j+k]+=y-mod),upd(a[j+k+c]=x-y);
127             }
128     }
129     if(!f){
130         for(int i=0;i<lim;++i)a[i]=(ll)a[i]*vv%mod;
131         std::reverse(a+1,a+lim);
132     }
133 }
134 void INV(const int*a, int*B, int n){
135     if(n == 1){
136         *B = ksm(*a, mod - 2);
137         return;
138     }
139     INV(a, B, n + 1 >> 1);
140     init(n << 1);
141     static int A[N];
142     for(int i = 0; i < n; ++i) A[i] = a[i];
143     for(int i = n; i < lim; ++i) A[i]=0;
144     NTT(A, 1), NTT(B, 1);
145     for(int i = 0; i < lim; ++i)
146         B[i] = B[i] * (2 - (ll)A[i] * B[i] % mod + mod) % mod;
147     NTT(B, 0);
148     for(int i = n; i < lim; ++i) B[i] = 0;
149 }
150 void REV(int*A, int n) {std::reverse(A, A + n + 1);}
151 void MOD(const int*a, const int*b, int*R, int n, int m){
152     static int A[N], B[N], D[N];
153     for(int i = 0; i < n << 1; ++i) D[i] = 0;
154     for(int i = 0; i ≤ m; ++i) B[i] = b[i];
155     REV(B, m);
156     for(int i = n - m + 1; i < n << 1; ++i) B[i] = 0;
157     INV(B, D, n - m + 1);
158     init(n - m + 1 << 1);
159     for(int i = 0; i ≤ n - m + 1; ++i) A[i] = a[n - i];
160     for(int i = n - m + 2; i < lim; ++i) A[i] = 0;
161     NTT(A, 1), NTT(D, 1);
162     for(int i = 0; i < lim; ++i) D[i] = (ll)D[i] * A[i] % mod;
163     NTT(D, 0);
164     REV(D, n - m);
165     init(n + 1 << 1);
166     for(int i = n - m + 1; i < lim; ++i) D[i] = 0;
167     for(int i = 0; i < lim; ++i) A[i] = B[i] = 0;
168     for(int i = 0; i ≤ m; ++i) B[i] = b[i];
169     for(int i = 0; i ≤ n; ++i) A[i] = a[i];
170     NTT(B, 1), NTT(D, 1);
171     for(int i = 0; i < lim; ++i) B[i] = (ll)B[i] * D[i] % mod;
172     NTT(B, 0);
173     for(int i = 0; i < m; ++i) upd(R[i] = A[i] - B[i]);
174 }
175 //Poly end

```

```

176 int A[N], a[N], *P[N], len[N];
177 void solve(int l, int r, int o){
178     if(l == r){
179         len[o] = 1;
180         P[o] = new int[2];
181         upd(P[o][0] -= a[l]), P[o][1] = 1;
182         return;
183     }
184     const int mid = l + r >> 1, L = o << 1, R = L | 1;
185     solve(l, mid, L), solve(mid + 1, r, R);
186     len[o] = len[L] + len[R];
187     P[o] = new int[len[o] + 1];
188     init(len[o] + 1 << 1);
189     static int A[N], B[N];
190     for(int i = len[L]; ~i; --i) A[i] = P[L][i];
191     for(int i = len[L] + 1; i < lim; ++i) A[i] = 0;
192     for(int i = len[R]; ~i; --i) B[i] = P[R][i];
193     for(int i = len[R] + 1; i < lim; ++i) B[i] = 0;
194     NTT(A, 1), NTT(B, 1);
195     for(int i = 0; i < lim; ++i) A[i] = (ll) A[i] * B[i] % mod;
196     NTT(A, 0);
197     for(int i = len[o]; ~i; --i) P[o][i] = A[i];
198 }
199 int cnt = 0;
200 vector<int> gxx;
201 void solve(int l, int r, int o, const int*A){
202     if(l == r) {gxx.push_back(*A);return;}
203     const int mid = l + r >> 1, L = o << 1, R = L | 1;
204     int B[len[o] + 2 << 1];
205     MOD(A, P[L], B, len[o] - 1, len[L]);
206     solve(l, mid, L, B);
207     MOD(A, P[R], B, len[o] - 1, len[R]);
208     solve(mid + 1, r, R, B);
209 }
210 void work(int _n, int _m, vector<int>& fx, vector<int> x) {
211     int n = 0, m = 0;
212     for(int i = 0; i < 19; ++i){
213         int* G = g[i];
214         G[0] = 1;
215         const int gi = G[1] = ksm(3, (mod - 1) / (1 << i+1));
216         for(int j = 2; j < 1 << i; ++j)
217             G[j] = (ll) G[j - 1] * gi % mod;
218     }
219     n = _n - 1, m = _m;
220     for(int i = 0; i ≤ n; i++){
221         A[i] = fx[i];
222     }
223     for(int i = 1; i ≤ m; i++){
224         a[i] = x[i];
225     }
226     solve(1, m, 1);
227     if(n ≥ m) MOD(A, P[1], A, n, m);
228     solve(1, m, 1, A);
229 }
230
231 void Deri(vector<int> &gx) {
232     for(int i = 0; i < gx.size() - 1; i++) {
233         gx[i] = ((i + 1) * gx[i + 1]) % mod;

```

```

234     // cout << gx[i] << " ";
235 } //cout << endl;
236 }
237 int calc(int k, vector<int> &gg) {
238     int sum = 0;
239     for(int i = 0; i < gg.size(); i++) {
240         int now = 0;
241         now = ksm(k, i);
242         now = (now * gg[i]) % mod;
243         sum = (sum + now) % mod;
244     }
245     return sum;
246 }
247 signed main(){
248     //freopen("1.in", "r", stdin);
249     int n = read(), k = read(); //已知n个点, 求f(k)
250     vector<int> x, y;
251     x.resize(n + 5);
252     y.resize(n + 5);
253     vector<vector<int>> fx;
254     fx.resize(n + 5);
255     for(int i = 1; i ≤ n; i++) {
256         x[i] = read(); //输入n个点对(xi, yi)
257         y[i] = read();
258         fx[i].push_back(-x[i]);
259         fx[i].push_back(1);
260     }
261     DC_FFT sol;
262     vector<int> gg = sol.work(n, fx); //分治FFT
263     vector<int> gx = gg;
264     Deri(gx); //求导
265     gx.resize(gx.size() - 1);
266     work(gx.size(), n, gx, x); //多项式多点求值
267     //gxx是g'的具体值
268     //f(k)=gg(k)*\sum (yi/gxx(i))*inv(k-xi)
269     int gk = calc(k, gg); //gg(k)
270
271     int ANS = 0;
272     for(int i = 1; i ≤ n; i++) { //O(nlogn)求和
273         int inv1 = ksm(k - x[i], mod - 2);
274         int inv2 = ksm(gxx[i - 1], mod - 2);
275         int now = (y[i] * inv2) % mod;
276         now = (now * inv1) % mod;
277         ANS = (ANS + now) % mod;
278     }
279     ANS = (ANS * gk) % mod;
280     cout << ANS;
281     return 0;
282 }
283 //月雪·薇婷
284 /*
285 941094
286 */

```

19. 二维凸包

定义：二维平面上 n 个点，用一根橡皮筋把它们包围住（即使得面积最小且能包住所有点）的多边形称为凸包

显然凸包由这 n 个点的某些点构成

注意，凸包的边在某些情况下可能是圆弧

方法1. 模拟法

找到 y 值最小的一个点，以它为中心展开一条 $k = 0$ 的直线，然后不断逆时针旋转，以碰到的第一个点为新的中心，继续旋转，知道 $p_{new} = p_1$ 为止。如果同时有 ≥ 2 个点都是同时出现，选择离当前点最远的点为新的中心。

缺点：代码复杂度极高，且遇到 $k = INF$ 则基本上没办法正常工作

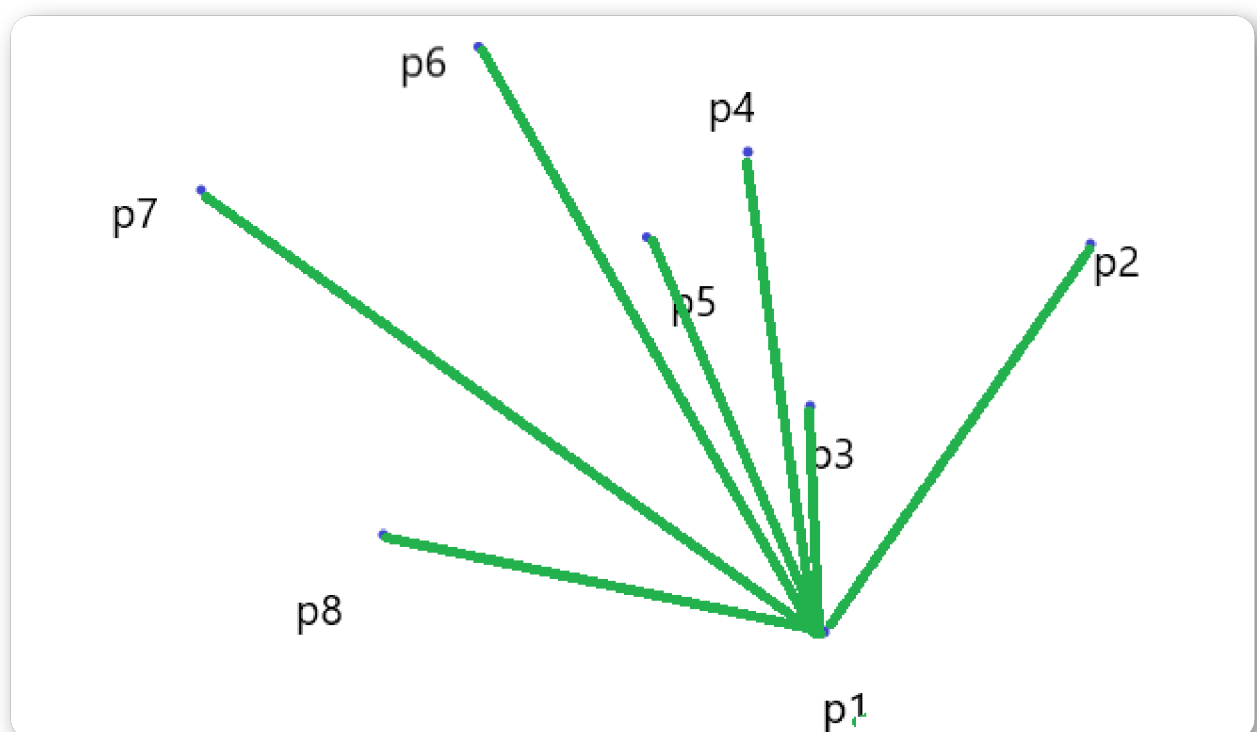
方法2. *Graham*算法

step1 :去重

step2 :选取 y 最小（若有多个，选 x 最小）的点设为 P_1

step3 :其它的点按照对于 P_1 的极角排序，依此为 $p_{2...n}$ ，如下：

即，令其它点坐标为 $(p_{ix} - p_{1x}, p_{iy} - p_{1y})$ ，然后计算 $\text{atan2}(y, x)$ （极角）



step4 :按照顺序依次加入（入栈）。不难发现，若当前搜寻到 i ，那第 i 个点一定会成为凸包上的一个点

step5 :假设当前把第 i 个点加入凸包，假设现在栈内如下： $[p_i, p_j, p_k]$ ，如果发现 p_j 在 p_i 和 p_k 所连的线的下面，则说明 p_j 不应该在凸包上，将 p_j 删掉（判断 $\vec{p_k p_j} \times \vec{p_j p_i}$ 的大小， ≤ 0 代表需要弹出栈）

step6 :不断搜寻，直至结束，栈内即为凸包上的点

时间复杂度： $O(N \log N)$

Code

```
1  #include<bits/stdc++.h>
2  #include<cmath>
3  #define ll long long
4  #define int long long
5  using namespace std;
6  const ll N = 1e5 + 100, M = 1e9 + 7, INF = 1e16;
7  int n;
8  ll read() {
9      ll x = 0, f = 1; char ch = getchar();
10     while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
11     while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
12     return x * f;
13 }
14 struct Point { //点集都用结构体存
15     double x, y;
16 } s[N], st[N];
17 //st: 栈
18 double calc_Len(Point a, Point b) { //计算两点距离
19     double dx = a.x - b.x;
20     double dy = a.y - b.y;
21     return sqrt(dx * dx + dy * dy);
22 }
23 double calc_S(Point a, Point b) {
24     double ans = (a.x * b.y - a.y * b.x);
25     ans = ans / 2.0;
26     return ans;
27 }
28 double check(Point a1, Point a2, Point b1, Point b2) {
29     // 向量a1→a2和向量b1→b2
30     double ux = a2.x - a1.x;
31     double uy = a2.y - a1.y;
32     double vx = b2.x - b1.x;
33     double vy = b2.y - b1.y;
34     return ux * vy - uy * vx;
35     // 返回向量叉乘积
36 }
37 bool cmp(Point a, Point b) {
38     double now = check(s[1], a, s[1], b); // 这里顺序不要错了!!
39     // 通过叉乘来替代极角，避免浮点误差
40     if(now > 0) return true;
41     else if(now < 0) return false;
42     // 如果共线
43     return calc_Len(s[1], a) < calc_Len(s[1], b); // 近的排前面
```

```

44 }
45 void Printf(double x, int L) {
46     cout << fixed << setprecision(L) << x << endl;
47 }
48 signed main(){
49     n = read();
50     int new_n = 0, num = 0;
51     double Minx = 1e18, Miny = 1e18; //这里记得根据实际情况调整!!!!
52     for(int i = 1; i ≤ n; i++) {
53         double x, y;
54         cin >> x >> y;
55
56         //将(x,y)压入s
57         s[++new_n].x = x;
58         s[new_n].y = y;
59         if(new_n ≥ 2) { //令s[1]为基准点, 找到符合条件基准点
60             if(s[new_n].y < s[1].y) {
61                 swap(s[new_n], s[1]);
62             }
63             else if(s[new_n].y == s[1].y) {
64                 if(s[new_n].x < s[1].x) {
65                     swap(s[new_n], s[1]);
66                 }
67             }
68         }
69     }
70     n = new_n;
71     sort(s + 2, s + n + 1, cmp); //按照极角排序
72     //从第二个开始, 第一个是基准点
73
74     s[0].x = s[1].x - 1; //保证不同
75     for(int i = 1; i ≤ n; i++) { //依次入栈
76         if(s[i].x == s[i - 1].x && s[i].y == s[i - 1].y)
77             continue; //去重
78         while(num ≥ 2) {
79             if(check(st[num - 1], st[num], st[num], s[i]) ≤ 0) {
80                 num--;
81                 //这里check顺序不要写错了!!!!
82             }
83             else break;
84         }
85         st[++num] = s[i];
86     }
87     st[++num] = s[1]; //最后一个和第一个相同, 形成循环
88     double C = 0, S = 0; //周长 和 面积
89     for(int i = 1; i < num; i++) {
90         C += calc_Len(st[i], st[i + 1]); //计算凸包周长
91         S += calc_S(st[i], st[i + 1]);
92     }
93     Printf(C, 2); //四舍五入两位输出
94     cout << endl;
95     printf("%.2lf\n", S); //保留两位输出
96     return 0;
97 }
98 //月雪·薇婷

```

20. 向量叉积

有两个向量 $\vec{a} = (ax, ay)$, $\vec{b} = (bx, by)$

令 $p = \vec{a} \times \vec{b} = ax \cdot by - ay \cdot bx$

表示含义

1. 表示三角形面积
 2. 若 $p > 0$, 代表 $\vec{a}$ 通过顺时针旋转 $(0, \pi)$ 的角度到 $\vec{b}$;
通过逆时针旋转 $(0, \pi)$ 的角度到 $\vec{b}$
- $p = 0$, $\vec{a}, \vec{b}$ 共线; $p < 0$, $\vec{a}$

21. 扩展欧几里得 *Exgcd*

这里直接讲 $a \cdot x + b \cdot y = d$ (假设 a, b, d 都是正整数)

💡 Important

如果 $b \% \gcd(a, b) \neq 0$ 那么这组方程无解

exgcd 求的是 $a \cdot x + b \cdot y = \gcd(a, b)$ 的一组解

21.1 基本代码

```
1 int Exgcd(int a, int b, int &x, int &y) {
2     if (!b) {
3         x = 1;
4         y = 0;
5         return a;
6     }
7     int d = Exgcd(b, a % b, x, y);
8     int t = x;
9     x = y;
10    y = t - (a / b) * y;
11    return d;
12 }
13 void solve(int a, int b, int d) {
```



```

14     int x, y;
15     int g = Exgcd(a, b, x, y); //返回值是gcd, x和y自动赋值
16     if(d % g != 0) return false; //无解
17     x = x * (d / g); y = y * (d / g); //此时(x, y)即为一组解
18 }

```

C++

21.2 通解

得到一组解 (x_0, y_0) 之后（通过 $exgcd$ 求得的 (x_0, y_0) 记得先 乘上 $\frac{d}{g}$ 才对），可以得到通解：

$$\begin{cases} x = x_0 + t \cdot \frac{b}{g} \\ y = y_0 - t \cdot \frac{a}{g} \end{cases}$$

21.3 最小非负整数解

以 x 为例，写出不等式可以有： $x_0 + t \cdot \frac{b}{g} \geq 0$

于是有 $t \geq -x_0 / (\frac{b}{g})$

这里可以用到向上取整得到最小的 t

```

1 int calc_up(int x, int y) {
2     if(x % y == 0) return x / y;
3     if((x <= 0 && y < 0) || (x >= 0 && y > 0)) //异号时需特判
4         return x / y + 1;
5     return x / y;
6 }

```

C++

七. 图论

1. 网络流

1.1 网络流-最大流

假设存在一张图，每条边有容量限制

现在增加起始点（源点）和终点（汇点）

起点可以流出无限量的流量，终点最终能得到多少？

最优解为：*HLPP*预流推进算法，时间复杂度 $O(N^2\sqrt{m})$

1.1.1 使用条件

遇到：**不能超过**、**容量**等有限制类型的词的时候，大概率就是网络流

1.1.2 Solution

Edmonds-Karp算法（不推荐）

1. 每次寻找一条从源点到汇点的路，路径上每条边都还有剩余流量，最后要使得搜寻到汇点时最小的剩余流量，用的是*BFS*寻找经过边数最少的路径。（不会证明正确性）
2. 每条有向边不仅要记录当前剩余可以使用流量，同时还要**新增一条反向边**，**剩余可使用流量设置为0**，这样做的机制是有反悔机制，从正着走过去还可以反着回来
3. 正向边的*num*为奇数，反向边的*num*为正向边+1，这样假设正向边的编号为*i*，那么*i* ⊕ 1即为反向边，很方便。**所以*num* = 1才是初始条件!!!**
4. 时间复杂度： $O(NM^2)$ ($O(|V||E|^2)$)

Code

```
1  #include<bits/stdc++.h>
2  #include<cmath>
3  #define ll long long
4  #define int long long
5  using namespace std;
6  const ll N = 1e4 + 10, M = 998244353, INF = 1e18;
7  int num = 1, fa[N], ne[N], to[N], fi[N], w[N], n, m, vis[N];
8  int s, t, d[N], ans, f[1010][1010];
9  ll read() {
10     ll x = 0, f = 1; char ch = getchar();
11     while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
12     while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
13     return x * f;
14 }
15 void add(int u, int v, int val) {
16     ne[++num] = fi[u];
17     fi[u] = num;
18     to[num] = v;
19     w[num] = val; // 正向边边剩余流量
20
21     swap(u, v); // 记录反向边
22     ne[++num] = fi[u];
23     fi[u] = num;
24     to[num] = v;
25     w[num] = 0; // 反向边最开始剩余流量为0
26 }
27 queue<int> q;
28 bool BFS() { // Ek增广路
```

```

29     vis[s] = true;
30     d[s] = INF;
31     q.push(s);
32     while(!q.empty()) {
33         int u = q.front();
34         q.pop();
35         for(int i = fi[u]; i; i = ne[i]) {
36             int v = to[i];
37             if(w[i] ≤ 0) continue; // 剩余流量 ≤ 0 则不能走
38             if(vis[v]) continue; // 是否查找过
39             d[v] = min(d[u], w[i]); // d[v] 为从源点到v这个点路径上能使用的最小的流量 (木桶短板效应)
40             fa[v] = i; // 记录前驱边的编号
41             vis[v] = true;
42             q.push(v);
43             if(v == t) return true; // 找到就返回1
44         }
45     }
46     return false; // 不存在返回0
47 }
48 void update() {
49     int now = t;
50     while(true) {
51         if(now == s) break;
52         w[fa[now]] -= d[t]; // 正向边减少d[t]流量
53         w[fa[now] ^ 1] += d[t]; // 反向边增加d[t]流量
54         now = to[fa[now] ^ 1];
55     }
56     ans += d[t]; // d[t] 为这条路径能增加的流量
57 }
58 void init() {
59     while(!q.empty()) q.pop();
60     for(int i = 1; i ≤ nn; i++) // 注意这里n可能不是正确的节点个数! 请自行判断
61         vis[i] = 0;
62 }
63 signed main() {
64     n = read(), m = read();
65     s = read(), t = read();
66     for(int i = 1; i ≤ m; i++) {
67         int u = read(), v = read(), val = read();
68         if(!f[u][v]) { // 如果有重边, 注意记录
69             add(u, v, val);
70             f[u][v] = num;
71         }
72         else {
73             w[f[u][v] - 1] += val; // 有重边, 则正向边的总剩余流量增加
74         }
75     }
76     while(true) {
77         if(BFS()) update();
78         else break;
79         init();
80     }
81     cout << ans;
82     return 0;
83 }
84 // 月雪·薇婷

```

1.1.3 Dinic算法（下面还有更优解）

1. *Dinic*一次增广能找到多条增广路径，所以可以加速
2. 判断是否存在增广路径还是用*BFS*，但是*Update*改成了*DFS*，同时在*BFS*的时候，记录当前可用网络（去除掉剩余流量为0的边，或者叫**残量网络**）每个节点的层数（编号） $h_v = h_u + 1$
3. 在*Dinic*算法中，采用分层图来使得一次性能找到多条路径。在*DFS*中，只有 $h[v] = h[u] + 1$ （**分层图条件**）的才能继续往下搜。（同样不知道怎么证明正确性）
4. **当前弧优化**：寻找过的边就不用继续找了，所以用 $nowfi_u$ 来表示最新的 fi_u ，找过的就不用找了
5. 时间复杂度： $O(N^2M)$ ($O(|V|^2|E|)$)

Code

```
1  #include<bits/stdc++.h>
2  #include<cmath>
3  #define ll long long
4  #define int long long
5  using namespace std;
6  const ll N = 1e4 + 10, M = 998244353, INF = 1e18;
7  int num = 1, ne[N], to[N], fi[N], w[N], n, m, vis[N];
8  int s, t, ans, f[1010][1010], nowfi[N], h[N];
9  ll read() {
10     ll x = 0, f = 1; char ch = getchar();
11     while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
12     while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
13     return x * f;
14 }
15 void add(int u, int v, int val) {
16     ne[++num] = fi[u];
17     fi[u] = num;
18     to[num] = v;
19     w[num] = val; // 正向边边剩余流量
20
21     swap(u, v); // 记录反向边
22     ne[++num] = fi[u];
23     fi[u] = num;
24     to[num] = v;
25     w[num] = 0; // 反向边最开始剩余流量为0
26 }
27 queue<int> q;
28 bool BFS() { // Ek增广路，这一部分几乎无变化
29     vis[s] = true;
30     h[s] = 1;
31     q.push(s);
32     while(!q.empty()) {
33         int u = q.front();
34         q.pop();
35         for(int i = fi[u]; i; i = ne[i]) {
36             int v = to[i];
37             if(w[i] ≤ 0) continue;
38             if(vis[v]) continue;
39             h[v] = h[u] + 1;
40             vis[v] = true;
41             q.push(v);
42             if(v == t) return true;
```

```

43     }
44 }
45 return false; //不存在返回0
46 }
47 int dfs(int u, int flow) {
48     //sum是从源点到u路径上能使用的最小流量
49     //同时flow也是当前从u出发能使用的最大流量
50     if(u == t) return flow;
51     int sum = 0;
52     for(int i = nowfi[u]; i; i = ne[i]) {
53         int v = to[i];
54         nowfi[u] = i; //已经查找过边i, 就不用继续找
55         if(w[i] ≤ 0) continue; //剩余流量 ≤ 0则continue
56         if(h[v] == h[u] + 1) { //满足分层图条件才能往下搜
57             int now = dfs(v, min(flow - sum, w[i]));
58             //因为已经使用了sum的流量, 所以要减去
59             //下一层能使用的就是min(flow-sum, w[i])
60             w[i] -= now; //正向边剩余流量-now
61             w[i ^ 1] += now; //反向边+now
62             sum += now; //当前从u出发使用了多少流量
63             if(sum == flow) break; //使用完可以提前退出查找
64         }
65     }
66     return sum; //返回u用了多少流量
67 }
68 void init() {
69     while(!q.empty()) q.pop();
70     for(int i = 1; i ≤ nn; i++) { //注意这里n可能不是正确的节点个数! 请自行判断
71         vis[i] = 0;
72         nowfi[i] = fi[i]; //重置nowfi
73         h[i] = 0; //重置深度
74     }
75 }
76 signed main() {
77     n = read(), m = read();
78     s = read(), t = read();
79     for(int i = 1; i ≤ m; i++) {
80         int u = read(), v = read(), val = read();
81         if(!f[u][v]) { //如果有重边, 注意记录
82             add(u, v, val);
83             f[u][v] = num;
84         }
85         else {
86             w[f[u][v] - 1] += val;
87         }
88     }
89     while(true) {
90         if(BFS()) ans += dfs(s, INF); //update改成dfs
91         else break;
92         init();
93     }
94     cout << ans;
95     return 0;
96 }
97 //月雪·薇婷

```

1.1.4 HLPP 预流推进算法

没看过，但是建图方式一样，所以可以直接套用

时间复杂度 $O(N^2\sqrt{m})$

Code

```
1  #include <cstdio>
2  #include <cstring>
3  #include <queue>
4  #include <stack>
5  using namespace std;
6  constexpr int N = 1200, M = 120000, INF = 0x3f3f3f3f;
7  int n, m, s, t;
8
9  struct qxx {
10     int nex, t;
11     long long v;
12 };
13
14 qxx e[M * 2 + 1];
15 int h[N + 1], cnt = 1;
16
17 void add_path(int f, int t, long long v) {
18     e[++cnt] = qxx{h[f], t, v}, h[f] = cnt;
19 }
20
21 void add_flow(int f, int t, long long v) {
22     add_path(f, t, v);
23     add_path(t, f, 0);
24 }
25
26 int ht[N + 1]; // 高度;
27 long long ex[N + 1]; // 超额流;
28 int gap[N]; // gap 优化. gap[i] 为高度为 i 的节点的数量
29 stack<int> B[N]; // 桶 B[i] 中记录所有 ht[v]=i 的v
30 int level = 0; // 溢出节点的最高高度
31
32 int push(int u) { // 尽可能通过能够推送的边推送超额流
33     bool init = u == s; // 是否在初始化
34     for (int i = h[u]; i; i = e[i].nex) {
35         const int &v = e[i].t;
36         const long long &w = e[i].v;
37         // 初始化时不考虑高度差为1
38         if (!w || (init == false && ht[u] != ht[v] + 1) || ht[v] == INF) continue;
39         long long k = init ? w : min(w, ex[u]);
40         // 取到剩余容量和超额流的最小值, 初始化时可以使源的溢出量为负数。
41         if (v != s && v != t && !ex[v]) B[ht[v]].push(v), level = max(level, ht[v]);
42         ex[u] -= k, ex[v] += k, e[i].v -= k, e[i ^ 1].v += k; // push
43         if (!ex[u]) return 0; // 如果已经推送完就返回
44     }
45     return 1;
46 }
47
48 void relabel(int u) { // 重贴标签 (高度)
```

```

49     ht[u] = INF;
50     for (int i = h[u]; i; i = e[i].nex)
51         if (e[i].v) ht[u] = min(ht[u], ht[e[i].t]);
52     if (++ht[u] < n) { // 只处理高度小于 n 的节点
53         B[ht[u]].push(u);
54         level = max(level, ht[u]);
55         ++gap[ht[u]]; // 新的高度, 更新 gap
56     }
57 }
58
59 bool bfs_init() {
60     memset(ht, 0x3f, sizeof(ht));
61     queue<int> q;
62     q.push(t), ht[t] = 0;
63     while (q.size()) { // 反向 BFS, 遇到没有访问过的结点就入队
64         int u = q.front();
65         q.pop();
66         for (int i = h[u]; i; i = e[i].nex) {
67             const int &v = e[i].t;
68             if (e[i ^ 1].v && ht[v] > ht[u] + 1) ht[v] = ht[u] + 1, q.push(v);
69         }
70     }
71     return ht[s] != INF; // 如果图不连通, 返回 0
72 }
73
74 // 选出当前高度最大的节点之一, 如果已经没有溢出节点返回 0
75 int select() {
76     while (level > -1 && B[level].size() == 0) level--;
77     return level == -1 ? 0 : B[level].top();
78 }
79
80 long long hlpp() { // 返回最大流
81     if (!bfs_init()) return 0; // 图不连通
82     memset(gap, 0, sizeof(gap));
83     for (int i = 1; i ≤ n; i++)
84         if (ht[i] != INF) gap[ht[i]]++; // 初始化 gap
85     ht[s] = n;
86     push(s); // 初始化预流
87     int u;
88     while ((u = select())) {
89         B[level].pop();
90         if (push(u)) { // 仍然溢出
91             if (!--gap[ht[u]])
92                 for (int i = 1; i ≤ nn; i++) // 这里n可能不是实际的节点数! 请自行判断
93                     if (i != s && ht[i] > ht[u] && ht[i] < n + 1)
94                         ht[i] = n + 1; // 这里重贴成 n+1 的节点都不是溢出节点
95             relabel(u);
96         }
97     }
98     return ex[t];
99 }
100
101 int main() {
102     scanf("%d%d%d%d", &n, &m, &s, &t);
103     for (int i = 1, u, v, w; i ≤ m; i++) {
104         scanf("%d%d%d", &u, &v, &w);
105         add_flow(u, v, w);
106     }

```

```

107     printf("%lld", h1pp());
108     return 0;
109 }

```

C++

2. 分层图(注意事项)

*BFS*中分层图很简单，但是有小细节：

1. 比较的时候，一定要每层对照着比较，不能第一层和第二层比较
2. ★★★★★最后如果比如说是找路径，**最后一定要确保回到的是最初的那一层！**很重要，被坑了4hour。比如说起点对应的是*s*点的第0层，那返回的时候找到的如果是*s*点的第1层就不行

3. LCA的求法

LCA :两个点的最近公共祖先

非常基础的内容，很多地方都需要用到

单次求*LCA*时间复杂度 $O(\log N)$ ，预处理时间复杂度 $O(N \log N)$

3.1 预处理

首先倍增求*st*表以及深度

*dep*记得从1开始！！

```

1 void dfs(int u, int fa, int d) {
2     dep[u] = d; f[u][0] = fa; //深度和父亲节点
3     for(int i = fi[u]; i; i = ne[i]) {
4         int v = to[i];
5         if(v == fa) continue; //对于树，只要不走“回头路”即不会重复查询
6         dfs(v, u, d + 1);
7     }
8 }
9 void init() {
10     dfs(1, 0, 1);
11     for(int j = 1; j ≤ 20; j++) { //20对应深度为1e6
12         for(int i = 1; i ≤ n; i++) {
13             f[i][j] = f[f[i][j - 1]][j - 1];
14         }
15     }
16 }

```

C++

3.2 求LCA

第一步：使得 $dep[x]$ 一定大于 $dep[y]$ ，这样只考虑移动 x

第二步：倍增移动 x 使得 $dep[x] = dep[y]$

第三步：同时向上移动 x, y 求出LCA

```
1 int lca(int x, int y) {
2     if(dep[x] < dep[y]) swap(x, y);
3     for(int i = 20; i ≥ 0; i--) { //20对应log(1e6)
4         if(dep[f[x][i]] ≥ dep[y]) {
5             x = f[x][i];
6         }
7     }
8     if(x == y) return x; //一定要注意这里需要判断！！否则后面会出问题
9     for(int i = 20; i ≥ 0; i--) {
10        if(f[x][i] != f[y][i]) {
11            x = f[x][i];
12            y = f[y][i];
13        }
14    }
15    //这里的x和y差一个相等，所以返回x的father
16    return f[x][0];
17 }
```

C++

4. 图论·一些定义

4.1 重链

在一棵树中，会被分为 k 条不相交的链。详细的，从顶点开始，若子节点 u 连接的子树拥有的节点最多，那选择 u 作为“重”节点，有多个符合条件的只选一个。其他的被称为“轻”节点。

如下图：

```
1      A
2     / \
3    C   B
4   / \   \
5  F  E   D
6 A→C→F, E, B→D 是一组重链
```

C++

5. 最小生成树

5.1 *Kruskal*算法

按照边的权值排序，根据并查集判断是否在同一连通块内进行合并

时间复杂度 $O(m\log m + m)$

```
1 sort();
2 for(i: 1~m)
3     if fa_u≠fa_v
4         merge()
```

C++

5.2 *Prim*算法

核心思想：只维护一个连通块，假设 u 属于连通块， v 不属于，那每次需要找到 $d_{u,v}$ 最小的一个，把 v 加入即可

$dis[u]$ 表示在最小生成树内，把 u 加入这棵树的最短边

每次使用在连通块内，且未使用的点进行更新。枚举这个点的所有边，按边权大小进行更新

(也有 $O(N^2)$ 的算法，用数组 d 记录其他点到连通块的最短距离，每次选择距离连通块最近的点，然后更新其他点的距离 d 即可)

时间复杂度 $O((n + m)\log n)$

```
1 priority_queue<pair<int, int>> q;
2 long long solve() {
3     for(int i = 1; i ≤ n; i++) {
4         dis[i] = INF;
5         vis[i] = false;
6         //初始化
7     }
8     dis[1] = 0; //起点无边
9     q.push({0, 1});
10    int cnt = 0;
11    while(!q.empty()) {
12        if(cnt ≥ n) break; //已经找到n个点
13        int u = q.top().second;
14        int d = -q.top().first;
15        q.pop();
16        if(vis[u]) continue; //已经走过，不用再走
17        vis[u] = true;
18        cnt++;
19        ans += d; //更新答案
20        for(int i = fi[u]; i; i = ne[i]) {
21            int v = to[i];
22            if(dis[v] > w[i]) {
23                dis[v] = w[i];
24                q.push({-w[i], v});
```

```

25         }
26     }
27 }
28 if(cnt < n) ans = INF; // 如果无法形成最小生成树
29 return ans;
30 }

```

C++

6. 简单环&最小环

简单环定义：在无向图中的一个环，且环内无环套环

最小环：整个图中长度最小的环

6.1 简单环求法

显而易见简单环非常多，所以这里讲的是[经过某条特定边的简单环](#)

1. 把这条边断开，假设这条边连接的节点是 s 和 t
2. 从 s 出发，用 bfs 找到到达 t 的最短路径，记录每个节点的 fa ，方便回溯

Code

```

1  for(int i = 1; i ≤ n; i++) {
2      fa[i] = d[i] = 0;
3  }
4  queue<int> q;
5  q.push(s);
6  d[s] = 1; // 记住是从1开始的
7  while(!q.empty()) {
8      int u = q.front();
9      q.pop();
10     for(auto v: p[u]) {
11         if(u == s && v == t) continue;
12         if(u == t && v == s) continue; // 断开s-t这条边
13         if(d[v]) continue; // d[v]>0代表访问过
14         d[v] = d[u] + 1;
15         fa[v] = u; // 最后从t往上回溯到s即可，无法回溯说明找不到环
16         q.push(v);
17     }
18 }

```

C++

时间复杂度 $O(n + m)$

6.2 最小环求法

最小环稍微麻烦一点

首先枚举起点，代表这个环的起点

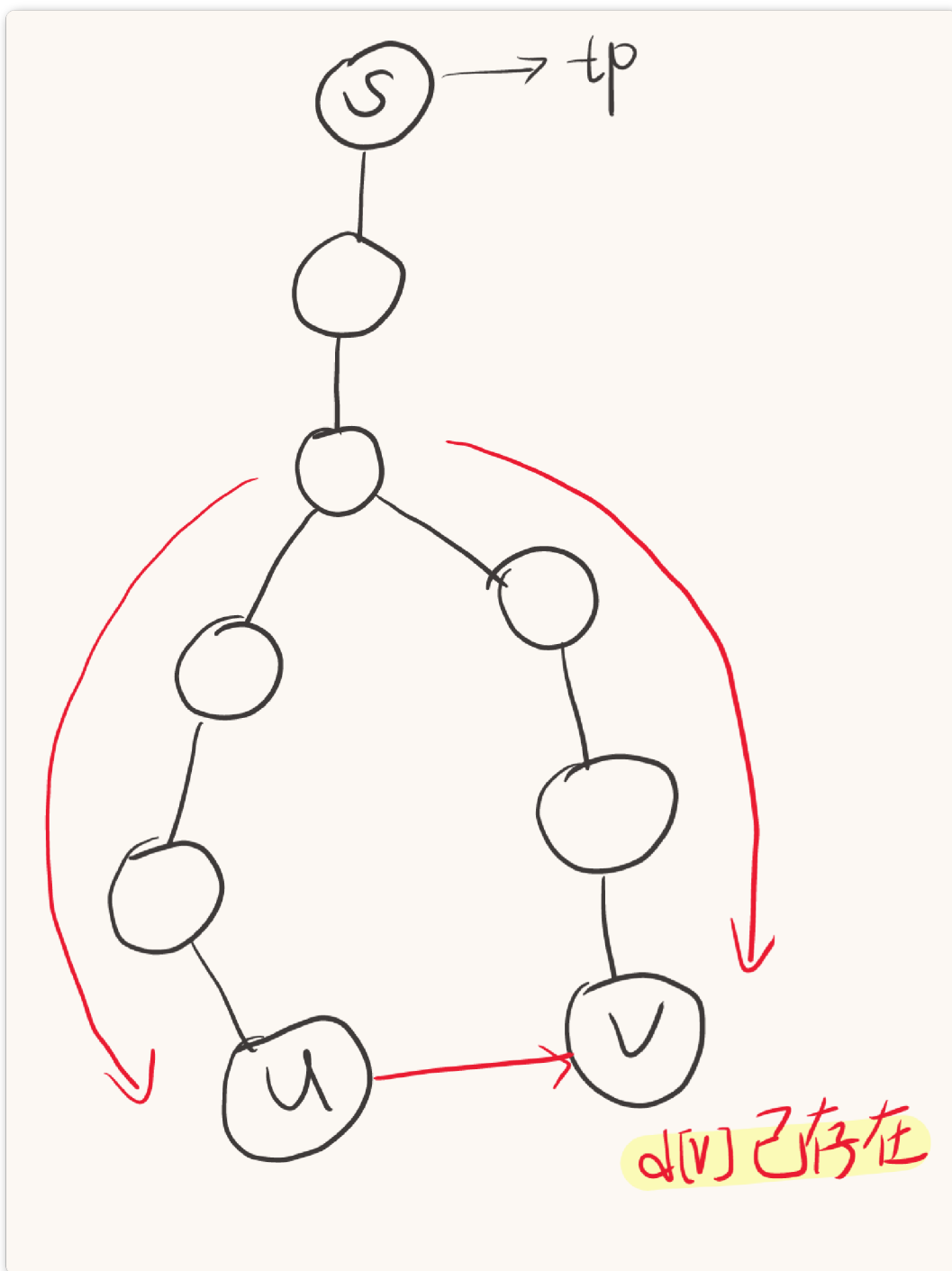
然后每个起点做一次**bfs**，每次**bfs**找到经过起点的最小简单环

具体来说，记录 $d[u]$ 代表到起点的深度，如果 $d[v]$ 存在代表访问过，说明能构成一个环（从其他方向能到 v ，从 u 这个方向也能到 v ，那这就是一个环）

如果是一个非简单环（环套环），那一定能找到一个更小环，根据此原理进行更新，最后找到的一定是最小简单环

时间复杂度： $O(n^2 + nm)$

注意，如果只进行一次**bfs**，那么找到的环可能如下图（带了一条链），所以回溯的时候需要用到类似于**lca**那样回溯，具体见代码：



Code

```

1  int Min = INF, s1, s2, sp;
2  void bfs(int s) {
3      queue<int> q;
4      for(int i = 1; i ≤ n; i++) {
5          fa_now[i] = d_now[i] = 0;
6      }
7      q.push(s);
8      d_now[s] = 1; // 注意从1开始赋值
9      int minn = INF, t1 = 0, t2 = 0, tp = 0;
10     // t1, t2代表环闭合的位置, tp代表起始点
11     while(!q.empty()) {
12         int u = q.front();
13         q.pop();
14         for(auto v: p[u]) {

```

```

15         if(v == fa_now[u]) continue; //不能又找回去了
16         if(d_now[v]) {
17             int len = d_now[u] + d_now[v] - 1; //len代表环的长度
18             if(len ≤ 2) continue; //len ≤ 2不成环
19             if(len < minn) { //找到更小环
20                 minn = len, t1 = u, t2 = v, tp = s;
21             }
22             continue;
23         }
24         d_now[v] = d_now[u] + 1;
25         fa_now[v] = u;
26         q.push(v);
27     }
28 }
29 if(minn < Min) {
30     Min = minn, s1 = t1, s2 = t2, sp = tp;
31     for(int i = 1; i ≤ n; i++) {
32         fa[i] = fa_now[i];
33         d[i] = d_now[i];
34     }
35     //每次结束后更新全局最小环
36 }
37 }
38 void solve() {
39     Min = INF, s1 = s2 = sp = 0;
40     for(int i = 1; i ≤ n; i++) bfs(i);
41     int now1 = s1, now2 = s2;
42     vis[s1] = vis[s2] = 1; //标记为环
43     while(true) { //回溯过程, 类似于lca, 谁更深谁先跳
44         if(d[now1] > d[now2]) {
45             now1 = fa[now1];
46             vis[now1] = 1; //vis=1, 代表是环
47         }
48         else if(d[now2] > d[now1]) {
49             now2 = fa[now2];
50             vis[now2] = 1;
51         }
52         else { //深度相同
53             if(now1 ≠ now2) { //now1不等于now2, 说明now1和now2还在环上
54                 now1 = fa[now1];
55                 now2 = fa[now2];
56                 vis[now1] = vis[now2] = 1;
57                 //放在now=fa[now]下面进行标记, 可以保证跳到第一个now1=now2的时候也标记为
58                 环
59             }
60             else { //now1=now2说明已经跳出环, 在链上
61                 if(now1 == sp) break; //到sp说明到终点了, break
62                 now1 = fa[now1];
63                 now2 = fa[now2];
64                 vis[now1] = vis[now2] = 2; //vis=2, 代表是sp到环的链
65             }
66         }
67     }
}

```

7. 二分图

7.1 二分图最大匹配 (最小顶点覆盖)

定理: **最大匹配数=最小顶点覆盖数**

最小顶点覆盖: 二分图中, 选最少的点使得每条边至少有一个顶点包含在被选点集合里

时间复杂度: $O(m\sqrt{n})$

```
1  #include<bits/stdc++.h>
2  #define int long long
3  using namespace std;
4  struct BipartiteGraph {
5      int n1, n2; // number of vertices in X and Y, resp.
6      std::vector<std::vector<int>>> g; // edges from X to Y
7      std::vector<int> ma, mb; // matches from X to Y and from Y to X, resp.
8      std::vector<int> dist; // distance from unsaturated vertices in X.
9
10     BipartiteGraph(int n1, int n2)
11         : n1(n1), n2(n2), g(n1), ma(n1, -1), mb(n2, -1) {}
12
13     // Add an edge from u in X to v in Y.
14     void add_edge(int u, int v) { g[u].emplace_back(v); }
15
16     // Build the level graph.
17     bool bfs() {
18         dist.assign(n1, -1);
19         std::queue<int> q;
20         for (int u = 0; u < n1; ++u) {
21             if (ma[u] == -1) {
22                 dist[u] = 0;
23                 q.emplace(u);
24             }
25         }
26         // Build the level graph for all reachable vertices.
27         bool succ = false;
28         while (!q.empty()) {
29             int u = q.front();
30             q.pop();
31             for (int v : g[u]) {
32                 if (mb[v] == -1) {
33                     succ = true;
34                 } else if (dist[mb[v]] == -1) {
35                     dist[mb[v]] = dist[u] + 1;
36                     q.emplace(mb[v]);
37                 }
38             }
39         }
40         return succ;
41     }
42
43     // Find an augmenting path starting at u.
44     bool dfs(int u) {
```

```

45     for (int v : g[u]) {
46         if (mb[v] == -1 || (dist[mb[v]] == dist[u] + 1 && dfs(mb[v]))) {
47             ma[u] = v;
48             mb[v] = u;
49             return true;
50         }
51     }
52     dist[u] = -1; // Mark this point as unreachable after one visit.
53     return false;
54 }
55
56 // Hopcroft-Karp maximum matching algorithm.
57 std::vector<std::pair<int, int>> hopcroft_karp_maximum_matching() {
58     // Build the level graph and then find a blocking flow.
59     while (bfs()) {
60         for (int u = 0; u < n1; ++u) {
61             if (ma[u] == -1) {
62                 dfs(u);
63             }
64         }
65     }
66     // Collect the matched pairs.
67     std::vector<std::pair<int, int>> matches;
68     matches.reserve(n1);
69     for (int u = 0; u < n1; ++u) {
70         if (ma[u] != -1) {
71             matches.emplace_back(u, ma[u]);
72         }
73     }
74     return matches;
75 }
76 };
77
78 signed main() {
79     int n1, n2, m;
80     std::cin >> n1 >> n2 >> m;
81     BipartiteGraph gr(n1 + 3, n2 + 3); // 注意这里是0-base
82     // 左边编号为0~n1-1
83     // 右边编号为0~n2-1
84     // 两边一共m条边
85     for (int i = 0; i < m; ++i) {
86         int u, v;
87         std::cin >> u >> v;
88         gr.add_edge(u, v);
89         // 只连单向边
90     }
91     auto res = gr.hopcroft_karp_maximum_matching();
92     std::cout << res.size() << '\n'; // 最大匹配数
93     for (int i = 0; i < res.size(); ++i) {
94         std::cout << res[i].first << ' ' << res[i].second << '\n';
95         // 最大匹配方案
96     }
97     return 0;
98 }

```

C++

注意，此模板0 — base!!

8. 树上差分

树上选择 m 条路径，每次选择一条 u 到 v 的路径

求最后每个点经过了多少次

那么可以用差分思想： $cnt_u + 1, cnt_v + 1, cnt_{fa[lca(u,v)]} - 2$

再从叶子节点往上求和即可

八. 字符串

1. Manacher-马拉车算法（回文字符串）

定义： $r[i]$ 表示 $[i - r[i], i + r[i]]$ 都是回文串且 $[i - r[i] - 1, i + r[i] + 1]$ 不是

初始化：

e.g. 原字符串是`"abcd"`，现初始化为`"@#a#b#c#d#&"`

前后的`@`是为了防止越界，中间的隔板`#`是为了更方便的计算 $r[i]$

对于某一点 mid ，若当前点 i 在 mid 的回文串内 ($i \leq mid + r[mid]$)

那么 $r[i]$ 的初始值可以根据 mid 对称过去，即 $r[i] = r[2 * mid - i]$

但是 $i - r[i]$ 之前的不在 mid 的回文内，所以如果 $i + r[i] > mid + r[mid]$ 就不行

所以有：

```
1 if(i ≤ mid+r[mid]) r[i]=min(r[2*mid-i], mid+r[mid]-i+1);
2 else r[i]=1;
```

C++

后面就一个一个的往外跳

```
1 while(s[i-r[i]]==s[i+r[i]]) r[i]++;
```

C++

Code:

```
1 for(int i=2;i<=L-2;i++){
2     if(i<=mid+r[mid])r[i]=min(r[2*mid-i],mid+r[mid]-i+1);
3     else r[i]=1;
4     while(s[i-r[i]]==s[i+r[i]])r[i]++;
5     r[i]--; //r[i]最后会多一位
6     if(i+r[i]>mid+r[mid])mid=i; //更新最后的回文
7     ans=max(ans,r[i]); //更新答案
8 }
```

C++

Code

```
1 #include<bits/stdc++.h>
2 #define ll long long
3 using namespace std;
4 const int N=2e7+2e6+10,M=1e9+7;
5 int n,ans,f[N],num;
6 string s1;
7 char s[N];
8 ll read(){
9     ll x=0,f=1;char ch=getchar();
10    while(ch<'0' || ch>'9'){if(ch=='-')f=-1;ch=getchar();}
11    while(ch>='0'&&ch<='9'){x=x*10+ch-'0';ch=getchar();}
12    return x*f;
13 }
14 void Pre(){
15     s[0]='&';
16     s[1]='#';
17     int L=s1.size();
18     num=1;
19     for(int i=0;i<L;i++)s[++num]=s1[i],s[++num]='#';
20     s[++num]='^';
21 }
22 int main(){
23     getline(cin,s1)
24     Pre();int mid=0;
25     for(int i=2;i<=num-2;i++){
26         if(i<=mid+f[mid])f[i]=min(f[2*mid-i],mid+f[mid]-i+1);
27         else f[i]=1;
28         while(s[i-f[i]]==s[i+f[i]])f[i]++;
29         f[i]--;
30         if(i+f[i]>mid+f[mid])mid=i;
31         ans=max(ans,f[mid]);
32     }
33     cout<<ans;
34     return 0;
35 }
```

C++

2. 双模数哈希（降低冲突率）

使用哈希的时候，一个模数必然能构造出冲突样例

所以如果真的有某道题卡你哈希，那请一定使用双模数哈希

具体做法

其实非常非常的简单。使用两个模数即可，冲突概率呈指数级下降

```
1 h1[i] = (h1[i - 1] * p + s[i]) % M1;
2 h2[i] = (h2[i - 1] * p + s[i]) % M2; // 第二个模数
```

C++

只有在两个模数下都相同才能说明相同

3. KMP

KMP本质是用于找字符串前缀函数

前缀函数：对于字符串 s ，假设 $p_i = x (x < i)$ ，那么 $s[1 \dots x] = s[i - x + 1 \dots i]$ （子串），且 p_i 是满足条件中最大值

可以发现，如果 $s_i = s_{p_{i-1}+1}$ ，那么 $s_i = p_{i-1} + 1$ （比较 s_{i-1} 的最大前缀）

由于 $s[1 \dots p_{i-1}] = s[i - p_{i-1} \dots i - 1]$ ，那么对于 $s_{p_{i-1}}$ ，可以继续比较它的前缀+1是否和 s_i 相等（相当于比较 s_{i-1} 的次大前缀）

```
1 p[1] = 0; // 第一个设置为0，否则会出错
2 for(int i = 2; i ≤ n; i++) {
3     int j = p[i - 1]; // 初始判断i-1的前缀
4     while(true) {
5         if(s[j + 1] == s[i] || !j) break; // 匹配成功或者匹配到头
6         j = p[j]; // 匹配不成功，往前跳
7     }
8     if(s[j + 1] == s[i]) j++;
9     p[i] = j;
10 }
```

C++

t 为文本串， s 为匹配串，需要求 s 在 t 中出现的位置

那么先对 s 进行一次KMP，然后再用 t 求在 s 中的最大前缀即可

```
1 cin >> t >> s;
2 n = s.size();
3 m = t.size();
4 t = " " + t;
```

```

5  s = " " + s;
6  p[1] = 0; // 第一个设置为0, 否则会出错
7  for(int i = 2; i ≤ n; i++) {
8      int j = p[i - 1]; // 初始判断i-1的前缀
9      while(true) {
10         if(s[j + 1] == s[i] || !j) break; // 匹配成功或者匹配到头
11         j = p[j]; // 匹配不成功, 往前跳
12     }
13     if(s[j + 1] == s[i]) j++;
14     p[i] = j;
15 }
16 int j = 0; // 这里无法更新t对于s的前缀, 所以在循环外定义
17 for(int i = 1; i ≤ m; i++) {
18     while(true) {
19         if(!j || s[j + 1] == t[i]) break;
20         j = p[j];
21     }
22     if(s[j + 1] == t[i]) j++;
23     if(j == n) {
24         cout << i - j + 1 << endl;
25     }
26 }

```

C++

4. Trie

```

1  struct Trie {
2      int son[30], cnt, fail;
3  } tr[N];
4  void build(string s) {
5      int u = 0;
6      for(int i = 0; i < s.size(); i++) {
7          int c = s[i] - 'a'; // 注意这里减的是'a'
8          if(!tr[u].son[c]) tr[u].son[c] = ++num;
9          u = tr[u].son[c];
10     }
11     tr[u].cnt++; // 结尾标记
12 }

```

C++

5. AC自动机

AC自动机解决的是 多模式串匹配问题

比如, 有多少个模式串在文本串中出现过

- 1 步骤:
- 2 1. 建立Trie树
- 3 2. 用fail数组记录最长后缀 (类似于kmp)
- 4 3. 求答案

```

5  第二步: 如果tr[fail[i]][c]存在, fail[i]=tr[fail[fa[i]]][c]
6  struct Trie {
7      int son[30], cnt, fail;
8  } tr[N];
9  void build(string s) {
10     int u = 0;
11     for(int i = 0; i < s.size(); i++) {
12         int c = s[i] - 'a'; //注意这里减的是'a'
13         if(!tr[u].son[c]) tr[u].son[c] = ++num;
14         u = tr[u].son[c];
15     }
16     tr[u].cnt++;
17 }
18 void getfail() {
19     //bfs一层一层求
20     //fail求的是最长后缀
21     queue<int> q;
22     for(int i = 0; i ≤ 25; i++) {
23         if(tr[0].son[i]) {
24             q.push(tr[0].son[i]);
25         }
26     }
27     //第一层的fail全为0, 从第二层开始
28     while(!q.empty()) {
29         int u = q.front();
30         q.pop();
31         for(int i = 0; i ≤ 25; i++) {
32             //下一层
33             if(tr[u].son[i]) {
34                 q.push(tr[u].son[i]);
35                 tr[tr[u].son[i]].fail = tr[tr[u].fail].son[i];
36                 //now的后缀和fail[now]的后缀一样
37                 //所以tr[now][i]的后缀和tr[fail[now]][i]的后缀一样
38                 //(如果存在的话)
39             }
40             else tr[u].son[i] = tr[tr[u].fail].son[i];
41             //如果tr[now][i]不存在, 那么
42             //相当于构建一个虚拟的点, 这个点指向tr[fail[now]][i]
43             //这样会一直跳知道遇到一个符合条件的位置
44         }
45     }
46 }
47 int solve(string t) { //求答案, t是文本串
48     //跟kmp一样, 不断往前跳, 直到找到一个模式串
49     //然后清空模式串数量
50     int ans = 0, u = 0;
51     for(int i = 0; i < t.size(); i++) {
52         int c = t[i] - 'a'; //注意这里减的是'a'
53         u = tr[u].son[c]; //放在这里
54         for(int j = u; j && tr[j].cnt ≠ -1; j = tr[j].fail) {
55             //tr[j].cnt=-1说明已经来过, 直接退出
56             ans += tr[j].cnt;
57             tr[j].cnt = -1;
58             //有些时候也不用.cnt=-1退出
59         }
60     }
61     return ans;
62 }

```

5.1 AC自动机的优化

用 j 不断往上跳 $fail$ 的时间复杂度很大，有时候会 TLE

可以发现 $fail$ 构建的其实是一棵树，所以用拓扑序计算答案即可线性时间内求出答案

求的是 n 个不同模式串在文本串中各自出现了多少次

```
1 void getfail() {
2     //bfs一层一层求
3     //fail求的是最长后缀
4     queue<int> q;
5     for(int i = 0; i ≤ 25; i++) {
6         if(tr[0].son[i]) {
7             q.push(tr[0].son[i]);
8         }
9     }
10    //第一层的fail全为0，从第二层开始
11    while(!q.empty()) {
12        int u = q.front();
13        q.pop();
14        for(int i = 0; i ≤ 25; i++) {
15            //下一层
16            if(tr[u].son[i]) {
17                q.push(tr[u].son[i]);
18                tr[tr[u].son[i]].fail = tr[tr[u].fail].son[i];
19                tr[tr[tr[u].fail].son[i]].ru++; //新增项，指向
                tr[tr[u].fail].son[i]的入度+1
20                //拓扑序需要
21            }
22            else tr[u].son[i] = tr[tr[u].fail].son[i];
23        }
24    }
25 }
26 void solve(string t) {
27     int ans = 0, u = 0;
28     //初状态赋值
29     for(int i = 0; i < t.size(); i++) {
30         int c = t[i] - 'a'; //注意这里减的是'a'
31         u = tr[u].son[c];
32         tr[u].w++; //初始值
33     }
34 }
35 void topu() {
36     queue<int> q;
37     for(int i = 0; i ≤ num; i++) {
38         if(!tr[i].ru) {
39             q.push(i);
40             //把入度为0的初始点加入队列
41         }
42     }
43     while(!q.empty()) {
44         int u = q.front();
45         int v = tr[u].fail; //这是当前点指向的父亲节点
46         q.pop();
47         ans[tr[u].id] = tr[u].w; //答案赋值，每个串出现了多少次
```

```

48         tr[v].w += tr[u].w; //往前相当于前缀和
49         tr[v].ru--; //去掉这条边
50         if(!tr[v].ru) q.push(v); //入度为0入队
51     }
52 }
53 build(s[1], s[2]...);
54 getfail();
55 solve(t); //注意和前面的不同
56 topu(); //根据拓扑序往上求答案

```

C++

5.3 优化版本各种求法

5.3.1 每个模式串在文本串出现次数

Code

在*solve*里面给*tr[u].w*赋值

```

1  map<string, int> mp;
2  struct Trie {
3      int son[30], cnt, fail, id, w, ru;
4  } tr[N];
5  void build(string s, int id) {
6      int u = 0;
7      for(int i = 0; i < s.size(); i++) {
8          int c = s[i] - 'a';
9          if(!tr[u].son[c]) tr[u].son[c] = ++num;
10         u = tr[u].son[c];
11     }
12     tr[u].id = id;
13 }
14 void getfail() {
15     //bfs一层一层求
16     //fail求的是最长后缀
17     queue<int> q;
18     for(int i = 0; i ≤ 25; i++) {
19         if(tr[0].son[i]) {
20             q.push(tr[0].son[i]);
21         }
22     }
23     //第一层的fail全为0, 从第二层开始
24     while(!q.empty()) {
25         int u = q.front();
26         q.pop();
27         for(int i = 0; i ≤ 25; i++) {
28             //下一层
29             if(tr[u].son[i]) {
30                 q.push(tr[u].son[i]);
31                 tr[tr[u].son[i]].fail = tr[tr[u].fail].son[i];
32                 tr[tr[tr[u].fail].son[i]].ru++; //新增项, 指向
33                 tr[tr[u].fail].son[i]的入度+1
34             }
35             //拓扑序需要
36             else tr[u].son[i] = tr[tr[u].fail].son[i];

```

```

36     }
37 }
38 }
39 void solve(string t) {
40     int u = 0;
41     //初状态赋值
42     for(int i = 0; i < t.size(); i++) {
43         int c = t[i] - 'a';
44         u = tr[u].son[c];
45         tr[u].w++; //初始值
46     }
47 }
48 void topu () {
49     queue<int> q;
50     for(int i = 0; i ≤ num; i++) {
51         if(!tr[i].ru) {
52             q.push(i);
53             //把入度为0的初始点加入队列
54         }
55     }
56     while(!q.empty()) {
57         int u = q.front();
58         int v = tr[u].fail; //这是当前点指向的父亲节点
59         q.pop();
60         ans[tr[u].id] = tr[u].w; //答案赋值
61         tr[v].w += tr[u].w; //往前相当于前缀和
62         tr[v].ru--; //去掉这条边
63         if(!tr[v].ru) q.push(v); //入度为0入队
64     }
65 }
66 signed main() {
67     //freopen("1.in", "r", stdin);
68     string t = "";
69     n = read();
70     for(int i = 1; i ≤ n; i++) {
71         cin >> s[i];
72         if(!mp[s[i]]) mp[s[i]] = i;
73         else continue;
74         build(s[i], i);
75     }
76     getfail();
77     cin >> t;
78     solve(t);
79     topu();
80     for(int i = 1; i ≤ n; i++) {
81         int t = mp[s[i]];
82         cout << ans[t] << endl;
83     }
84     return 0;
85 }

```


5.3.2 无文本串，每个模式串在所有模式串出现次数

Code

在`build`里面给每个`tr[u].w`赋值

```
1  #include<bits/stdc++.h>
2  #define ll long long
3  #define int long long
4  using namespace std;
5  const int N = 1e6 + 200, INF = 2e18;
6  int n, cnt, ans[N], to[N];
7  ll read() {
8      ll x = 0, f = 1; char ch = getchar();
9      while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
10     while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
11     return x * f;
12 }
13 struct Trie {
14     int son[30], id, fail, ru, w;
15 } tr[N];
16 void init() {
17     for(int i = 1; i ≤ n; i++) {
18         ans[i] = 0;
19     }
20     for(int i = 0; i ≤ cnt; i++) {
21         tr[i].w = 0;
22     }
23 }
24
25 void build(string s, int id) {
26     int u = 0;
27     for(int i = 0; i < s.size(); i++) {
28         int v = s[i] - 'a';
29         if(!tr[u].son[v]) tr[u].son[v] = ++cnt;
30         u = tr[u].son[v];
31         tr[u].w++; //在这里++
32     }
33     tr[u].id = id;
34     to[id] = u;
35 }
36 void getfail() {
37     queue<int> q;
38     for(int i = 0; i ≤ 25; i++) {
39         if(tr[0].son[i]) {
40             q.push(tr[0].son[i]);
41         }
42     }
43     while(!q.empty()) {
44         int u = q.front();
45         q.pop();
46         for(int i = 0; i ≤ 25; i++) {
47             if(tr[u].son[i]) {
48                 q.push(tr[u].son[i]);
49                 tr[tr[u].son[i]].fail = tr[tr[u].fail].son[i];
```

```

50         tr[tr[tr[u].fail].son[i]].ru++;
51     }
52     else tr[u].son[i] = tr[tr[u].fail].son[i];
53 }
54 }
55 }
56 void topu() {
57     queue<int> q;
58     for(int i = 0; i ≤ cnt; i++) {
59         if(!tr[i].ru) {
60             q.push(i);
61             // 把入度为0的初始点加入队列
62         }
63     }
64     while(!q.empty()) {
65         int u = q.front();
66         int v = tr[u].fail; // 这是当前点指向的父亲节点
67         q.pop();
68         ans[u] = tr[u].w; // 答案赋值, 每个串出现了多少次
69         // 字典树上从0到u这个字符串出现的次数
70         tr[v].w += tr[u].w; // 往前相当于前缀和
71         tr[v].ru--; // 去掉这条边
72         if(!tr[v].ru) q.push(v); // 入度为0入队
73     }
74 }
75 signed main() {
76     // freopen("1.in", "r", stdin);
77     n = read();
78     string s[202];
79     for(int i = 1; i ≤ n; i++) {
80         cin >> s[i];
81         build(s[i], i);
82     }
83     getfail();
84     topu();
85     for(int i = 1; i ≤ n; i++) {
86         cout << ans[to[i]] << endl;
87     }
88     return 0;
89 }

```

C++

1. 如果只需要判断以文本串第 i 位作为结尾是否能匹配到某个模式串, 那用不到优化, 也不需要 $j = fail[j]$ 这个操作, 直接判断就行了

```

1  struct Trie {
2      int son[30], cnt, fail;
3  } tr[N];
4  void build(string s) {
5      int u = 0;
6      for(int i = 0; i < s.size(); i++) {
7          int c = s[i] - 'a'; // 注意这里减的是'a'
8          if(!tr[u].son[c]) tr[u].son[c] = ++num;
9          u = tr[u].son[c];
10     }

```

```

11     tr[u].cnt++;
12 }
13 void getfail() {
14     //bfs一层一层求
15     //fail求的是最长后缀
16     queue<int> q;
17     for(int i = 0; i ≤ 25; i++) {
18         if(tr[0].son[i]) {
19             q.push(tr[0].son[i]);
20         }
21     }
22     //第一层的fail全为0, 从第二层开始
23     while(!q.empty()) {
24         int u = q.front();
25         q.pop();
26         for(int i = 0; i ≤ 25; i++) {
27             //下一层
28             if(tr[u].son[i]) {
29                 q.push(tr[u].son[i]);
30                 tr[tr[u].son[i]].fail = tr[tr[u].fail].son[i];
31                 //now的后缀和fail[now]的后缀一样
32                 //所以tr[now][i]的后缀和tr[fail[now]][i]的后缀一样
33                 //(如果存在的话)
34             }
35             else tr[u].son[i] = tr[tr[u].fail].son[i];
36             //如果tr[now][i]不存在, 那么
37             //相当于构建一个虚拟的点, 这个点指向tr[fail[now]][i]
38             //这样会一直跳知道遇到一个符合条件的位置
39         }
40     }
41 }
42 int solve(string t) { // 求答案, t是文本串
43     //跟kmp一样, 不断往前跳, 直到找到一个模式串
44     //然后清空模式串数量
45     int ans = 0, u = 0;
46     for(int i = 0; i < t.size(); i++) {
47         int c = t[i] - 'a'; //注意这里减的是'a'
48         u = tr[u].son[c]; //放在这里
49         if(tr[u].id) {
50             //说明这里能匹配到一个模式串
51         }
52     }
53     return ans;
54 }

```

C++

1. (需要精准到每个模式串的计数则需要优化, 否则其实根本不需要回溯 *fail*)

6. 回文自动机 (*PAM*)

经典结论: 一个字符串, 所有不同的回文串最多只有 n 个。

证明：以 s_i 结尾的次长回文串，一定能在以 s_i 结尾的最长回文串中翻转，从而在前面找到一个一样的回文串，所以每个位置贡献最多+1

而回文自动机可以找到所有本质不同的字符串，以及以这个字符串结尾的最长回文后缀是哪个字符串

回文自动机可以完成的操作：

1. 一个字符串中，所有回文串有哪些，以及每种回文串出现的次数
2. 以 s_i 结尾的回文串有多少个

可以很显然的发现，当把回文串取半之后，往前只需要找到最长后缀就能不断回溯，这和AC自动机是一样的，所以树上挂的边是一半的回文串

唯一区别在于，PAM需要分一下长度是奇数还是偶数

需要记录的数组（都是在树上）：

Len_u ：以 u 结尾的回文串有多长

$fail_u$ ：失配指针，和AC自动机一样，最长的后缀能匹配到的前缀，在回文自动机里的含义是以 u 结尾的次大回文后缀的位置

cnt_u ：以 u 作为结尾的回文串有多少个，显然每次刚好增加1

注意，初始状态有两个根节点，0代表长度为偶数，1代表长度为奇数，还有 $fail_0 = 1$ ，即0号点指向1号点， $len_1 = -1$

```
1  #include<bits/stdc++.h>
2  #define ll long long
3  #define int long long
4  using namespace std;
5  const int N = 1e6 + 200, INF = 2e18;
6  int n;
7  string s;
8  ll read() {
9      ll x = 0, f = 1; char ch = getchar();
10     while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
11     while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
12     return x * f;
13 }
14 struct Trie {
15     int cnt, son[30], fail, len, w;
16 } tr[N];
17 int getfail(int u, int t) {
18     while(true) {
19         if(t - tr[u].len - 1 ≥ 0 && s[t - tr[u].len - 1] == s[t]) {
20             // 找到符合条件的最长的
21             break;
22         }
23         u = tr[u].fail; // 否则回溯
24     }
25     return u;
26 }
```

```

27 int num = 1; //注意，从1开始，因为一开始就有两个根节点了
28 void build() {
29     tr[0].fail = 1;
30     tr[0].len = 0;
31     tr[1].fail = 0;
32     tr[1].len = -1;
33     num = 1; //因为有两个根节点，所以从1开始
34     cin >> s;
35     int u = 0;
36     for(int i = 0; i < s.size(); i++) {
37         int v = s[i] - 'a';
38         int p = getfail(u, i); //找到以i结尾的最长回文位置
39         //即最大的后缀回文
40         if(!tr[p].son[v]) {
41             //如果不存在，那么只能新建一个点
42             tr[++num].fail = tr[getfail(tr[p].fail, i)].son[v];
43             tr[p].son[v] = num; //这个必须放在后面！！
44             //fail指向的是次大的后缀回文
45             tr[num].len = tr[p].len + 2; //回文长度
46             tr[num].cnt = tr[tr[num].fail].cnt + 1; //以i结尾的回文串的个数
47             //比如：0102010，以最后一位结尾的回文串有010和0102010，所以cnt=2
48         }
49         u = tr[p].son[v];
50         cout << tr[u].cnt << " "; //以i结尾的回文串的个数
51         tr[u].w++; //代表有以u结尾的最长的回文串出现次数
52         //最后拓扑排序一下就能得到所有回文串各自出现次数
53     }
54     for(int i = num; i >= 2; i--) {
55         //枚举所有回文串，一定要从n开始枚举，这样才正确
56         tr[tr[i].fail].w += tr[i].w; //相当于拓扑序
57         //比如010010的fail指向1001
58         //那么1001不仅在别处出现了，也在010010出现了，所以1001出现次数+=010010出现次数
59     }
60 }
61 signed main() {
62     build();
63     return 0;
64 }

```

C++

根据需求使用，*cnt*是以某一位结尾的回文串有多少，*w*是整个字符串中，某个回文出现了多少次

九. 计算几何

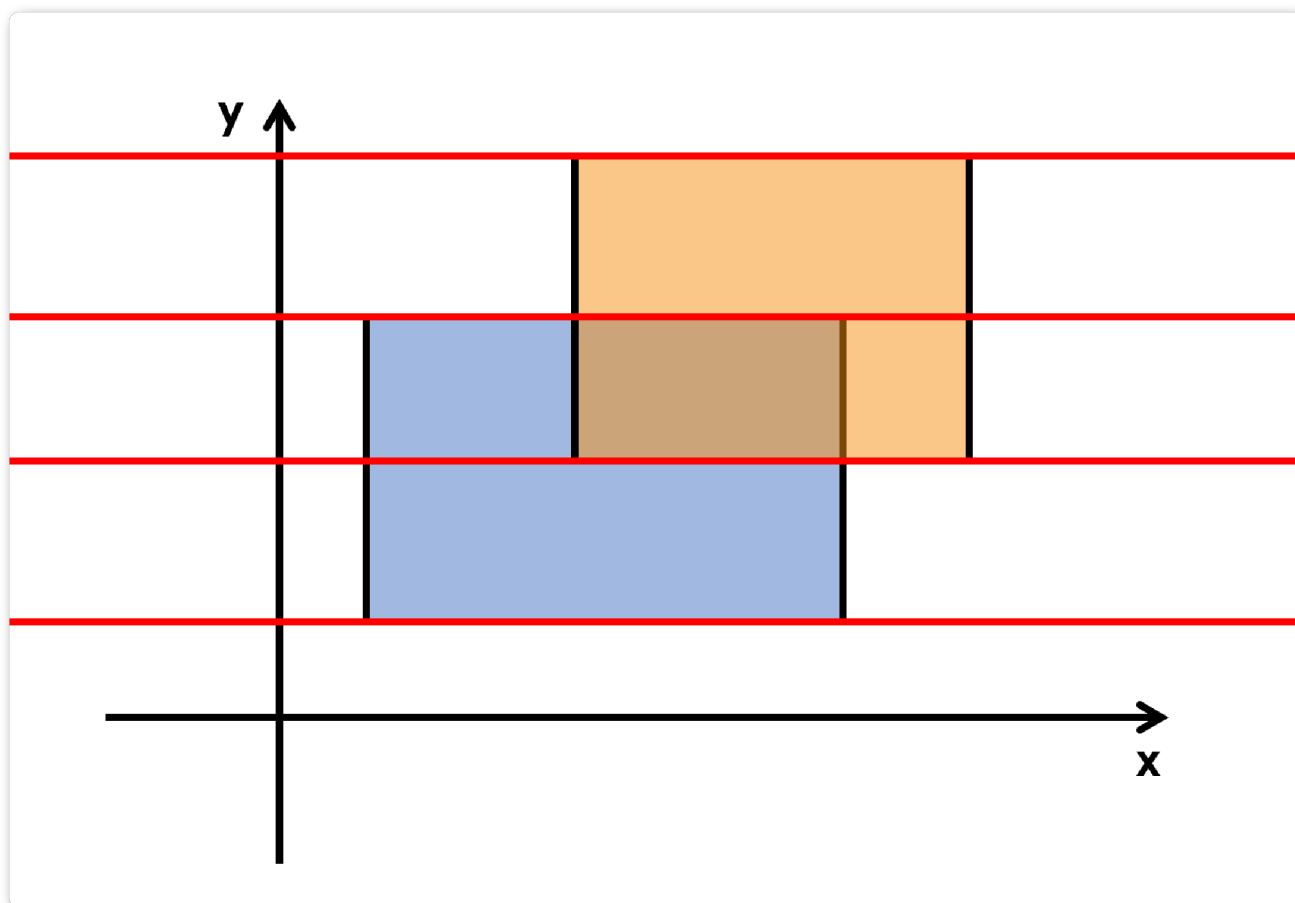
1. 扫描线

题目：P5490 [【模板】扫描线 & 矩形面积并](#)

运用了差分思想

假设有一条无限长的线，从 $y = 0$ 开始向上扫描

那么扫到 $y = i$ 的时候增加的面积即为从 $y = i$ 到 $y = i + 1$ 中间被占据的长度



那么这个长度我们可以用线段树维护，从下到上不断更新，用差分思想

先离散化，假设 L_i 表示离散化后的 l 值

假设线段树区间为 $[l, r]$ ，那么说明是 $L_l \rightarrow L_{r+1}$ 的范围（ L_i 是点，线段树代表的区间，需要分辨）

假设有一个矩形从 (x_1, y_1) 到 (x_2, y_2) 其中 $x_1 < x_2, y_1 < y_2$

那么在扫到 $y = y_1$ 时，将 (x_1, x_2) 增加1

在扫到 $y = y_2$ 的时候，将 (x_1, x_2) 减少1

在 $y = y_i$ 扫描完之后，答案增加 $(y_{i+1} - y_i) * sum[1]$ （ $sum[1]$ 为当前长度）

注意： 本题并不需要`pushdown`，原因如下：

假设我们令 $sumx_p$ 表示编号为 p 的子树被完全覆盖了多少次， sum_p 为当前区间答案

那么假如 $sumx_p > 0$ ，显然这段区间全都应该加上， $sum_p = L_{r+1} - L_l$

假如 $sumx_p = 0$ ，那么 $sum_p = sum_{p*2} + sum_{p*2+1}$

在更新节点时， $sumx_p$ 无法也无需由子树更新，原因如下：

假如 p 所在区间未更新，而 p 的子树有（减少）更新，那说明 p 所在区间至少还有一条线能全覆盖，否则就该轮到 p 这个区间减少更新了，所以子树的更新对父亲节点没有任何影响

1.1 注意！

排序的 cmp 函数不仅要按 r 进行排序， r 相同的时候还要考虑需不需要对其他变量排序，以取得最大值 or 需要求的值等等

（这个问题 参见线段树例题，P1502窗口的星星）

1.2 Code

```
1  #include<bits/stdc++.h>
2  #define ll long long
3  using namespace std;
4  const ll N = 6e6 + 100, M = 998244353, CN = 2e5;
5  ll sum[N], sumx[N], pd[N], n, m, fr[N], ls[N];
6  ll ans;
7  ll read() {
8      ll x = 0, f = 1; char ch = getchar();
9      while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
10     while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
11     return x * f;
12 }
13 map<int, int> mp;
14 struct Point{
15     int l1, r;
16     int c, l2;
17 } s[N];
18 bool cmp(Point x, Point y) {
19     return x.r < y.r;
20     // 本题不用考虑x.r=y.r的情况
21     // 一定要按顺序来，从下往上搜才不会出错
22 }
23 void update_sum(int l, int r, int p, int c) {
24     sumx[p] += c;
25     if(sumx[p] > 0) sum[p] = fr[r + 1] - fr[l];
26     else sum[p] = sum[p * 2] + sum[p * 2 + 1];
27 }
28 void update(int l, int r, int lx, int rx, int p, int c) {
29     if(rx < l || r < lx) return;
30     if(lx ≤ l && r ≤ rx) {
31         update_sum(l, r, p, c);
32         return;
33     }
34     int mid = (l + r) >> 1;
35     update(l, mid, lx, rx, p * 2, c);
36     update(mid + 1, r, lx, rx, p * 2 + 1, c);
37     update_sum(l, r, p, 0); // 搜完之后记得更新
38 }
39 int main() {
40     n = read(); int t = 0, num = 0;
```

```

41     for(int i = 1; i ≤ n; i++) {
42         int l1 = read(), r1 = read();
43         int l2 = read(), r2 = read();
44         if(l1 > l2) swap(l1, l2);
45         if(r1 > r2) swap(r1, r2);
46         //保证大小顺序
47         s[i].l1 = l1, s[i].l2 = l2, s[i].r = r1;
48         s[i + n].l1 = l1, s[i + n].l2 = l2, s[i + n].r = r2;
49         s[i].c = 1, s[i + n].c = -1;
50         ls[++t] = l1;
51         ls[++t] = l2;
52     }
53     sort(ls + 1, ls + t + 1);
54     sort(s + 1, s + 2 * n + 1, cmp);
55     for(int i = 1; i ≤ t; i++) { //离散化
56         if(!mp[ls[i]]) mp[ls[i]] = ++num;
57         fr[mp[ls[i]]] = ls[i];
58     }
59     for(int i = 1; i ≤ 2 * n; i++) { //离散化
60         s[i].l1 = mp[s[i].l1];
61         s[i].l2 = mp[s[i].l2];
62     }
63     s[2 * n + 1].r = s[2 * n]; //边注意界
64     for(int i = 1; i ≤ 2 * n; i++) {
65         update(1, CN, s[i].l1, s[i].l2 - 1, 1, s[i].c);
66         ans += (s[i + 1].r - s[i].r) * sum[1];
67     }
68     cout << ans;
69     return 0;
70 }
71 //月雪·薇婷

```

C++

十. 杂项

1. 莫队

1.1 莫队阅读须知

用莫队别开 `define int long long!!`

你是否因为暴力不够优雅而苦恼？那就快试试莫队吧！

使用条件：

a. 对于 $n, m \leq 5e5$ 的数据

b. 如果是区间离线求答案

那么可以用莫队

众所周知，莫队通过改变区间顺序从而降低复杂度

那么询问区间排序方法到底是什么？

假如某个题的排序需要用到 $x = 2$ or 3 个关键字

A. 对于前2个关键字，一定是先 L 然后 R

B. 对于前 $x - 1$ 个关键字，一定是所在块的编号，最后一个关键字则为本身大小

可以对比带修莫队和普通莫队

1.2 莫队(基础版)

使用条件：

A. $O(n * \text{sqrt}(\min(n, m)))$ 满足时间限制

B. 求的是区间问题，并且 $[l, r]$ 的答案能转移到 $[l + 1, r]$, $[l - 1, r]$, $[l, r + 1]$, $[l, r - 1]$

莫队本质上就是**暴力的优化**（条件B很好的说明了这一点）

那么满足条件后，证明时间复杂度可以见 [Oj - Wiki\(莫队\)](#)

前排提醒：当 $m \ll n$ 的时候，块的区间长度为 $n/\sqrt{m}$ （分成 $\sqrt{m}$ 块）是最优解

1.2.1 解法

A. 首先找到区间之间的关系

B. 然后以查询区间的 L 所在块编号作为第一关键字排序，以 R 作为第二关键字排序

（最重要的一条，这样使得时间复杂度成立）

C. 令 $[l, r]$ 表示当前区间，然后**暴力**计算。需要注意的是 l, r 如何变换

特别注意！！第一关键字和第二关键字有区别！！

```
1 bool cmp1(Mo x, Mo y) {
2     if(vis[x.l] == vis[y.l])
3         return x.r < y.r; // 第二关键字是R！！注意
4     return vis[x.l] < vis[y.l];
5 }
```

C++

特别注意！！有可能出现 $l > r$ ，考虑这种情况下 add 和 sub 函数会不会出问题，这点非常重要！！

1.2.2 Code

```
1 int l = 1, r = 0;
2 for(int i = 1; i ≤ m; i++) {
3     while(l > s[i].l) Solve(--l);
4     while(l < s[i].l) Solve(l++);
5     while(r > s[i].r) Solve(r--);
6     while(r < s[i].r) Solve(++r);
7 }
```

C++

A. 从 $[l, r]$ 到 $[l - 1, r]$: 需要加上 $l - 1$ 位置上的贡献, 所以是 $--l$

B. 从 $[l, r]$ 到 $[l + 1, r]$: 需要减去 l 位置上的贡献, 所以是 $l++$

C. 从 $[l, r]$ 到 $[l, r - 1]$: 需要减去 r 位置上的贡献, 所以是 $r--$

D. 从 $[l, r]$ 到 $[l, r + 1]$: 需要加上 $r + 1$ 位置上的贡献, 所以是 $++r$

根据变化规则, 所以 l 和 r 的初始值分别是1, 0 (可以验算一下)

例题: [小 Z 的袜子](#)

1.2.3 Solution & Code

首先把式子推一下, 发现 $[L, R]$ 区间假如:

有 $a, b, c, \dots$ 颜色的袜子, 数量分别为 $x, y, z, \dots$

那么 $ans = (x * (x - 1) / 2 + y * (y - 1) / 2 + \dots) / ((R - L + 1) * (R - L) / 2)$

即 $ans = (x^2 + y^2 + z^2 + \dots - (x + y + \dots)) / ((R - L + 1) * (R - L) / 2)$

所以 $ans = ((\sum num_i^2) - (R - L + 1)) / ((R - L + 1) * (R - L))$

```
1 #include<bits/stdc++.h>
2 #define ll long long
3 using namespace std;
4 const ll N = 1e5 + 100, M = 998244353, INF = 1e9 + 10;
5 ll ans1, num[N];
6 int n, m, a[N], vis[N];
7 ll read() {
8     ll x = 0, f = 1; char ch = getchar();
9     while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
10    while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
11    return x * f;
12 }
13 struct Mo {
14     int l, r, id;
15 } s[N];
16 struct Ans {
17     ll ans1, ans2;
18 } ans[N];
```

```

19 bool cmp1(Mo x, Mo y) {
20     if(vis[x.l] == vis[y.l])
21         return x.r < y.r;
22     return vis[x.l] < vis[y.l];
23 }
24 void update(int x, ll y) {
25     ans1 = ans1 + 2ll * num[a[x]] * y + 1;
26     num[a[x]] += y;
27 }
28 int main() {
29     n = read(), m = read();
30     for(int i = 1; i ≤ n; i++) a[i] = read();
31     int Len = sqrt(min(n, m)), L = 0, R = 0, t = 0;
32     while(true) {
33         t++;
34         L = R + 1;
35         R = min(L + Len - 1, n);
36         for(int i = L; i ≤ R; i++) vis[i] = t;
37         if(R == n) break;
38     }
39     for(int i = 1; i ≤ m; i++) {
40         s[i].l = read();
41         s[i].r = read();
42         s[i].id = i;
43     }
44     sort(s + 1, s + m + 1, cmp1); //排序
45     int l = 1, r = 0;
46     for(int i = 1; i ≤ m; i++) {
47         while(l > s[i].l) update(--l, 1);
48         while(l < s[i].l) update(l++, -1);
49         while(r > s[i].r) update(r--, -1);
50         while(r < s[i].r) update(++r, 1);
51         ll Lx = s[i].r - s[i].l + 1, u = 0, v = 0;
52         u = ans1 - Lx;
53         v = Lx * (Lx - 1);
54         if(s[i].l == s[i].r) {
55             ans[s[i].id].ans1 = 0;
56             ans[s[i].id].ans2 = 1;
57             continue;
58         }
59         ll g = __gcd(u, v);
60         u /= g, v /= g;
61         ans[s[i].id].ans1 = u; ans[s[i].id].ans2 = v;
62     }
63     for(int i = 1; i ≤ m; i++) {
64         printf("%lld/%lld\n", ans[i].ans1, ans[i].ans2);
65     }
66     return 0;
67 }

```

C++

1.3 回滚莫队

有时候发现进行“减”操作的时候，无法回溯到上一个状态

那么这个时候就要用到回滚莫队

具体来说

A. 假如查询的区间都在一个块内，那直接暴搜即可

(注意暴搜的所有数据和莫队数据是分开的，以防数据污染！)

B. 假如不在一个块内，那么必然 R 所在块的编号大于 L 所在块的编号

I. 假如当前搜索的区间的 l (左端点) 所在的块和上一次搜索的不一样，那么把区间重置

(假设 l 所在块的右端点为 R ，假设当前查询区间为 $[l, r]$)

步骤：

①把 l, r 分别移动到 $R + 1$ 和 R ，移动的同时把所有数据清零

(这里显然 l 必定会往后移，而 r 不确定如何移动，所以都需要写上)

② ans 清零，同时令 $last_block = block[l]$ (更新上一次查找 l 所在的块)

由于排序的时候，按照同一个块内 r 从小到大排序，所以可以保存 $[R, r]$ 的答案

每一次更新答案从这个区间更新 (l 每次都从 R 往前加， r 从上一轮的 r 往后加，如此保证了“减”操作的时候根本不需要更新答案!)，最后把区间从 $[l, r]$ 又减回 $[R, r]$ 即可，减的时候不用更新答案

可以证明时间复杂度依然为 $O(N^{\frac{3}{2}})$

尤其需要注意!!!

A. 排序以 r 的大小为第二关键字，而不是 r 所在块编号!!!

B. 每次 $ansx$ 更新 $[R, r]$ 的答案，而 $ansy$ 更新当前询问的答案，需要分开!!

C. 每次搜寻完之后需要令 l 回溯到 $l = R$

关键部分代码：

```
1 void add(int x, int y) {
2     //进行更改
3     if(y == 1) //更新 ansx
4     if(y == 2) //更新 ansy
5     if(y == 3) //不需要更新答案，因为只是区间重置，反正最后答案也会清空
6 }
7 void sub(int x) {
8     //回溯
9 }
10 for(int i = 1; i ≤ m; i++) {
11     if(s[i].lbl == s[i].rbl) { //s[i].rbl为l所在块的编号
12         int __ansx = 0;
13         //暴搜
14         ans[s[i].id] = __ansx;
15         continue;
16     }
17 }
```

```

18     if(s[i].lbl != last_block) { //s[i].lbl为l所在块的编号
19         last_block = s[i].lbl;
20         while(r > BR[s[i].lbl]) sub(r--); //BR为某个块的右端点位置
21         while(r < BR[s[i].lbl]) add(++r, 3);
22         while(l < BR[s[i].lbl] + 1) sub(l++);
23         ansx = 0;
24     }
25     while(r < s[i].r) add(++r, 1);
26     int lx = l; ansy = ansx;
27     while(lx > s[i].l) add(--lx, 2);
28     ans[s[i].id] = ansy;
29     while(lx < l) sub(lx++); //用lx进行回滚
30 }

```

C++

例题：回滚莫队&不删除莫队

题目地址：[回滚莫队&不删除莫队](#)

Code

```

1  #include<bits/stdc++.h>
2  #define ll long long
3  using namespace std;
4  const ll N = 1e6 + 100, M = 998244353, INF = 1e9 + 10;
5  int n, m, bl[N], a[N], ans[N], ansx, ansy;
6  int NowL[N], NowR[N], ToL[N], ToR[N], BR[N], __NowL[N];
7  ll read() {
8      ll x = 0, f = 1; char ch = getchar();
9      while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
10     while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
11     return x * f;
12 }
13 map<int, int> mp;
14 struct Mo {
15     int l, r, id;
16     int lbl, rbl;
17 } s[N];
18 bool cmp(Mo x, Mo y) {
19     if(bl[x.l] == bl[y.l])
20         return x.r < y.r;
21     return bl[x.l] < bl[y.l];
22 }
23 void add(int x, int y) {
24     if(NowL[a[x]] > x) NowL[a[x]] = x;
25     if(NowR[a[x]] < x) NowR[a[x]] = x;
26     if(y == 1) ansx = max(ansx, NowR[a[x]] - NowL[a[x]]);
27     else ansy = max(ansy, NowR[a[x]] - NowL[a[x]]);
28 }
29 void sub(int x) {
30     if(NowL[a[x]] == x) NowL[a[x]] = ToR[x];
31     if(NowR[a[x]] == x) NowR[a[x]] = ToL[x];
32 }
33 int main() {
34     n = read(); int cnt = 0;

```

```

35     for(int i = 1; i ≤ n; i++) {
36         a[i] = read();
37         if(!mp[a[i]]) mp[a[i]] = ++cnt;
38         a[i] = mp[a[i]]; //离散化
39     }
40     //ToR, ToL分别求i处这个数右边, 左边跟它最近的相同的数的下标
41     mp.clear();
42     for(int i = 1; i ≤ n; i++) {
43         if(mp[a[i]]) ToL[i] = mp[a[i]];
44         mp[a[i]] = i;
45     }
46     mp.clear();
47     for(int i = n; i ≥ 1; i--) {
48         ToR[i] = n + 1;
49         if(mp[a[i]]) ToR[i] = mp[a[i]];
50         mp[a[i]] = i;
51     }
52
53     int Len = pow(n, 1.0 / 2.0);
54     int L = 0, R = 0, px = 0;
55     while(true) {
56         px++;
57         L = R + 1;
58         R = min(L + Len - 1, n);
59         BR[px] = R;
60         for(int i = L; i ≤ R; i++) bl[i] = px;
61         if(R == n) break;
62     }
63
64     m = read();
65     for(int i = 1; i ≤ m; i++) {
66         s[i].l = read();
67         s[i].r = read();
68         s[i].id = i;
69         s[i].lbl = bl[s[i].l];
70         s[i].rbl = bl[s[i].r];
71     }
72
73     sort(s + 1, s + m + 1, cmp);
74     int l = 1, r = 0, last_block = 0;
75     for(int i = 1; i ≤ n; i++) NowL[i] = n + 1; //NowL, NowR为某个数在区间内最右边
和最左边的位置
76     int F = 0;
77     for(int i = 1; i ≤ m; i++) {
78         if(s[i].lbl == s[i].rbl) {
79             int __ansx = 0;
80             for(int j = s[i].l; j ≤ s[i].r; j++)
81                 __NowL[a[j]] = n + 1;
82             for(int j = s[i].l; j ≤ s[i].r; j++) {
83                 if(__NowL[a[j]] == n + 1) __NowL[a[j]] = j;
84                 __ansx = max(__ansx, j - __NowL[a[j]]);
85             }
86             ans[s[i].id] = __ansx;
87             continue;
88         }
89         if(s[i].lbl ≠ last_block) {
90             last_block = s[i].lbl;
91             while(r > BR[s[i].lbl]) sub(r--);

```

```

92         while(r < BR[s[i].lbl]) add(++r, 3);
93         while(l < BR[s[i].lbl] + 1) sub(l++);
94         ansx = 0;
95     }
96     while(r < s[i].r) add(++r, 1);
97     int lx = l; ansy = ansx;
98     while(lx > s[i].l) add(--lx, 2);
99     ans[s[i].id] = ansy;
100    while(lx < l) sub(lx++);
101
102    }
103    for(int i = 1; i ≤ m; i++) cout << ans[i] << endl;
104    return 0;
105 }
106

```

C++

例题2：历史研究

题目地址：[历史研究](#)

Code

```

1  #include<bits/stdc++.h>
2  #define ll long long
3  using namespace std;
4  const ll N = 1e6 + 100, M = 998244353, INF = 1e9 + 10;
5  ll n, m, bl[N], a[N], ans[N], ansx, ansy, num[N];
6  ll BR[N], _num[N], b[N];
7  ll read() {
8      ll x = 0, f = 1; char ch = getchar();
9      while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
10     while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
11     return x * f;
12 }
13 map<int, int> mp;
14 struct Mo {
15     int l, r, id;
16     int lbl, rbl;
17 } s[N];
18 bool cmp(Mo x, Mo y) {
19     if(bl[x.l] == bl[y.l])
20         return x.r < y.r;
21     return bl[x.l] < bl[y.l];
22 }
23 void add(int x, int y) {
24     num[a[x]]++;
25     if(y == 1) ansx = max(ansx, num[a[x]] * b[a[x]]);
26     else ansy = max(ansy, num[a[x]] * b[a[x]]);
27 }
28 void sub(int x) {
29     num[a[x]]--;
30 }
31 int main() {
32     n = read(); m = read(); int cnt = 0;

```

```

33     for(int i = 1; i ≤ n; i++) {
34         a[i] = read();
35         if(!mp[a[i]]) mp[a[i]] = ++cnt, b[cnt] = a[i];
36         a[i] = mp[a[i]]; //离散化
37     }
38     int Len = pow(n, 1.0 / 2.0);
39     ll L = 0, R = 0, px = 0;
40     while(true) {
41         px++;
42         L = R + 1;
43         R = min(L + Len - 1, n);
44         BR[px] = R;
45         for(int i = L; i ≤ R; i++) bl[i] = px;
46         if(R == n) break;
47     }
48
49     for(int i = 1; i ≤ m; i++) {
50         s[i].l = read();
51         s[i].r = read();
52         s[i].id = i;
53         s[i].lbl = bl[s[i].l];
54         s[i].rbl = bl[s[i].r];
55     }
56     sort(s + 1, s + m + 1, cmp);
57     ll l = 1, r = 0, last_block = 0;
58     for(int i = 1; i ≤ m; i++) {
59         if(s[i].lbl == s[i].rbl) {
60             ll _ansx = 0;
61             for(int j = s[i].l; j ≤ s[i].r; j++) _num[a[j]] = 0;
62             for(int j = s[i].l; j ≤ s[i].r; j++) {
63                 _num[a[j]]++;
64                 _ansx = max(_ansx, _num[a[j]] * b[a[j]]);
65             }
66             ans[s[i].id] = _ansx;
67             continue;
68         }
69         if(s[i].lbl ≠ last_block) {
70             last_block = s[i].lbl;
71             while(r > BR[s[i].lbl]) sub(r--);
72             while(r < BR[s[i].lbl]) add(++r, 3);
73             while(l < BR[s[i].lbl] + 1) sub(l++);
74             ansx = 0;
75         }
76         while(r < s[i].r) add(++r, 1);
77         int lx = l; ansy = ansx;
78         while(lx > s[i].l) add(--lx, 2);
79         ans[s[i].id] = ansy;
80         while(lx < l) sub(lx++);
81     }
82 }
83 for(int i = 1; i ≤ m; i++) cout << ans[i] << endl;
84 return 0;
85 }

```


1.4 带修莫队(带修改)

说实话感觉用处不是特别大, 限制比较多

带修莫队和普通莫队区别不大

因为带了修改, 那么我们考虑把修改也看成一维(当作时间节点来算)(原先的 $[l, r]$ 算作两维)

同时记录 $[l, r]$ 所处的时间, 这样修改 $[l, r]$ 的时候同时就能知道该怎么修改了

设置区间长度为 $n^{\frac{2}{3}}$ 为最优解, 时间复杂度为 $O(n^{\frac{5}{3}})$ (所以限制很大)

注意!! 带修莫队 仅支持 单点修改!!

1.4.1 New Code

新增代码:

```
1 void timeUp()  
2 void timeDown()  
3 int t = 0; //t和r差不多  
4 for(int i = 1; i ≤ q; i++) {  
5     while(t < s[i].t) timeUp(++t, l, r);  
6     while(t > s[i].t) timeDown(t--, l, r);  
7 }
```

C++

*timeUp*和*timeDown*都有两个操作

A. 判断修改区间是否被当前区间包含(这一步也比较重要), 若不包含则无需修改当前答案

B. 无论是否在区间内, 都必须交换修改条件(详情见下)

注意: *timeUp*和*timeDown*修改的时候需要*swap*一下

比如, 操作是把 a_i 的值改成 b_j , 那么需要 $swap(a_i, b_j)$

这样操作能满足必要条件:

A. a_i 的值一定正确, 是最新状态的值

还有一点, 用*swap*的*compare(cmp)*的时候, *time*维度放在最后一层

注意!! 和普通莫队cmp不一样

```

1 bool cmp(Mo x, Mo y) {
2     if(bl[x.l] == bl[y.l]) {
3         if(bl[x.r] == bl[y.r])
4             return x.t < y.t;
5         return bl[x.r] < bl[y.r];
6     }
7     return bl[x.l] < bl[y.l];
8 }

```

C++

例题：数颜色 / 维护队列

题目地址：[数颜色 / 维护队列](#)

Code

```

1  #include<bits/stdc++.h>
2  #define ll long long
3  using namespace std;
4  const ll N = 1e6 + 100, M = 998244353, INF = 1e9 + 10;
5  int n, m, bl[N], num[N], a[N], ans[N], ansx;
6  ll read() {
7      ll x = 0, f = 1; char ch = getchar();
8      while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
9      while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
10     return x * f;
11 }
12 struct Mo {
13     int l, r, t, id;
14 } s[N];
15 struct R {
16     int p, c;
17 } rc[N];
18 bool cmp(Mo x, Mo y) {
19     if(bl[x.l] == bl[y.l]) {
20         if(bl[x.r] == bl[y.r])
21             return x.t < y.t;
22         return bl[x.r] < bl[y.r];
23     }
24     return bl[x.l] < bl[y.l];
25 }
26 void add(int x) {
27     if(!num[x]) ansx++;
28     num[x]++;
29 }
30 void sub(int x) {
31     num[x]--;
32     if(!num[x]) ansx--;
33 }
34 void timeup(int x, int l, int r) {
35     if(rc[x].p ≥ l && rc[x].p ≤ r) {//需要在区间才能修改
36         add(rc[x].c);

```

```

37     sub(a[rc[x].p]);
38 }
39 swap(a[rc[x].p], rc[x].c); //必须swap
40 }
41 void timedown(int x, int l, int r) {
42     if(rc[x].p ≥ l && rc[x].p ≤ r) {
43         add(rc[x].c);
44         sub(a[rc[x].p]);
45     }
46     swap(a[rc[x].p], rc[x].c);
47 }
48 int main() {
49     n = read(), m = read();
50     for(int i = 1; i ≤ n; i++) a[i] = read();
51     int Len = pow(n, 2.0 / 3.0), L = 0, R = 0, px = 0;
52     while(true) {
53         px++;
54         L = R + 1;
55         R = min(L + Len - 1, n);
56         for(int i = L; i ≤ R; i++) bl[i] = px;
57         if(R == n) break;
58     }
59     int cntr = 0, cntq = 0;
60     for(int i = 1; i ≤ m; i++) {
61         string x; int l = 0, r = 0;
62         cin >> x;
63         l = read(), r = read();
64         if(x[0] == 'Q') {
65             s[++cntq].l = l, s[cntq].r = r;
66             s[cntq].id = cntq, s[cntq].t = cntr;
67             //当前存了多少修改即可看做当前的时间
68         }
69         else {
70             //询问和修改分开保存
71             rc[++cntr].p = l, rc[cntr].c = r;
72         }
73     }
74     sort(s + 1, s + cntq + 1, cmp);
75     int l = 1, r = 0, t = 0;
76     for(int i = 1; i ≤ cntq; i++) {
77         while(l < s[i].l) sub(a[l++]);
78         while(l > s[i].l) add(a[--l]);
79         while(r < s[i].r) add(a[++r]);
80         while(r > s[i].r) sub(a[r--]);
81         //新增维度: 时间维度
82         while(t < s[i].t) timeup(++t, l, r);
83         while(t > s[i].t) timedown(t--, l, r);
84         ans[s[i].id] = ansx;
85     }
86     for(int i = 1; i ≤ cntq; i++) {
87         printf("%d\n", ans[i]);
88     }
89     return 0;
90 }

```

1.5 莫队·例题

1.5.1 大爷的字符串题

题目地址: [大爷的字符串题](#)

Code

```
1  #include<bits/stdc++.h>
2  #define ll long long
3  using namespace std;
4  const ll N = 1e6 + 100, M = 998244353, INF = 1e9 + 10;
5  int n, m, bl[N], ansx = 0, a[N], num[N], ans[N], numx[N];
6  ll read() {
7      ll x = 0, f = 1; char ch = getchar();
8      while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
9      while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
10     return x * f;
11 }
12 map<int, int> mp;
13 struct Mo {
14     int l, r, id;
15 } s[N];
16 bool cmp(Mo x, Mo y) {
17     if(bl[x.l] == bl[y.l])
18         return x.r < y.r;
19     return bl[x.l] < bl[y.l];
20 }
21 void add(int x) {
22     numx[num[a[x]]]--;
23     num[a[x]]++;
24     numx[num[a[x]]]++;
25     ansx = max(ansx, num[a[x]]);
26 }
27 void sub(int x) {
28     numx[num[a[x]]]--;
29     if(num[a[x]] == ansx) {
30         if(!numx[num[a[x]]]) ansx--;
31     }
32     num[a[x]]--;
33     numx[num[a[x]]]++;
34 }
35 int main() {
36     n = read(), m = read();
37     int Len = pow(min(n, m), 1.0 / 2.0);
38     int L = 0, R = 0, blt = 0, cnt = 0;
39     for(int i = 1; i ≤ n; i++) {
40         a[i] = read();
41         if(!mp[a[i]]) mp[a[i]] = ++cnt;
42         a[i] = mp[a[i]];
43     }
44     while(true) {
45         blt++;
46         L = R + 1;
47         R = min(L + Len - 1, n);
```

```

48     for(int i = L; i ≤ R; i++) bl[i] = blt;
49     if(R == n) break;
50 }
51 for(int i = 1; i ≤ m; i++) {
52     s[i].l = read();
53     s[i].r = read();
54     s[i].id = i;
55 }
56 sort(s + 1, s + m + 1, cmp);
57 int l = 1, r = 0;
58 numx[0] = INF;
59 for(int i = 1; i ≤ m; i++) {
60     while(l < s[i].l) sub(l++);
61     while(l > s[i].l) add(--l);
62     while(r < s[i].r) add(++r);
63     while(r > s[i].r) sub(r--);
64     ans[s[i].id] = ansx;
65 }
66 for(int i = 1; i ≤ m; i++) printf("%d\n", -ans[i]);
67 return 0;
68 }

```

C++

1.5.2 小清新人渣的本愿

题目地址: [小清新人渣的本愿](#)

Solution

考虑莫队

思考一个问题: 假设我们已知区间 $[l, r]$ 中出现过哪些数, 如何验证是否存在两个数 a, b (可以重复选择) 满足 $a + b = x$ 或者 $a - b = x$ 或者 $a * b = x$

对于 $a * b = x$ 很好做, 直接 $O(\sqrt{x})$ 暴搜即可做

对于 $a + b = x$ 和 $a - b = x$, 可以发现 a, b 都非常小, 所以可以直接用BitSet做

其实这种思想还挺模板化的

用bitset记录哪些数出现过 (假设 $bitset < N + 10 >$ $bs1, bs2$; $bs2[i] = bs1[N - i]$)

A. 对于 $a - b = x$ 是否存在, 验证:

```
1 bs3 = bs1 & (bs1 >> x)
```

C++

若 $bs3$ 不等于0, 那么说明至少存在一组 a, b 满足 $a - b = x$

B. 对于 $a + b = x$ 是否存在, 验证:

```
1 bs3 = bs1 & (bs2 >> (N - x))
```

C++

若 $bs3$ 不等于0, 那么说明至少存在一组 a, b 满足 $a + b = x$

 1749738432791-screenshot

Code

```
1  #include<bits/stdc++.h>
2  #define ll long long
3  using namespace std;
4  const ll N = 1e5 + 100, M = 998244353, INF = 1e9 + 10, Nx = 1e5;
5  int n, m, bl[N], ansx = 0, a[N], num[N], ans[N], ansy, cnt[4];
6  int BR[N], _vis[N];
7  ll read() {
8      ll x = 0, f = 1; char ch = getchar();
9      while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
10     while(ch ≥ '0' && ch ≤ '9') {x = x * 10 + ch - '0'; ch = getchar();}
11     return x * f;
12 }
13 bitset<N> bt1, bt2;
14 struct Mo {
15     int l, r, id, x, op;
16 } s[N];
17 bool cmp(Mo x, Mo y) {
18     if(bl[x.l] == bl[y.l])
19         return x.r < y.r;
20     return bl[x.l] < bl[y.l];
21 }
22
23 void add(int x) {
24     num[a[x]]++;
25     bt1[a[x]] = true;
26     bt2[Nx - a[x]] = true;
27 }
28 void sub(int x) {
29     num[a[x]]--;
30     if(!num[a[x]]) {
31         bt1[a[x]] = false;
32         bt2[Nx - a[x]] = false;
33     }
34 }
35
36 void Solve() {
37     sort(s + 1, s + m + 1, cmp);
38     ansx = ansy = 0;
39     int l = 1, r = 0;
40     for(int i = 1; i ≤ m; i++) {
41         while(r < s[i].r) add(++r);
42         while(r > s[i].r) sub(r--);
43         while(l < s[i].l) sub(l++);
44         while(l > s[i].l) add(--l);
45         if(s[i].op == 1) {
46             bitset<N> bt3 = bt1 & (bt1 >> s[i].x);
47             if(bt3.count()) {
48                 ans[s[i].id] = true;
49                 continue;
50             }
51             ans[s[i].id] = false;
```

```

52     }
53     else if(s[i].op == 2) {
54         bitset<N> bt3 = bt1 & (bt2 >> (Nx - s[i].x));
55         if(bt3.count()) {
56             ans[s[i].id] = true;
57             continue;
58         }
59         ans[s[i].id] = false;
60     }
61     else {
62         for(int j = 1; j ≤ sqrt(s[i].x); j++) {
63             if(s[i].x % j ≠ 0) continue;
64             if(bt1[j] && bt1[(s[i].x / j)]) {
65                 ans[s[i].id] = true;
66                 break;
67             }
68         }
69     }
70 }
71 }
72
73 int main() {
74     n = read(), m = read();
75     int Len = pow(min(n, m), 1.0 / 2.0);
76     int L = 0, R = 0, blt = 0;
77     for(int i = 1; i ≤ n; i++) a[i] = read();
78     while(true) {
79         blt++;
80         L = R + 1;
81         R = min(L + Len - 1, n);
82         for(int i = L; i ≤ R; i++) bl[i] = blt;
83         if(R == n) break;
84     }
85     for(int i = 1; i ≤ m; i++) {
86         int op = read(), l = read(), r = read(), x = read();
87         s[i].l = l;
88         s[i].r = r;
89         s[i].id = i;
90         s[i].x = x;
91         s[i].op = op;
92     }
93     Solve();
94     for(int i = 1; i ≤ m; i++) {
95         if(ans[i] == true) printf("hana\n");
96         else printf("bi\n");
97     }
98     return 0;
99 }

```

C++

2. 博弈论

无上准则：对于当前操作的选手，必须要选择对于 ta 来说最优解

何为最优解？若进行某一步操作之后，劣势增大，则绝对不是最优解

常见最优解：1. 和上一位选手进行一样的操作 2. 通过这次操作，使得某些数据的总和为特定的数（比如这一次和上一次取数的总数为3，之类的） 3. 模仿上一位选手的操作，比如上一位选手选择最大/最小，那本次操作选当前集合的最大/最小

因为是博弈论，两者最优解**一定是相同的**

所以**操作具有同质化**

同时，很重要的一点，在博弈论中，数据有两种分类：**无效数据和有效数据**

如果某个数据对双方影响为0，即双方都有方法使得这个数据无影响，那这个数据就称为**无效数据**

反之，**有效数据一定对答案有贡献。那么只需要判断有效数据的最优选择方式即可**

总结1:

- 1. 双方最优解具有同质化特征，若推断出双方操作方法大不相同，则几乎可以肯定是错的
- 2. 计算最优解之前，一定要先判断什么数据是**无效数据**，即双方一定能通过某种操作达到平衡的数据

十一. 专题合集

专题1：线段+区间类问题

待做题单：

题号	标题	关键词	链接	完成情况
CF1300B	Assigning to Classes	排序+中位数	链接	
洛谷P1440	区间最小值	单调队列	链接	
洛谷P1886	滑动窗口	单调队列	链接	
CF1300C	Playlist	区间去重+滑动	链接	
洛谷P2082	区间覆盖	差分+扫描	链接	
CF427D	Match & Catch	区间交集+哈希	链接	
CF1296D	Fight with Monsters	贪心+堆	链接	
洛谷P1803	凌乱的yyy	贪心+堆	链接	
CF1311E	Construct the Binary Tree	区间构造	链接	

题号	标题	关键词	链接	完成情况
洛谷P2367	语文成绩	差分模板	链接	
CF276C	Little Girl and Maximum Sum	差分+贪心	链接	
洛谷P2671	区间求和	差分+前缀	链接	
洛谷P5490	区间面积并	离散+扫描线	链接	
CF652D	Nested Segments	逆序对+离散	链接	
洛谷P1908	逆序对	离散+树状数组	链接	
CF1679E	Typical Segment DP	区间 DP	链接	
洛谷P4170	涂色	区间 DP	链接	✓
CF1498E	Two Houses	区间贪心构造	链接	
洛谷P4553	八十天环游世界	区间覆盖建模流	链接	
CF1428F	Interesting Function	差分/数学	链接	

专题2： 括号序列·相关问题

前置知识（通用）

把左括号（看成 $+1$ ，右括号）看成 -1

那么括号序列 合法条件为： $f_i \geq 0$ 且 $f_n = 0$ (f_i 代表前 i 项的和)

专题3： DP 相关问题

树形 DP

例题1

地址：[A. AVL tree](#)

大致题意：二叉树上 $h_u = \max(h_{v1}, h_{v2}) + 1$ ，空节点 $h = 0$ 。每次能删一个叶子节点或者在空节点处添加一个节点，求所有点 $abs(h_{v1} - h_{v2}) \leq 1$ 的最小操作树

不管数据范围，先把转移方程写出来

$f_{u,i}$ 代表 u 节点的 $h_u = i$ 的时候最小的操作次数

显然 $f_{u,i} = \min(f_{v1,i-1} + f_{v2,i-1}, f_{v1,i-1} + f_{v2,i-2}, f_{v1,i-2} + f_{v2,i-1});$

对于空节点， $f_{0,i} = val_i$ ， val_i 代表构造一个 $h = i$ 的树最少要用多少节点

显然有 $h_i = h_{i-1} + h_{i-2} + 1$ 。又可以发现 i 不会超过26，所以可以暴力求解

十二. 进阶

12.1 异或相关

12.1.1 线性基

见[数学相关-线性基](#)

12.1.2 异或和方程

有 n 个未知数，同时有 n 个异或和方程

求解方法：按照高斯消元那样变成阶梯状，不过消去最高项的方法变成了 XOR

可以用`bitset`加速，某一组方程中，出现的系数为1，没出现的系数为0（注意是 $1 - base!$ ）

```
1 pair<bitset<N>, int> eq[N];
2 //bitset存的是每一组方程的系数
3 //假如第3组方程为: a1^a4^a5^an=w0
4 //那么eq[3]={010011...1}, w0}
5 //即eq[3]第i位代表a[i]是否出现过，注意是1-base!!
```

C++

```
1
2 struct Gauss_rk2 {
3     pair<bitset<N>, int> eq[N];
4     int rank = 0, vis[N], ans[N];
5
6     Gauss_rk2() { //初始化，构造函数
7         memset(vis, 0, sizeof(vis));
8         memset(ans, 0, sizeof(ans));
9     }
10    bool add(pair<bitset<N>, int> now) { //判断&添加新的方程
11        //方程即为: \XOR x[i]*now.first[i] = now.second
12        for(int i = 1; i <= n; i++) { //1-base
```

```

13         if(!now.first[i]) continue; //方程中第i位不为 1
14         if(!vis[i]) { //最高为1的在第i列的方程 不存在
15             rank++; //秩++
16             eq[i] = now; //添加到已存在方程中
17             vis[i] = true; //标记为true
18             return true; //成功添加一组线性无关的异或和方程
19         }
20         else {
21             now.first ^= eq[i].first;
22             now.second ^= eq[i].second;
23         }
24     }
25     return false; //能被构造出来, 线性相关, 返回false
26 }
27
28 bool check() { //判断是否满足求解条件
29     if(rank == n) return true; //已经满足求解条件, 即秩=n
30     return false;
31 }
32
33 void solve() { //求解出答案
34     for(int i = 1; i ≤ n; i++) {
35         for(int j = i + 1; j ≤ n; j++) {
36             if(eq[i].first[j] == 1) {
37                 eq[i].first ^= eq[j].first;
38                 eq[i].second ^= eq[j].second;
39             }
40         }
41         ans[i] = eq[i].second;
42     }
43 }
44
45 };
46
47 signed main() {
48     Gauss_rk2 a;
49     for(int i = 1; i ≤ n; i++) {
50         //输入一组方程, 变换为bitset
51         //a.add({bitset, int})
52     }
53     if(a.check()) a.solve();
54     return 0;
55 }

```

C++